31231027506_TranChiCong_DATN

by Công 31231027506 - Trần Chí

Submission date: 28-Oct-2026 08:40AM (UTC+0700)
Submission ID: 3072805875
File name: 32309_C_ng_31231027506_-_Tr_n_Ch__31231027506_TranChiCong_DATN_608_1211717143.pdf (6.58M)
Word count: 25980
Character count: 110110

ĐẠI HỌC KINH TẾ THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG CÔNG NGHỆ VÀ THIẾT KẾ UEH
KHOA CÔNG NGHỆ THÔNG TIN KINH DOANH

BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI

THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG POS ĐA CHI NHÁNH HỖ TRỢ HOẠT ĐỘNG NGOẠI TUYẾN VÀ QUẢN LÝ KHO NGUYÊN LIỆU CHO CHUỖI CÀ PHÊ KAFFEIN TRÊN NỀN TẢNG AWS

Sinh viên thực hiện:	Trần Chí Công
Mã số sinh viên:	31231027506
Lớp:	IT001
Khóa:	K49
Ngành:	Công nghệ thông tin
Giảng viên hướng dẫn:	TS. Trần Hồng Thái

Thành phố Hồ Chí Minh, ngày 28 tháng 10 năm 2026

Tóm tắt

KAFFEIN là nguyên mẫu hệ thống bán hàng cho một chuỗi cà phê có nhiều chi nhánh. Bài toán được giới hạn ở khả năng hoàn tất thanh toán tiền mặt khi POS mất kết nối, ghi nhận tiêu hao nguyên liệu theo công thức và đồng bộ dữ liệu về máy chủ. Mỗi chi nhánh có tối đa một POS Android đang hoạt động, giữ vai trò ghi nhận bán hàng và tiêu hao nguyên liệu tại quầy.

Ứng dụng POS được xây dựng bằng Kotlin, Jetpack Compose và Room. Giao diện đọc dữ liệu từ Room; khi thu tiền mặt, ứng dụng lưu hóa đơn, lượng tiêu hao, biến động tồn kho và operation đồng bộ trong cùng transaction. Máy chủ NestJS tiếp nhận operation theo định danh ổn định, kiểm tra thiết bị cùng nội dung yêu cầu và ghi dữ liệu vào PostgreSQL. Web quản trị dùng React và TypeScript. Luồng QR ZaloPay/MoMo Sandbox cần xác nhận trực tuyến; trạng thái chờ, đối soát, hủy và nhận tiền muộn được xử lý riêng. Quy trình kiểm đếm kín giữ bảng kê gốc và tách tiền bù khỏi doanh thu.

Phần đánh giá tập trung vào gửi lặp thao tác, bảo toàn dữ liệu giao dịch, xác nhận QR và quyền chi nhánh. Log ngày 23/10/2026 ghi nhận 164 bài kiểm thử đơn vị API, 63 bài tích hợp API và 11 bài Web đạt; hai bài tích hợp tùy chọn bị bỏ qua. Hồ sơ cùng đợt ghi nhận phát hành Web/API thành công lên AWS với ALB, một task ECS Fargate và RDS PostgreSQL Single-AZ. Nghiệm thu trọn luồng trên APK AWS và đánh giá với người dùng cửa hàng còn cần bổ sung. Khả năng ngoại tuyến hiện giới hạn ở nghiệp vụ tiền mặt sau khi xác thực đầu phiên; thanh toán điện tử sử dụng môi trường Sandbox.

Từ khóa: POS ngoại tuyến; đồng bộ dữ liệu; transactional outbox; kho nguyên liệu; thanh toán QR; AWS.

Mục lục

Danh mục hình và sơ đồ v
Danh mục bảng vii
Thuật ngữ và từ viết tắt viii
1 Tổng quan đề tài 1
1.1 Bài toán bán hàng và dữ liệu tại chi nhánh 1
1.2 Mục tiêu và câu hỏi nghiên cứu 1
1.3 Phạm vi và điều kiện áp dụng 2
1.4 Phương pháp thực hiện và nguồn dữ liệu 2
1.5 Cấu trúc báo cáo 3
2 Cơ sở lý thuyết và lựa chọn kỹ thuật 4
2.1 Dữ liệu cục bộ trong ứng dụng ngoại tuyến 4
2.2 Transaction và phạm vi bảo đảm nguyên tử 4
2.3 Transactional outbox và yêu cầu có thể gửi lặp 4
2.4 Nhất quán cuối cùng, bản chụp dữ liệu và điểm đồng bộ 5
2.5 Phiên bản công thức và sổ biến động kho 6
2.6 Ranh giới mô-đun và phạm vi giao dịch 6
2.7 Xác thực và quyền theo chi nhánh 6
2.8 Xác nhận thanh toán và trạng thái chưa kết luận 7
2.9 Hạ tầng như mã nguồn và giới hạn triển khai 7
3 Phân tích và thiết kế hệ thống 8
3.1 Bài toán vận hành và các ràng buộc thiết kế 8
3.2 Yêu cầu chức năng và tiêu chí kiểm tra 9
3.3 Tác nhân và đặc tả các trường hợp sử dụng 10
3.3.1 UC-01. Kích hoạt thiết bị tại chi nhánh 11
3.3.2 UC-02. Chấm công và đăng nhập thu ngân 12
3.3.3 UC-03. Hoàn tất bán hàng bằng tiền mặt 12
3.3.4 UC-04. Thanh toán đơn bằng QR 13
3.3.5 UC-05. Lập và đăng phiếu nhập kho 14
3.3.6 UC-06. Kiểm đếm và bàn giao quỹ 14
3.3.7 UC-07. Đồng bộ operation và nhận dữ liệu mới 15
3.3.8 Ma trận vai trò và phạm vi thao tác 15
3.4 Kiến trúc tổng thể và ranh giới dữ liệu 16
3.5 Thiết kế dữ liệu và lưu vết lịch sử 18
3.5.1 Tổ chức, nhân sự và thiết bị 18
3.5.2 Thực đơn, công thức và tiêu hao 18
3.5.3 Hóa đơn, thanh toán và quỹ tiền mặt 19
3.5.4 Từ điển dữ liệu của các ràng buộc trọng tâm 21
3.6 Ghi nhận tiền mặt và đồng bộ operation 22
3.7 Vòng đời QR và xử lý kết quả chưa xác định 26

3.8 Đăng kho và kiểm soát thay đổi từ Web 27
3.9 Đối soát phiên quỹ và nhận bàn giao 31
4 Hiện thực và triển khai hệ thống 33
4.1 Tổ chức mã nguồn và kiểm soát truy cập 33
4.2 Hoàn tất giao dịch tiền mặt trên POS 34
4.2.1 Chọn món và ghi giỏ hàng 34
4.2.2 Ghi nguyên tử sale và tiêu hao nguyên liệu 34
4.3 Đẩy outbox và cập nhật dữ liệu cục bộ 36
4.4 Thanh toán QR và phục hồi đối soát 37
4.5 Kiểm đếm và bàn giao quỹ 38
4.6 Web quản trị và dữ liệu theo chi nhánh 39
4.7 Nhập nguyên liệu và điều kiện cập nhật tồn 41
4.8 Triển khai AWS và phát hành ứng dụng 42
4.8.1 Vị trí thành phần và kết nối 42
4.8.2 Bí mật, giám sát và trình tự phát hành 44
5 Kiểm thử và đánh giá 47
5.1 Phạm vi dữ liệu và cách đánh giá 47
5.2 Môi trường và tổ chức kiểm thử 47
5.3 Kết quả kiểm thử tại mốc phát hành 48
5.4 Quan sát bổ sung từ bộ ảnh AWS 48
5.5 Đánh giá các cơ chế giữ nhất quán 49
5.5.1 Truy vết từ yêu cầu đến phép kiểm chứng 49
5.5.2 Bán tiền mặt, retry và snapshot kho 50
5.5.3 Thanh toán QR khi lỗi hoặc hủy 51
5.5.4 Chi nhánh, quyền thu ngân và kiểm đếm 51
5.6 Giới hạn đánh giá 51
6 Kết luận và hướng phát triển 53
6.1 Kết quả đạt được trong phạm vi đề tài 53
6.2 Các giới hạn còn lại 53
6.3 Hướng phát triển 54
Phụ lục A. Mô hình lớp và dữ liệu bổ sung 55
Phụ lục B. Danh mục nguồn đối chiếu 61
Phụ lục C. Danh mục API và hợp đồng trao đổi dữ liệu 62
C.1. Các route nghiệp vụ chính 62
C.2. Hợp đồng push, ACK và cursor 64
C.3. Định danh và kết quả thanh toán online 64
Phụ lục D. Quy trình tái lập và thu thập bằng chứng nghiệm thu 66
D.1. Điều kiện thực hiện và hồ sơ đầu vào 66
D.2. Cash ngoại tuyến và phục hồi: AC-01, AC-02 66
D.3. Gửi lặp, transaction và tồn kho: AC-03–AC-06 66
D.4. Attendance và quyền chi nhánh: AC-07, AC-08 67

D.5. QR trực tuyến và tiền về muộn: AC-09 67
D.6. Kiểm đếm kín và bàn giao quỹ bổ sung 67
D.7. Build, AWS smoke và kết quả: AC-10 67
Phụ lục E. Ảnh cấu hình và vận hành AWS 69
Tài liệu tham khảo 71

Danh mục hình và sơ đồ

Hình 1 Phân cấp chức năng của hệ thống quản lý chuỗi cà phê KAFFEIN. 9
Hình 2 Các tác nhân và trường hợp sử dụng tổng quát của KAFFEIN. 10
Hình 3 Các trường hợp sử dụng trên phân hệ Android POS. 11
Hình 4 Ngữ cảnh hệ thống, người sử dụng và các cổng thanh toán tích hợp. 17
Hình 5 Các thành phần Android POS, Web Admin, API và cơ sở dữ liệu theo ranh giới local/cloud. 17
Hình 6 Quan hệ dữ liệu của chi nhánh, nhân sự, thiết bị và phiên làm việc. 18
Hình 7 Cấu trúc sản phẩm, biến thể, topping và phiên bản công thức. 19
Hình 8 Quan hệ hóa đơn, thanh toán, tiêu hao và phiên quỹ tiền mặt. 20
Hình 9 Trình tự hoàn tất bán tiền mặt và ghi dữ liệu nguyên tử tại POS (phần 1/3) 23
Hình 10 Trình tự hoàn tất bán tiền mặt và ghi dữ liệu nguyên tử tại POS (phần 2/3) 24
Hình 11 Trình tự hoàn tất bán tiền mặt và ghi dữ liệu nguyên tử tại POS (phần 3/3) 25
Hình 12 Vòng đời operation trong Room khi gửi thành công, lỗi, retry hoặc bị từ chối. 26
Hình 13 Vòng đời attempt QR trên POS và đối soát biên lai. 27
Hình 14 Trình tự đăng phiếu nhập và cập nhật số dư kho về POS (phần 1/3) 28
Hình 15 Trình tự đăng phiếu nhập và cập nhật số dư kho về POS (phần 2/3) 29
Hình 16 Trình tự đăng phiếu nhập và cập nhật số dư kho về POS (phần 3/3) 30
Hình 17 Hoạt động kiểm đếm kín, ghi tiền bù và bàn giao quỹ sang phiên tiếp theo. 31
Hình 18 Màn hình chọn món trên POS khi giỏ hiện tại chưa có sản phẩm 34
Hình 19 Thanh toán tiền mặt cho đơn 49.000 đồng, tiền nhận 100.000 đồng và tiền thối 51.000 đồng 35
Hình 20 Biên lai tiền mặt trên POS sau khi hoàn tất đơn 49.000 đồng 36
Hình 21 Danh sách ba operation mở quỹ, bán tiền mặt và đóng quỹ đã được máy chủ xác nhận trên màn hình đồng bộ 37
Hình 22 Hộp thoại QR ZaloPay cho đơn 49.000 đồng ở trạng thái chờ xác nhận, có thao tác kiểm tra và yêu cầu hủy 38
Hình 23 Kiểm đếm quỹ theo mệnh giá trước khi nhập số tờ và tiếp tục đối soát 39
Hình 24 Tổng quan Web ở phạm vi toàn chuỗi khi chưa có đơn trong ngày và các POS hiển thị ngoại tuyến 40
Hình 25 Danh sách món và giá từng size trên Web; giá và VAT được cấu hình dùng chung trong chuỗi 40

Hình 26 Danh sách nhân viên ở quyền Chủ chuỗi, gồm chi nhánh chính, vai trò và thao tác điều chuyển 41
Hình 27 Số dư nguyên liệu của một chi nhánh và thông báo POS sẵn sàng cập nhật kho trên Web 42
Hình 28 Bố trí ALB, task Web/API và RDS trong cấu hình AWS của hệ thống 43
Hình 29 Trạng thái ECS service và hai container Web/API trong cùng task, revision 6 43
Hình 30 Listener HTTP/HTTPS của ALB và rule định tuyến theo host Web Admin .44
Hình 31 PostgreSQL 17.6 trên RDS Single-AZ, lưu trữ được mã hóa và giới hạn tự tăng 50 GiB 44
Hình 32 Quyền truy cập giữa các security group và quan hệ Secrets Manager, ECS, CloudWatch 45
Hình 33 Dashboard CloudWatch của ALB, ECS service và RDS trong cửa sổ ba giờ UTC 46
Hình 34 Lượt phát hành AWS thành công của commit 63f42cf trên nhánh desktop-work 46
Hình 35 Mô hình lớp miền của bán hàng, thanh toán và phiên quỹ 55
Hình 36 Mô hình lớp miền của sản phẩm, size, topping và công thức nguyên liệu . . 56
Hình 37 Các lớp triển khai phối hợp hoàn tất bán tiền mặt và ghi tiêu hao nguyên liệu 57
Hình 38 Các lớp triển khai lưu attempt QR, đối soát biên lai và phối hợp đồng bộ . . 58
Hình 39 Quan hệ dữ liệu của số dư, biến động nguyên liệu và chứng từ nhập kho . . 59
Hình 40 Quan hệ dữ liệu phục vụ đồng bộ, chống xử lý lặp và lưu dấu vết kiểm toán 60
Hình 41 Hai public subnet, hai DatabaseIsolated subnet và tuyến kết nối trong VPC KAFFEIN 69
Hình 42 RDS tại ap-southeast-1b, cổng PostgreSQL 5432 và cấu hình không truy cập công khai 69
Hình 43 Inbound rules của ECS cổng 3000/8080 và RDS cổng 5432; nguồn được đối chiếu với CDK 70
Hình 44 Hai target API cổng 3000 và Web cổng 8080 cùng hiển thị Healthy 70
Hình 45 Nhật ký API trên CloudWatch ghi nhận request GET / trả HTTP 404, với timestamp UTC 70

Danh mục bảng

Bảng 1 Giải thích thuật ngữ và từ viết tắt sử dụng trong báo cáo viii
Bảng 2 Câu hỏi nghiên cứu và hướng kiểm chứng của nguyên mẫu KAFFEIN 2
Bảng 3 Yêu cầu chức năng trọng tâm và điều kiện kiểm tra của MVP. 9
Bảng 4 Đặc tả trường hợp sử dụng kích hoạt POS 11
Bảng 5 Đặc tả chấm công và xác lập quyền thu ngân 12
Bảng 6 Đặc tả bán tiền mặt với ranh giới transaction cục bộ 13
Bảng 7 Đặc tả thanh toán QR và các kết quả cần đối soát 13
Bảng 8 Đặc tả nhận hàng và đăng kho tại chi nhánh 14
Bảng 9 Đặc tả kiểm đếm kín, bù quỹ và nhận bàn giao 15
Bảng 10 Đặc tả đồng bộ theo định danh ổn định và ACK 15
Bảng 11 Vai trò và phạm vi của các nghiệp vụ trọng tâm 16
Bảng 12 Các lựa chọn biểu diễn dữ liệu có ảnh hưởng đến giao dịch và đối soát. ... 20
Bảng 13 Từ điển dữ liệu nhân sự và nguồn thiết bị 21
Bảng 14 Từ điển dữ liệu bán hàng, công thức và sổ kho 21
Bảng 15 Từ điển dữ liệu đồng bộ, thanh toán và bàn giao 22
Bảng 16 Các điểm kiểm soát trong hiện thực API và POS 33
Bảng 17 Kết quả kiểm thử và phát hành ngày 23/10/2026, đối chiếu lại từ hồ sơ gốc. 48
Bảng 18 Trạng thái hạ tầng và phát hành được ghi nhận trong bộ ảnh tiếp nhận ngày 28/10/2026 49
Bảng 19 Truy vết yêu cầu, use case và phép kiểm chứng trong mã nguồn 49
Bảng 20 Đối chiếu tiêu chí ngoại tuyến và kho với bằng chứng kiểm thử hiện có. ... 50
Bảng 21 Đối chiếu tiêu chí phân quyền, QR và triển khai với giới hạn bằng chứng. .51
Bảng 22 Nguồn mã và tài liệu dùng đối chiếu nội dung báo cáo 61
Bảng 23 Các endpoint chính đối chiếu từ controller của phiên bản mã nguồn 62

Thuật ngữ và từ viết tắt

Bảng 1. Giải thích thuật ngữ và từ viết tắt sử dụng trong báo cáo

Thuật ngữ	**Nghĩa dùng trong báo cáo**
POS	Point of Sale; ứng dụng ghi nhận bán hàng tại quầy.
Room	Thư viện truy cập SQLite trên Android; nguồn dữ liệu mà giao diện POS đọc.
Outbox	Bảng lưu thao tác chờ đồng bộ, được ghi cùng transaction với nghiệp vụ cục bộ.
Operation	Một yêu cầu đồng bộ có định danh và payload ổn định qua các lần gửi.
Idempotency	Tính lũy đẳng: xử lý yêu cầu lặp mà không lặp lại tác động nghiệp vụ đã được chấp nhận.
ACK / FIFO	Phản hồi xác nhận tiếp nhận thao tác / thứ tự xử lý vào trước, ra trước.
Push / pull	Gửi thao tác cục bộ lên máy chủ / nhận dữ liệu từ máy chủ về POS.
Snapshot / checkpoint	Bản chụp dữ liệu tại một thời điểm / điểm đã tiếp nhận trong luồng đồng bộ.
Attempt	Lần thử thanh toán được lưu bền vững với mã giao dịch và trạng thái riêng.
BFD / BCE	Phân cấp chức năng / phân vai đối tượng giao diện, điều khiển và thực thể.
ERD	Mô hình thực thể và quan hệ dữ liệu.
RBAC	Phân quyền theo vai trò; được kết hợp với kiểm tra phạm vi chi nhánh.
ALB / RDS	Application Load Balancer / Relational Database Service của AWS.
Single-AZ	Cơ sở dữ liệu đặt tại một vùng sẵn sàng; không có cấu hình standby Multi-AZ trong nguyên mẫu.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1 Bài toán bán hàng và dữ liệu tại chi nhánh

Một giao dịch tại quầy tạo ra nhiều thay đổi có liên hệ: món khách gọi chuyển thành hóa đơn, tiền nhận được ghi vào phiên quỹ, nguyên liệu bị tiêu hao và doanh thu được tổng hợp theo chi nhánh. Khi POS không truy cập được máy chủ, cửa hàng vẫn có thể nhận tiền mặt. Vì vậy, thiết kế phải xác định được dữ liệu nào được phép ghi tại máy, dữ liệu nào phải đợi xác nhận trực tuyến và cách phục hồi khi thông điệp đồng bộ bị gửi lại.

Nếu chỉ lưu hóa đơn rồi mới ghi yêu cầu đồng bộ ở bước kế tiếp, ứng dụng có thể dừng giữa hai bước. Hóa đơn khi đó tồn tại tại quầy nhưng chưa có thông tin để máy chủ tiếp nhận. Trường hợp ngược lại cũng cần xét: máy chủ đã ghi giao dịch nhưng phản hồi không tới POS, khiến thiết bị không biết lần gửi trước đã thành công hay chưa. Đây là hai tình huống lỗi khác nhau; giao thức đồng bộ phải xử lý được cả tính nguyên tử ở nguồn và yêu cầu lặp ở đích.

Một món bán có thể sử dụng nhiều nguyên liệu; mỗi kích cỡ và lựa chọn topping làm thay đổi lượng tiêu hao. Nếu công thức được sửa sau khi bán, giao dịch cũ vẫn phải giữ được lượng nguyên liệu đã áp dụng tại thời điểm hoàn tất. Hệ thống vì vậy lưu phiên bản công thức, bản chụp tiêu hao và sổ biến động liên quan tới hóa đơn, bên cạnh số dư kho.[1]

KAFFEIN được đặt trong mô hình một doanh nghiệp quản lý nhiều cửa hàng. Chủ chuỗi cần nhìn toàn bộ hoạt động; quản lý cửa hàng chỉ thao tác trong chi nhánh được phân công. Nhân viên có thể chấm công ở chi nhánh khác theo quy tắc cho phép, nhưng quyền mở phiên bán hàng gắn với chi nhánh chính. Việc phân biệt các ngữ cảnh này là điều kiện để xác định trách nhiệm của giao dịch và không thể thay bằng việc ẩn nút trên giao diện.

1.2 Mục tiêu và câu hỏi nghiên cứu

Mục tiêu của đề tài là xây dựng nguyên mẫu POS Android và hệ thống quản trị tập trung, trong đó giao dịch tiền mặt đủ điều kiện vẫn hoàn tất được khi mất mạng. Dữ liệu sau đó được đồng bộ về máy chủ mà không tạo thêm hóa đơn hoặc biến động kho cho cùng một operation. Nguyên mẫu cũng cần giữ được dấu vết công thức, phân quyền theo chi nhánh và triển khai Web/API trên AWS bằng cấu hình có thể lặp lại.

Năm câu hỏi nghiên cứu trong đề cương được cụ thể hóa thành các kịch bản kiểm tra ở Bảng 2. Kết quả được đối chiếu qua dữ liệu lưu trữ, phản hồi API và trạng thái đồng bộ của POS.[2]

[1] Nguồn đối chiếu: docs/00-project-charter.md; docs/01-scope.md; apps/api/prisma/schema.prisma

[2] Nguồn đối chiếu: docs/09-proposal.md, mục 1.5

Bảng 2. Câu hỏi nghiên cứu và hướng kiểm chứng của nguyên mẫu KAFFEIN

Mã	Câu hỏi nghiên cứu	Hướng kiểm chứng
RQ1	Tổ chức POS và dữ liệu cục bộ thế nào để tiếp tục bán tiền mặt khi mất kết nối?	Đơn đủ điều kiện hoàn tất tại Room; giao diện đọc được receipt và số dư mới.
RQ2	Outbox, operation ID và checkpoint phối hợp ra sao để retry không lặp tác động nghiệp vụ?	Cùng ID/payload được gửi lại; máy chủ chỉ ghi một kết quả và POS nhận ACK hợp lệ.
RQ3	Phiên bản công thức, snapshot và movement duy trì quan hệ giữa bán hàng và tồn kho thế nào?	Đối chiếu lượng tiêu hao, recipe version và số dư trước/sau sale.
RQ4	Tách thiết bị, attendance, phiên thu ngân và store scope kiểm soát truy cập trái chi nhánh thế nào?	Chấm công khác chi nhánh và từ chối đăng nhập bán hàng ngoài home store trong các kịch bản xác định.
RQ5	Kiến trúc AWS và quy trình phát hành nào phù hợp với nguyên mẫu?	Đối chiếu HTTPS, mạng, secret, migration, log và giới hạn tài nguyên với CDK/runbook.

1.3 Phạm vi và điều kiện áp dụng

Hệ thống phục vụ một chuỗi cà phê duy nhất, có bán món điểm tâm và có nhiều chi nhánh. Mỗi chi nhánh chỉ có một POS active. Đơn treo và giỏ hàng được giữ cục bộ trên thiết bị; Web không tạo thêm đơn bán từ một kênh đặt hàng trực tuyến. Các điều kiện này giới hạn số nơi cùng ghi nhận tiêu hao nguyên liệu tại một cửa hàng, nhờ đó có thể phân tích tồn kho cục bộ trong phạm vi MVP.[3]

Khả năng ngoại tuyến áp dụng cho luồng bán tiền mặt sau khi POS đã kích hoạt, tải dữ liệu và có ngữ cảnh thu ngân, phiên quỹ hợp lệ. Kích hoạt thiết bị, chấm công và xác thực đầu phiên hiện cần truy cập máy chủ. QR ZaloPay/MoMo được tích hợp ở môi trường Sandbox và cần xác nhận online. Việc POS còn hiển thị giỏ hàng hoặc mã QR không được dùng làm bằng chứng khách đã thanh toán.

Đồ án giữ hóa đơn đã trả tiền ở trạng thái bất biến. Các nghiệp vụ hoàn tiền, sửa hóa đơn đã thanh toán, lô/hạn sử dụng, chuyển kho liên chi nhánh, tính lương và hóa đơn điện tử pháp lý nằm ngoài phạm vi. Phiếu bán hàng/PDF trong ứng dụng là chứng từ nội bộ của nguyên mẫu. Những giới hạn này tác động trực tiếp tới mô hình dữ liệu và cách đọc kết quả đánh giá, đặc biệt khi phân biệt quản lý tồn lý thuyết với kiểm kê nguyên liệu thực tế.[4]

1.4 Phương pháp thực hiện và nguồn dữ liệu

Mỗi chức năng được triển khai từ yêu cầu nghiệp vụ đến mô hình dữ liệu, API, giao diện và kiểm thử. Các quyết định về phạm vi và kiến trúc được lưu trong ADR. Mã nguồn xác định cơ chế đã hiện thực; nhật ký kiểm thử và tài liệu bàn giao cung cấp thông tin về môi trường, thời điểm và kết quả chạy.

Phần phân tích sử dụng sơ đồ phân cấp chức năng, use case, mô hình lớp, ERD, sơ đồ hoạt động, tuần tự và trạng thái. Các sơ đồ làm rõ phạm vi quyền, quan hệ dữ liệu, thứ tự tương tác và cách giữ giao dịch khi xảy ra lỗi.

[3]Nguồn đối chiếu: docs/adr/0001-single-chain-multi-branch.md; docs/adr/0002-one-active-pos-per-store.md

[4]Nguồn đối chiếu: docs/01-scope.md; docs/02-business-rules.md

Các tình huống vận hành được xây dựng cho nguyên mẫu; đề tài chưa có dữ liệu khảo sát người dùng tại cửa hàng. Phần đánh giá do đó giới hạn ở hành vi hệ thống và các kịch bản kỹ thuật, chưa đánh giá mức độ hài lòng hoặc hiệu quả vận hành thực tế.

Một chức năng có trong mã nguồn được xem là đã hiện thực. Kết luận về việc chạy thành công cần có kết quả kiểm thử kèm môi trường và phạm vi xác định. Các kết quả được sử dụng trong báo cáo phải có hồ sơ đối chiếu; những phép đo và kịch bản nghiệm thu chưa thực hiện được ghi riêng ở chương kiểm thử.

1.5 Cấu trúc báo cáo

Chương 1: Tổng quan đề tài.
Chương 2: Cơ sở lý thuyết và lựa chọn kỹ thuật.
Chương 3: Phân tích và thiết kế hệ thống.
Chương 4: Hiện thực và triển khai hệ thống.
Chương 5: Kiểm thử và đánh giá.
Chương 6: Kết luận và hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ LỰA CHỌN KỸ THUẬT

2.1 Dữ liệu cục bộ trong ứng dụng ngoại tuyến

Trong ứng dụng offline-first, một tập chức năng cốt lõi vẫn làm việc được khi mạng không sẵn sàng. Tầng giao diện đọc một nguồn dữ liệu cục bộ thống nhất; dữ liệu mạng được đưa về nguồn này thay vì tạo thêm một nhánh dữ liệu riêng cho màn hình. Tài liệu Android mô tả cách tổ chức đó ở tầng dữ liệu. [1]

KAFFEIN áp dụng cách tổ chức này cho danh mục, giỏ hàng, hóa đơn và tồn kho tại POS. Room tiếp tục phục vụ những thao tác được phép khi mất mạng. Khi kết nối trở lại, tiến trình đồng bộ cập nhật Room; giao diện nhận dữ liệu mới qua nguồn đang quan sát.

Room cung cấp lớp truy cập trên SQLite và hỗ trợ kiểm tra truy vấn khi biên dịch. Entity, DAO và database được khai báo trong ứng dụng, còn quy tắc bán hàng vẫn thuộc repository và dịch vụ nghiệp vụ. Room không tự tạo một giao thức hòa giải với máy chủ; việc quyết định khi nào push, khi nào pull và cách giữ operation lỗi phải được lập trình riêng. [2]

Sự khác biệt giữa dữ liệu hiển thị và dữ liệu tập trung cần được giữ rõ. Room là nguồn giao diện POS đọc, còn PostgreSQL là nơi tổng hợp và kiểm tra dữ liệu toàn chuỗi. Trong thời gian mất mạng, hai nơi có thể quan sát các phiên bản khác nhau. Trạng thái đã thu tiền tại quầy và trạng thái đã được máy chủ tiếp nhận vì thế phải được biểu diễn tách biệt.

2.2 Transaction và phạm vi bảo đảm nguyên tử

Một transaction xác định nhóm thay đổi cùng được commit hoặc rollback. Trong luồng bán tiền mặt, nhóm này gồm hóa đơn, chi tiết món, lượng nguyên liệu tiêu hao, biến động tồn kho và thao tác chờ đồng bộ. Nếu nguyên liệu không đủ hoặc một lệnh ghi thất bại, toàn bộ thay đổi phải được hoàn tác.

Tính nguyên tử của SQLite được xây dựng trên giao thức commit và phục hồi của cơ sở dữ liệu, với các giả định về khả năng ghi và đồng bộ của hệ thống lưu trữ. [3] Ở tầng ứng dụng, các lệnh ghi liên quan phải nằm trong cùng transaction Room; phép kiểm thử gây lỗi giữa các bước ghi được dùng để kiểm tra rollback.

Transaction tại POS và transaction trên PostgreSQL là hai phạm vi riêng. POS có thể commit khi backend chưa truy cập được. Vì vậy không có một transaction phân tán bao trùm cả hai nơi. Thiết kế phải giữ một dấu vết bền vững để việc chuyển dữ liệu qua khoảng cách mạng được tiếp tục sau đó. Chính yêu cầu này dẫn tới transactional outbox.

2.3 Transactional outbox và yêu cầu có thể gửi lặp

Outbox lưu thông điệp cần gửi trong cùng transaction với thay đổi nghiệp vụ ở nguồn. Worker chỉ đọc những bản ghi đã commit và gửi tới bên nhận. Nếu tiến trình dừng sau commit, bản ghi chờ vẫn có thể được đọc lại khi ứng dụng hoạt động. Mẫu thiết kế này giải quyết việc ghi dữ liệu và thông điệp ở hai bước rời nhau; phía nhận vẫn phải xử lý khả năng giao nhận lặp. [4]

Mỗi thao tác đồng bộ trong KAFFEIN giữ nguyên mã định danh và nội dung qua các lần gửi lại. Máy chủ kiểm tra mã đã xử lý, thiết bị nguồn và hàm băm nội dung yêu cầu. Yêu cầu dùng lại một mã nhưng đổi thiết bị hoặc nội dung bị từ chối.[5]

Tình huống đáng chú ý xảy ra khi máy chủ đã commit nhưng ACK bị mất. Lần gửi lại phải trả kết quả đã có mà không ghi hóa đơn hoặc stock movement lần thứ hai. Ngược lại, nếu máy chủ chưa commit, operation vẫn cần được xử lý. Điều kiện nghiệm thu đúng là tác động nghiệp vụ được ghi một lần trong các kịch bản kiểm tra; số lần gửi qua mạng có thể lớn hơn một. Phân biệt hai đại lượng này tránh diễn giải sai cơ chế retry thành cơ chế giao nhận chỉ một lần. [5]

Các thao tác trong hàng đợi có thể phụ thuộc nhau. Chẳng hạn, thao tác đóng quỹ cần được xử lý sau những giao dịch tiền mặt thuộc phiên đó. POS gửi theo thứ tự đưa vào hàng đợi và dừng khi gặp bản ghi REJECTED. Bản ghi bị từ chối được giữ để đối soát; các thao tác phía sau chỉ tiếp tục khi vướng mắc được giải quyết.

2.4 Nhất quán cuối cùng, bản chụp dữ liệu và điểm đồng bộ

Nhất quán cuối cùng (eventual consistency) cho phép các bản sao tạm khác nhau khi chưa trao đổi hết cập nhật. Khái niệm này không tự quy định cách xử lý xung đột, thứ tự sự kiện hay thời điểm một số dư được xem là có thể sử dụng. [5] Trong nguyên mẫu, các quy tắc push trước, pull khi outbox sạch và lưu số dư cùng cursor tạo thành giao thức cụ thể cho quá trình hội tụ.

Checkpoint xác định phần dữ liệu máy chủ mà POS đã tiếp nhận. Nếu cập nhật số dư và cursor ở hai transaction, thiết bị có thể ghi cursor mới mà chưa ghi đủ số dư, hoặc ghi số dư rồi tiếp nhận lại cùng trang dữ liệu. Vì vậy, áp dụng một trang pull phải cập nhật dữ liệu và cursor cùng nhau. Sau ACK bán tiền mặt, nguyên mẫu đưa cursor về 0 để nhận snapshot kho và đối chiếu lại số dư từ máy chủ.[6]

Một ràng buộc khác là snapshot đang tải không được ghi đè các thay đổi local mới tạo. Kiểm tra outbox trước khi gửi request chưa đủ nếu trong lúc chờ mạng lại phát sinh operation. Bởi vậy, việc kiểm tra lại trước khi áp dụng và phối hợp bằng khóa cục bộ là phần của giao thức. Dẫu vậy, heartbeat sạch từ POS vẫn chỉ là thông tin gần nhất; nó không khóa được một transaction Room đang bắt đầu sau khi heartbeat gửi đi.

Tương tự, một sequence tăng dần trong PostgreSQL không đồng nghĩa transaction có số nhỏ luôn commit trước transaction có số lớn. Nếu dùng số đó làm cursor, cần xét trường hợp writer chậm commit sau khi phía đọc đã tiến cursor. Tài liệu dự án ghi nhận giới hạn này; snapshot đầy đủ là đường đối chiếu, còn bảo đảm delta dưới mọi thứ tự commit cần thêm giao thức công bố thay đổi và kiểm thử cạnh tranh tương ứng.[7]

[5] Nguồn đối chiếu: apps/api/src/sync/sync.service.ts; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/sync/OutboxSyncRepository.kt

[6] Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/sync/CatalogSyncRepository.kt; docs/04-offline-sync.md

[7] Nguồn đối chiếu: docs/04-offline-sync.md, các giới hạn heartbeat và commit-order

2.5 Phiên bản công thức và sổ biến động kho

Số dư nguyên liệu cho biết lượng còn lại theo hệ thống, nhưng không giải thích được lịch sử tạo nên số dư đó. Stock movement bổ sung mỗi lần tăng hoặc giảm gắn với nguyên nhân nghiệp vụ. Với một nguyên liệu, quan hệ đối chiếu cơ bản là số dư cuối bằng số dư đầu cộng tổng thay đổi hợp lệ trong kỳ. Quan hệ này chỉ có ý nghĩa khi các movement trong phạm vi được xác định đầy đủ và dùng cùng đơn vị đo.

Khi một món có công thức phiên bản 1 sử dụng một lượng nguyên liệu, chỉnh công thức sang phiên bản 2 không được làm thay đổi lượng tiêu hao của hóa đơn đã trả tiền. Snapshot lưu lại lựa chọn đã áp dụng cho sale, bao gồm phiên bản và lượng tiêu hao tương ứng. Sự tách biệt giữa dữ liệu hiện tại và dữ liệu lịch sử cũng được dùng cho đơn giá, VAT và topping. Nhờ đó, việc tra cứu hóa đơn không phải tính lại bằng danh mục mới.[8]

Kho theo công thức vẫn là tồn lý thuyết. Sai số pha chế, hao hụt và kiểm kê thực tế có thể tạo chênh lệch ngoài các giao dịch bán. Báo cáo chỉ đánh giá việc hệ thống lưu và áp dụng lượng tiêu hao đã được mô hình hóa. Không có dữ liệu cân đo tại cửa hàng để kết luận rằng số dư đó luôn bằng lượng nguyên liệu vật lý còn lại.

2.6 Ranh giới mô-đun và phạm vi giao dịch

Backend được chia thành mô-đun nghiệp vụ nhưng phát hành như một ứng dụng NestJS. Với phạm vi cá nhân và thời gian thực hiện giới hạn, cách tổ chức này cho phép nghiệp vụ sale, inventory và audit phối hợp trong một PostgreSQL transaction. Chia thành nhiều dịch vụ triển khai riêng sẽ buộc thiết kế thêm giao thức nhất quán giữa các dịch vụ trước khi lợi ích về vận hành độc lập được sử dụng.[9]

Mô hình miền phân biệt các đối tượng theo ý nghĩa và quy tắc nghiệp vụ. [6] Trong KAFFEIN, mô-đun thanh toán tiếp nhận và kiểm tra kết quả từ cổng; mô-đun bán hàng ghi nhận hóa đơn và tiêu hao; mô-đun đồng bộ xử lý thao tác gửi lại từ POS. Chấm công, phiên thu ngân và phiên quỹ được mô hình hóa riêng vì lần lượt ghi nhận sự hiện diện, quyền bán hàng và trách nhiệm quản lý tiền mặt.[10]

2.7 Xác thực và quyền theo chi nhánh

Xác thực xác định tài khoản hoặc thiết bị gửi yêu cầu. Phân quyền kiểm tra chủ thể đó có được thực hiện thao tác trên dữ liệu của chi nhánh liên quan hay không. Máy chủ suy ra hoặc kiểm tra phạm vi chi nhánh từ thông tin đã xác thực; giá trị `storeId` trong yêu cầu chỉ được sử dụng sau bước kiểm tra này.[11]

Thông tin xác thực POS được cấp khi kích hoạt thiết bị và gắn với một cửa hàng. Chấm công ghi nhận sự hiện diện của nhân viên; phiên thu ngân xác định người bán hàng; phiên quỹ ghi nhận trách nhiệm tiền mặt. Việc tách các ngữ cảnh cho phép nhân viên chấm công ở chi nhánh hỗ trợ nhưng vẫn chỉ đăng nhập bán hàng tại chi nhánh chính.

---

[8] Nguồn đối chiếu: apps/api/prisma/schema.prisma; apps/api/src/sales/sales.service.ts; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/InventorySalePlanner.kt

[9] Nguồn đối chiếu: docs/adr/0003-modular-monolith.md; apps/api/src/app.module.ts

[10] Nguồn đối chiếu: apps/api/src/app.module.ts; apps/api/src/payments/; apps/api/src/sales/; apps/api/src/sync/

[11] Nguồn đối chiếu: apps/api/src/common/security/; docs/02-business-rules.md

2.8 Xác nhận thanh toán và trạng thái chưa kết luận

Khi yêu cầu QR hết thời gian chờ, POS chưa đủ thông tin để xác định khách đã thanh toán hay chưa. Bộ đếm trên màn hình hết hạn chỉ làm dừng hiển thị QR; giao dịch tiếp tục được đối soát. Với MoMo, thu ngân có thể yêu cầu máy chủ hủy nội bộ để tiếp tục thao tác. QR cũ có thể vẫn nhận tiền, nên hệ thống giữ giao dịch đã hủy và theo dõi khoản nhận tiền muộn theo ADR 0010.[12]

Với giao dịch chưa bị hủy, khi nhận xác nhận hợp lệ từ cổng thanh toán, máy chủ lưu xác nhận trước khi hoàn tất hóa đơn và tồn kho. POS lưu biên lai, tải lại số dư kho từ máy chủ rồi xóa các dòng đã thanh toán khỏi giỏ. Nếu một bước chưa hoàn tất, trạng thái đối soát được lưu để tiếp tục khi ứng dụng hoạt động trở lại. Khoản nhận tiền muộn của giao dịch MoMo đã hủy được xử lý riêng, không hoàn tất lại hóa đơn cũ.

2.9 Hạ tầng như mã nguồn và giới hạn triển khai

Infrastructure as Code mô tả tài nguyên và quan hệ cấu hình bằng mã nguồn. Trong KAFFEIN, CDK lưu cấu hình triển khai cùng dự án để kiểm tra thay đổi trước khi phát hành.[13] Kiểm tra cấu hình được bổ sung bằng kiểm tra trạng thái dịch vụ, kết nối và hành vi ứng dụng sau triển khai. Các thuộc tính chất lượng được đánh giá theo tình huống kiểm tra và cấu hình cụ thể. [7]

Web và API chạy chung một task ECS Fargate; PostgreSQL chạy trên RDS Single-AZ. Cách bố trí này giảm số đơn vị triển khai nhưng gắn vòng đời Web và API với nhau. Task sử dụng địa chỉ IP công cộng cho kết nối ra ngoài; security group giới hạn kết nối vào. RDS chưa có standby Multi-AZ. Hạ tầng không sử dụng NAT Gateway, nhưng chi phí vẫn gồm ALB, IPv4, lưu trữ, secret và thời gian tài nguyên hoạt động.[14]

[12]Nguồn đối chiếu: apps/api/src/payments/payments.service.ts; docs/adr/0010-momo-local-cancellation.md

[13]Nguồn đối chiếu: infra/lib/kaffein-stack.ts; infra/test/; docs/adr/0012-aws-demo-cdk.md

[14]Nguồn đối chiếu: infra/lib/kaffein-stack.ts; docs/adr/0012-aws-demo-cdk.md; docs/adr/0013-web-admin-ecs-alb.md

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Bài toán vận hành và các ràng buộc thiết kế

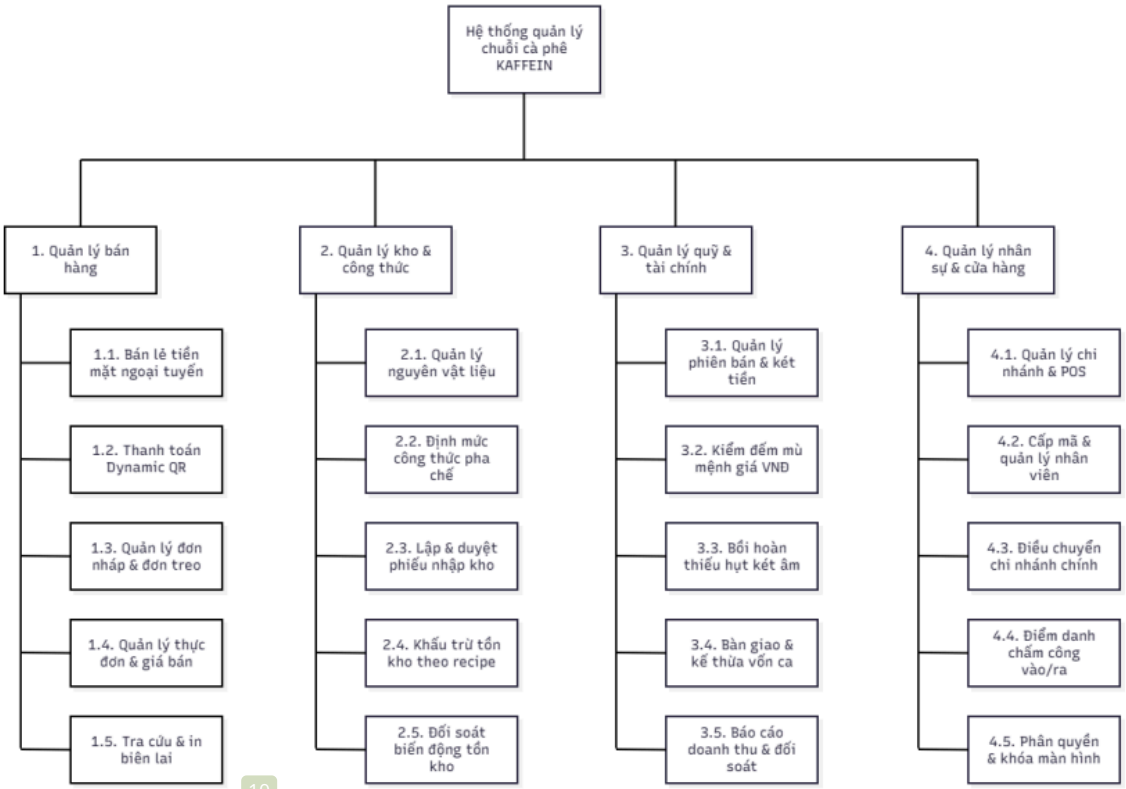
KAFFEIN được mô hình hóa là một chuỗi cà phê có nhiều chi nhánh. Tại quầy, nhân viên cần lập đơn, chọn kích cỡ và topping, nhận tiền, đồng thời để lại dữ liệu đủ cho việc đối chiếu doanh thu với nguyên liệu đã sử dụng. Khi kết nối bị gián đoạn, thời điểm thu tiền tại POS và thời điểm máy chủ tiếp nhận giao dịch có thể khác nhau. Thiết kế vì vậy phải xác định nơi được phép ghi nhận từng nghiệp vụ và cách giữ nguyên ý nghĩa của giao dịch trong khoảng chờ đồng bộ.

Mỗi chi nhánh có tối đa một POS đang hoạt động. Ràng buộc này giảm số bên có thể cùng bán trên một số dư kho cục bộ, nhưng vẫn giữ định danh và lịch sử thiết bị để xác thực, thu hồi quyền và truy vết thao tác. Hệ thống phục vụ một doanh nghiệp; dữ liệu phân tách theo chi nhánh, không có chức năng đăng ký doanh nghiệp thuê bao hoặc cột `tenant_id`. Hai quyết định này xác lập phạm vi cho các thiết kế ngoại tuyến ở phần sau.[15]

Quyền quản trị và quyền vận hành quầy được phân biệt. Chủ chuỗi quản lý phạm vi toàn chuỗi, còn quản lý cửa hàng thao tác trong chi nhánh được cấp quyền. Nhân viên có một chi nhánh chính. Một nhân viên hỗ trợ có thể chấm công tại chi nhánh khác được phép, nhưng chỉ đăng nhập thu ngân tại chi nhánh chính. Mã chi nhánh trong yêu cầu không tự tạo quyền; máy chủ phải suy ra hoặc kiểm tra phạm vi từ tài khoản đã xác thực và credential của POS.

Các chức năng được phân rã trong Hình 1. Nhóm bán hàng tại quầy có quan hệ trực tiếp với tồn kho và phiên tiền mặt; nhóm quản trị cung cấp dữ liệu và theo dõi kết quả. Nhóm đồng bộ nối hai phía khi mạng phục hồi. Cách phân rã này xác định trách nhiệm nghiệp vụ, chưa phải cấu trúc các lớp triển khai.

[15] Nguồn đối chiếu: docs/00-project-charter.md; docs/01-scope.md; docs/adr/0001-single-chain-multi-branch.md; docs/adr/0002-one-active-pos-per-store.md



Hình 1. Phân cấp chức năng của hệ thống quản lý chuỗi cà phê KAFFEIN.

3.2 Yêu cầu chức năng và tiêu chí kiểm tra

Các yêu cầu trong Bảng 3 được chọn từ phạm vi MVP và các đường xử lý đã có trong dự án. Mỗi yêu cầu có điều kiện kiểm tra về thay đổi dữ liệu, kết quả tiếp nhận hoặc từ chối thao tác.

Bảng 3. Yêu cầu chức năng trọng tâm và điều kiện kiểm tra của MVP.

Mã	Yêu cầu	Điều kiện cần kiểm tra
YC-01	Kích hoạt POS theo chi nhánh	Mã kích hoạt hợp lệ cấp định danh và credential; chi nhánh không có hai thiết bị ACTIVE.
YC-02	Phân biệt chấm công và thu ngân	Chấm công hỗ trợ không tạo quyền đăng nhập thu ngân tại chi nhánh khác.
YC-03	Hoàn tất bán tiền mặt khi mất mạng	Một transaction Room ghi đủ hóa đơn, tiêu hao, biến động kho và operation chờ đồng bộ.
YC-04	Gửi lại không ghi sổ hai lần	Cùng operationId và payload nhận kết quả cũ; khác thiết bị hoặc payload bị từ chối.
YC-05	Xác nhận thanh toán QR trực tuyến	Chỉ ghi nhận đã trả sau xác nhận hợp lệ của cổng thanh toán và hoàn tất ghi sổ tại server.
YC-06	Đăng phiếu nhập có kiểm soát	Nháp chưa đổi tồn; đăng kho kiểm tra POS, outbox, QR đang chờ và ghi biến động nguyên tử.
YC-07	Giữ dấu vết kiểm đếm và bàn giao	Thực đếm ban đầu, chênh lệch và tiền bù được lưu riêng; quỹ nguồn chỉ được nhận một lần.

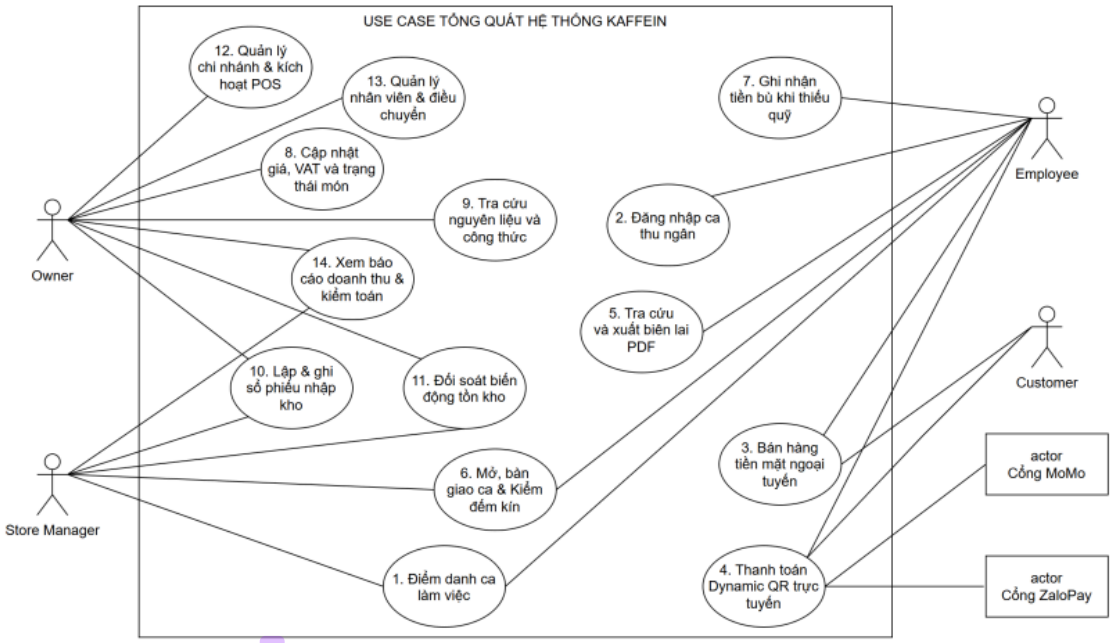
Yêu cầu về dữ liệu tập trung vào khả năng khôi phục và truy vết. Hóa đơn đã thanh toán giữ nguyên lịch sử giá, thuế và tiêu hao. Thao tác chưa gửi thành công phải còn

trong Room sau khi đóng ứng dụng. Khi cùng một yêu cầu được gửi lại, kết quả không được phụ thuộc vào việc người dùng đã thay đổi giá niêm yết hoặc công thức sau lần bán trước. Các ràng buộc này dẫn tới việc sử dụng snapshot và định danh ổn định cho giao dịch.

Phạm vi ngoại tuyến cũng cần xác định theo từng thao tác. Bán tiền mặt được tiếp tục sau khi POS đã tải dữ liệu, có ngữ cảnh thu ngân và phiên quỹ hợp lệ. Chấm công, đăng nhập lại và xác thực đầu nghiệp vụ kiểm đếm vẫn cần kết nối trong phiên bản hiện tại. Thanh toán ZaloPay và MoMo sử dụng môi trường Sandbox, phụ thuộc vào máy chủ và xác nhận trực tuyến. Hoàn tiền, chuyển kho liên chi nhánh và hóa đơn điện tử pháp lý nằm ngoài MVP.[16]

3.3 Tác nhân và đặc tả các trường hợp sử dụng

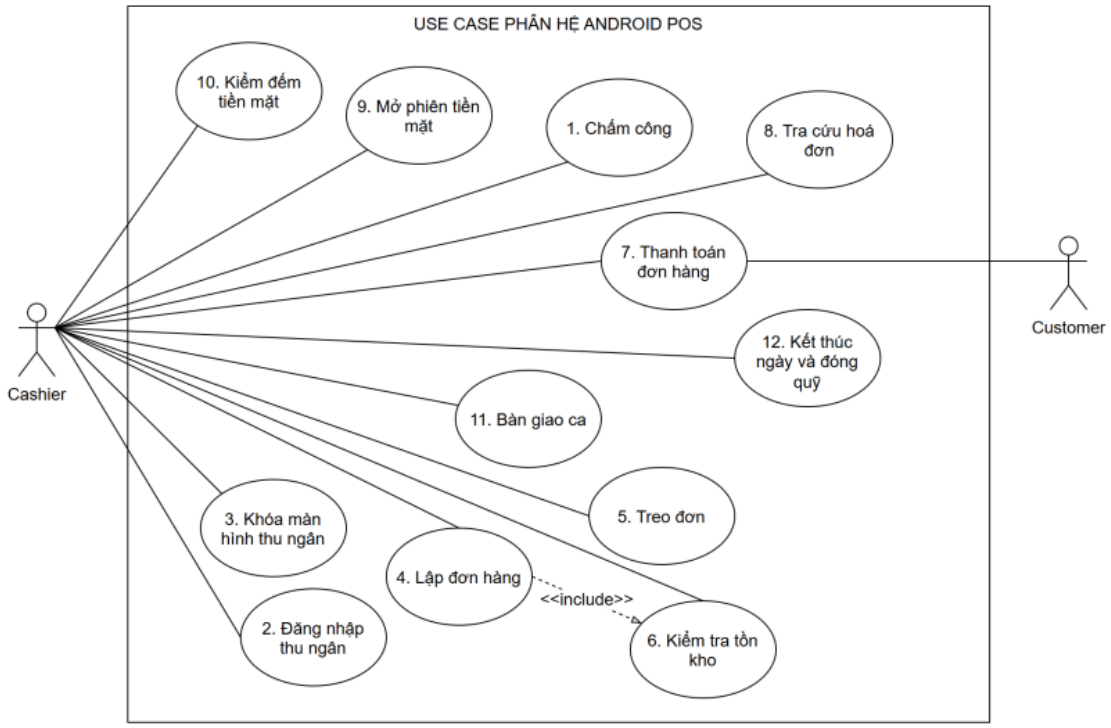
Sơ đồ tổng quát ở Hình 2 thể hiện quan hệ giữa người dùng quản trị, nhân viên quầy và hệ thống thanh toán. Thiết bị đã kích hoạt còn là chủ thể kỹ thuật khi gọi API: credential gắn yêu cầu với chi nhánh, còn thông tin thu ngân xác định người thực hiện thao tác tại quầy. Khách hàng thanh toán thông qua nhân viên hoặc ứng dụng ví, không có ứng dụng đặt món trực tuyến trong phạm vi này.



Hình 2. Các tác nhân và trường hợp sử dụng tổng quát của KAFFEIN.

Việc kích hoạt thiết bị, chấm công, đăng nhập thu ngân và mở phiên tiền mặt có mục đích riêng. Kích hoạt thiết lập quan hệ POS với chi nhánh; chấm công ghi nhận hiện diện; phiên thu ngân xác lập người được vận hành quầy; phiên tiền mặt giữ số vốn và đối soát két. Vì các vòng đời khác nhau, khóa phiên thu ngân do không hoạt động không xóa enrollment hay bản chấm công. Hình 3 trình bày các thao tác tại POS theo những trách nhiệm này.

[16] Nguồn đối chiếu: docs/01-scope.md; docs/02-business-rules.md; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/CashSaleRepository.kt; data/local/OnlinePaymentRepository.kt



Hình 3. Các trường hợp sử dụng trên phân hệ Android POS.

3.3.1 UC-01. Kích hoạt thiết bị tại chi nhánh

Owner tạo mã kích hoạt cho chi nhánh; người cài đặt nhập mã trên POS khi thiết bị có kết nối. Máy chủ kiểm tra mã, thời hạn, trạng thái cửa hàng và thiết bị đang hoạt động trước khi cấp credential. Credential xác định nguồn yêu cầu trong các nghiệp vụ tiếp theo; mã nhân viên không thay thế credential thiết bị. Thay thiết bị là một trường hợp có chủ đích riêng, không phải cách kích hoạt thêm POS thứ hai tại cùng chi nhánh.

Bảng 4. Đặc tả trường hợp sử dụng kích hoạt POS

Thành phần	Đặc tả UC-01
Tác nhân, kích hoạt	Owner cấp mã; người cài đặt chọn kích hoạt POS.
Tiền điều kiện	POS có mạng; mã do backend đích cấp; chi nhánh tồn tại và cho phép kích hoạt.
Luồng chính	1. Tạo mã gắn chi nhánh. 2. POS gửi mã và thông tin thiết bị. 3. Server kiểm tra, ghi enrollment và trả credential. 4. POS lưu credential để thực hiện tải dữ liệu ban đầu.
Ngoại lệ	Mã sai, hết hạn hoặc đã dùng bị từ chối. Khi chi nhánh đã có POS ACTIVE, luồng kích hoạt thông thường không được tạo thiết bị ACTIVE thứ hai.
Hậu điều kiện	Thiết bị có định danh và phạm vi chi nhánh; việc kích hoạt chưa tạo attendance, phiên thu ngân hoặc phiên quỹ.

Sau khi kích hoạt, POS tải danh mục và số dư để chuẩn bị bán. PostgreSQL dùng unique index có điều kiện để giới hạn một thiết bị ACTIVE tại mỗi chi nhánh, kể cả khi các yêu

cầu kích hoạt cùng đến.[17]

3.3.2 UC-02. Chấm công và đăng nhập thu ngân

Nhân viên nhập mã bảy chữ số và PIN cho nghiệp vụ đã chọn. Server xác thực thiết bị trước khi xác định chi nhánh thao tác. Với check-in, hệ thống lưu chi nhánh chính tại thời điểm ghi nhận và đánh dấu hỗ trợ nếu nhân viên đang chấm công ở cửa hàng khác. Với cashier login, server kiểm tra thêm sự trùng khớp giữa chi nhánh thiết bị và chi nhánh chính. Hai yêu cầu không dùng chung hậu điều kiện: check-in thành công không sinh quyền bán tại cửa hàng hỗ trợ.

Bảng 5. Đặc tả chấm công và xác lập quyền thu ngân

Thành phần	Đặc tả UC-02
Tác nhân, kích hoạt	Nhân viên chọn CHECK_IN, CHECK_OUT hoặc đăng nhập thu ngân trên POS đã enrollment.
Tiền điều kiện	POS trực tuyến; tài khoản ACTIVE; mã và PIN hợp lệ; credential thiết bị còn hiệu lực.
Luồng chính	1. Xác thực thiết bị và nhân viên. 2. Thực thi quy tắc của nghiệp vụ đã chọn. 3. Attendance lưu thời gian server; cashier login tạo phiên gắn nhân viên, thiết bị và chi nhánh.
Nhánh hỗ trợ	Cho phép attendance khác home store, giữ home_store_id_at_check_in và is_support; cashier login cùng nhân viên trên POS đó phải bị từ chối.
Ngoại lệ	Mất mạng không tạo attendance cục bộ. PIN sai hoặc bị khóa không tạo phiên hợp lệ. Ca chấm công cũ quá hạn đi qua quy trình xử lý quên check-out.
Hậu điều kiện	Chỉ vòng đời được yêu cầu thay đổi; attendance không thay cashier session hoặc enrollment. Phiên thu ngân chưa đồng nghĩa đã mở phiên quỹ.

Thông tin chi nhánh trong request chỉ là dữ liệu cần kiểm tra. Nguồn quyết định phạm vi là thiết bị đã xác thực và hồ sơ nhân viên trên server. Thời điểm check-in/out dùng đồng hồ máy chủ, tránh việc chỉnh giờ POS làm thay đổi thời gian attendance chính thức.[18]

3.3.3 UC-03. Hoàn tất bán hàng bằng tiền mặt

Thu ngân là tác nhân chính. Trước khi thanh toán, giỏ phải có món hợp lệ, công thức khả dụng và đủ nguyên liệu trong số dư cục bộ; thiết bị đã hoàn tất tải ban đầu và phiên quỹ đang cho phép bán. Thu ngân xác nhận tiền khách đưa, POS tính tiền thối và lập kế hoạch tiêu hao cho toàn bộ món, kích cỡ và topping. Một transaction Room sau đó lưu hóa đơn cùng các snapshot, cập nhật kho, ghi movement và tạo operation `CASH_SALE_COMPLETED`.

Nếu thiếu một nguyên liệu, thao tác bị từ chối trước khi ghi hóa đơn. Nếu transaction không hoàn tất, không tồn tại kết quả bán đã trả tiền nhưng thiếu movement tương ứng. Khi chỉ mất mạng, hóa đơn vẫn được hoàn tất cục bộ và operation ở lại hàng đợi. Hậu điều kiện của trường hợp sử dụng là một giao dịch PAID bất biến tại POS; việc máy chủ đã nhận giao dịch được biểu diễn bằng trạng thái đồng bộ riêng.

[17]Nguồn đối chiếu: apps/api/src/devices/devices.controller.ts; apps/api/src/devices/devices.service.ts; apps/api/prisma/migrations/20260916220814_foundation/migration.sql, devices_one_active_per_store_idx

[18]Nguồn đối chiếu: apps/api/src/attendance/attendance.service.ts; apps/api/src/cashier-sessions/cashier-sessions.service.ts; docs/02-business-rules.md

Bảng 6. Đặc tả bán tiền mặt với ranh giới transaction cục bộ

Thành phần	Đặc tả UC-03
Kích hoạt	Thu ngân chọn tiền mặt và xác nhận số tiền khách đưa.
Tiền điều kiện	Đã bootstrap; giỏ hợp lệ; đủ tồn local; có ngữ cảnh thu ngân và phiên quỹ cho phép bán; không có giao dịch QR đang chặn.
Luồng chính	1. Chốt snapshot giá, VAT và công thức. 2. Tổng hợp nhu cầu nguyên liệu. 3. Ghi sale, dòng món, tiêu hao, movement, balance và outbox trong transaction Room. 4. Hiển thị biên lai từ dữ liệu đã lưu.
Ngoại lệ	Tiền nhận chưa đủ hoặc thiếu tồn thì không ghi sale. Transaction lỗi phải giữ trạng thái trước bán. Mất mạng để operation chờ gửi bằng định danh cũ.
Hậu điều kiện	Sale PAID tồn tại cục bộ, tồn không âm và có operation CASH_SALE_ COMPLETED; dữ liệu server có thể chưa cập nhật.

3.3.4 UC-04. Thanh toán đơn bằng QR

Thu ngân yêu cầu tạo QR khi POS đang trực tuyến và dữ liệu cần thiết đã được đồng bộ. POS lưu lần thử thanh toán (attempt) với định danh ổn định trước khi gửi yêu cầu khởi tạo giao dịch. Máy chủ chuẩn bị yêu cầu đến cổng thanh toán và trả thông tin hiển thị QR; sau đó callback hoặc truy vấn trạng thái cung cấp căn cứ đối soát. Kết quả thanh toán được xác định từ xác nhận hợp lệ của cổng, độc lập với redirect và bộ đếm thời gian trên màn hình.

Nếu mạng lỗi trong lúc tạo, POS kiểm tra lại attempt cũ bằng định danh đã lưu. Nếu cổng đã xác nhận thanh toán nhưng ghi sổ chưa xong, giao dịch tiếp tục đối soát và bị chặn hủy. Sau khi máy chủ hoàn tất, POS lưu biên lai, nhận số dư kho rồi xóa các dòng đã thanh toán khỏi giỏ.[19]

Bảng 7. Đặc tả thanh toán QR và các kết quả cần đối soát

Thành phần	Đặc tả UC-04
Tiền điều kiện	POS và API có mạng; dữ liệu, outbox và phiên bán đáp ứng guard; chọn rõ ZaloPay hoặc MoMo Sandbox.
Luồng chính	1. Lưu attempt ổn định. 2. Server chuẩn bị đơn provider. 3. Nhận và kiểm tra xác nhận thanh toán. 4. Server ghi sổ sale và kho. 5. POS nhập biên lai, cập nhật kho rồi hoàn tất attempt.
Lỗi chưa kết luận	Timeout hoặc hết thời gian hiển thị QR giữ giao dịch để đối soát; không tự chuyển thành đã trả hoặc chưa trả.
Hủy MoMo	Chỉ giải phóng theo kết quả hủy tại server. Tiền tới QR đã hủy được ghi để đối soát; không hoàn tất lại sale bị hủy.
Hậu điều kiện	Nhánh thành công có sale PAID tại server và receipt tại POS; không tạo thêm cash outbox hoặc trừ kho QR lần nữa ở local.

[19]Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/OnlinePaymentRepository.kt; apps/api/src/payments/payments.service.ts; docs/adr/0010-momo-local-cancellation.md

3.3.5 UC-05. Lập và đăng phiếu nhập kho

Quản lý được phân quyền tạo chứng từ cho chi nhánh của mình; chủ chuỗi có phạm vi quản trị rộng hơn. Phiếu nháp lưu thông tin nguyên liệu và lượng nhập trên máy chủ, chưa cộng tồn. Vì vậy POS ngoại tuyến không cản việc soạn nháp nếu trình duyệt vẫn kết nối được API. Khi người dùng chọn đăng, hệ thống kiểm tra trạng thái POS và outbox, sau đó kiểm tra QR chưa giải quyết trong transaction trước khi ghi sổ kho.

Nếu còn thao tác chưa đồng bộ hoặc thanh toán cần đối soát, hệ thống giữ phiếu ở trạng thái chưa đăng và yêu cầu xử lý điều kiện đang cản trở. Nếu được phép, số dư, movement, trạng thái POSTED và dữ liệu phục vụ đồng bộ được ghi trong cùng transaction. Gửi lại khóa yêu cầu đã thành công phải trả kết quả đã có, không cộng lượng nhập lần hai. Chứng từ nhập từ kho tổng ở đây là nguồn nhận hàng, chưa biểu diễn một nghiệp vụ chuyển kho có đủ hai đầu xuất và nhập.

Bảng 8. Đặc tả nhận hàng và đăng kho tại chi nhánh

Thành phần	Đặc tả UC-05
Tác nhân, kích hoạt	Owner hoặc quản lý đúng chi nhánh tạo phiếu; chọn đăng khi chứng từ đã đủ điều kiện.
Tiền điều kiện	Web kết nối API. Đăng kho cần dữ liệu hợp lệ, POS sẵn sàng và không còn outbox hoặc QR chưa giải quyết theo guard.
Luồng chính	1. Lưu chứng từ và dòng nguyên liệu. 2. Kiểm tra phiếu và readiness POS. 3. Khi đăng, khóa chi nhánh và kiểm tra QR đang chờ. 4. Ghi balance, movement, sync change và POSTED trong transaction.
Ngoại lệ	POS offline vẫn có thể lưu nháp. Đăng không đạt guard giữ chứng từ chưa đăng; retry hợp lệ của phiếu đã đăng không cộng tồn lần hai.
Hậu điều kiện	Chỉ phiếu đăng thành công làm tăng tồn server; POS cần pull số dư trước khi dùng lượng mới cho bán offline.

3.3.6 UC-06. Kiểm đếm và bàn giao quỹ

Thu ngân xác thực trực tuyến tại chi nhánh chính trước khi bắt đầu nghiệp vụ. Với bàn giao hoặc kết thúc ngày, POS nhận bảng kê tiền thực tế mà không hiện số tiền theo sổ. Khi thu ngân tiếp tục đối soát, bảng kê ban đầu được cố định; việc đóng và mở lại màn hình không cho sửa số đếm sau khi đã biết thiếu. Nếu thiếu, thu ngân khai báo một bảng kê tiền bù riêng.

Khi chuyển ca, thu ngân tiếp theo nhận số thực đếm cộng tiền bù cùng bảng kê đã cộng theo từng mệnh giá. Liên kết nguồn bàn giao phải duy nhất để thao tác lại hoặc khởi động lại không mở hai phiên nhận cùng một quỹ. Kết thúc ngày không tạo quỹ chờ nhận; phiên sau bắt đầu bằng lần đếm mới. Trong cả hai trường hợp, khoản bù không trở thành doanh thu bán hàng và không xóa chênh lệch ban đầu.[20]

[20]Nguồn đối chiếu: docs/adr/0011-blind-cash-count-compensation-handover.md; apps/api/src/register-sessions/register-sessions.service.ts; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/RegisterCashRepository.kt

Bảng 9. Đặc tả kiểm đếm kín, bù quỹ và nhận bàn giao

Thành phần	Đặc tả UC-06
Tiền điều kiện	Xác thực online thu ngân tại home store; không còn QR chưa giải quyết; chọn HANDOVER hoặc END_OF_DAY cho phiên đang mở.
Luồng chính	1. Đếm theo mệnh giá, chưa biết số theo sổ. 2. Cố định bảng kê gốc khi tiếp tục đối soát. 3. Đếm bù nếu thiếu. 4. Xác nhận và lưu đóng phiên cùng operation.
Ngoại lệ	Bù chưa đủ không được hoàn tất. Restart không xóa bảng kê đã cố định. Lặp thao tác không nhận cùng nguồn bàn giao hai lần.
Hậu điều kiện	Giữ actual, variance gốc và compensation riêng. HANDOVER cho phiên nhận dùng actual cộng compensation; END_OF_DAY yêu cầu đếm mới khi mở phiên sau.

3.3.7 UC-07. Đồng bộ operation và nhận dữ liệu mới

Worker hoặc thao tác đồng bộ chủ động kích hoạt luồng khi POS có mạng. Danh tính gửi thuộc thiết bị đã enrollment; mỗi operation giữ nguyên định danh và payload đã ghi tại local. Các thao tác mở phiên, bán và đóng phiên có quan hệ phụ thuộc nên thứ tự hàng đợi là một phần của hợp đồng. Server xử lý từng operation, ghi hiệu ứng nghiệp vụ cùng kết quả tiếp nhận trong transaction riêng; một batch không được hiểu là một transaction chung.

Bảng 10. Đặc tả đồng bộ theo định danh ổn định và ACK

Thành phần	Đặc tả UC-07
Tiền điều kiện	Credential thiết bị hợp lệ; operation đã lưu bền vững; POS có mạng để gửi và nhận phản hồi.
Luồng chính	1. Đọc hàng đợi theo thứ tự. 2. Gửi operation với ID và payload cũ. 3. Server kiểm tra nguồn, hash và nghiệp vụ. 4. POS áp dụng ACK khớp operation. 5. Khi đủ điều kiện, pull và lưu dữ liệu cùng cursor.
Mất phản hồi	Giữ operation ở local để gửi lại. Server trả kết quả đã xử lý nếu ID, thiết bị và payload trùng lần trước.
Phản hồi không hợp lệ	ACK thiếu hoặc không khớp không được làm mất operation. Operation bị từ chối cần giữ thông tin lỗi để xử lý, không đổi ID nhằm bỏ qua quy tắc.
Hậu điều kiện	Operation được xác nhận có kết quả nghiệp vụ duy nhất; dữ liệu pull chỉ thay trạng thái local khi qua kiểm tra áp dụng và cập nhật cursor tương ứng.

Idempotency bảo vệ một operation trong ngữ cảnh nguồn đã xác thực. Dùng cùng ID với payload khác là xung đột cần từ chối, không phải một yêu cầu cập nhật giao dịch cũ. Chi tiết FIFO, ACK và checkpoint được phân tích cùng các sơ đồ tuần tự ở phần đồng bộ.[21]

3.3.8 Ma trận vai trò và phạm vi thao tác

Bảng 11 mô tả các quyết định quyền trọng tâm của MVP. Quyền sử dụng Web thuộc tài khoản quản trị, còn quyền gọi API POS gắn với thiết bị và nhân viên đang thao tác. Owner không có home store theo mô hình dữ liệu; mã Owner riêng không cấp thêm

[21] Nguồn đối chiếu: apps/api/src/sync/sync.service.ts, pushOperations; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/sync/OutboxSyncRepository.kt; data/sync/CatalogSyncRepository.kt

quyền bán tại POS. Với nhân viên, việc được ghi attendance tại một chi nhánh không làm chi nhánh đó trở thành home store.

Bảng 11. Vai trò và phạm vi của các nghiệp vụ trọng tâm

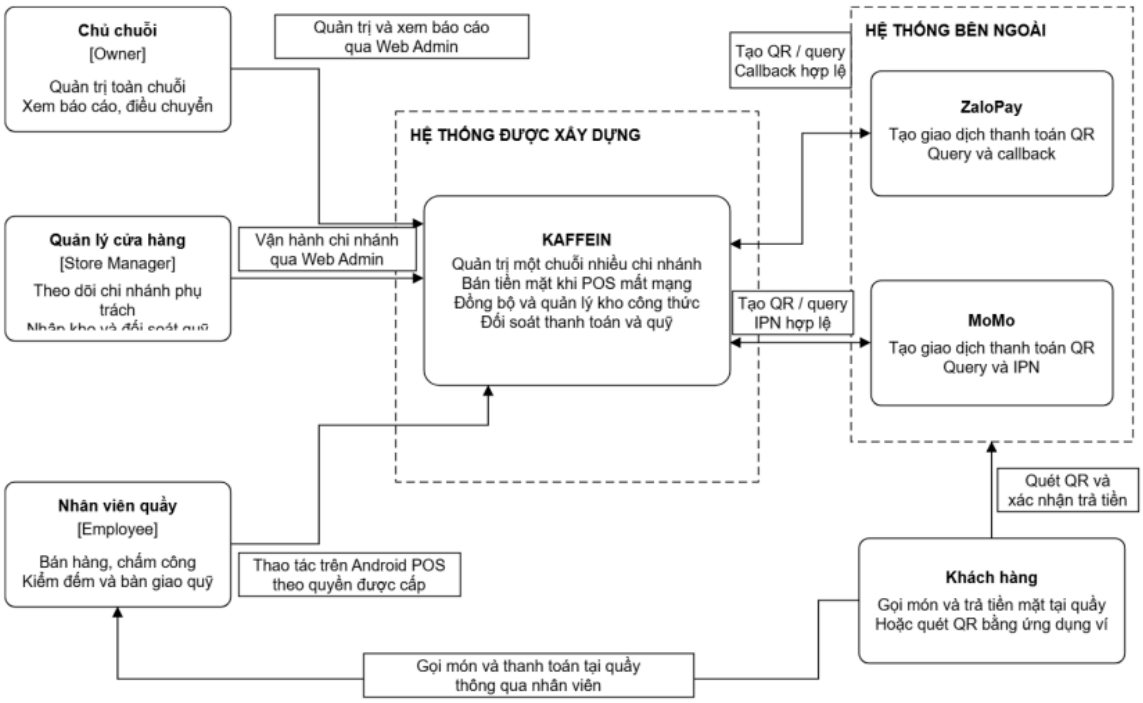
Nghiệp vụ	Owner	Store Manager	Nhân viên tại POS
Xem doanh thu, kho, attendance trên Web	Toàn chuỗi	Chi nhánh phụ trách	Không có quyền quản trị
Sửa giá, VAT và trạng thái món	Được phép	Không được sửa	Đọc catalog đã tải
Điều chuyển home store	Được phép, lưu lịch sử	Không được phép	Không được phép
Cấp mã kích hoạt	Được phép	Không được cấp	Nhập mã do Owner cấp
Lập và đăng phiếu nhập	Theo scope và guard	Trong chi nhánh, theo guard	Không có quyền quản trị
Check-in/out trên thiết bị	Không suy từ quyền Owner	Theo hồ sơ và xác thực POS	Được hỗ trợ khác home store
Đăng nhập thu ngân	Không có quyền POS từ mã Owner	Chỉ khi đúng home store	Chỉ khi đúng home store

Controller và service kiểm tra vai trò, trạng thái tài khoản và phạm vi chi nhánh từ hồ sơ hiện hành. Guard từ chối tài khoản đã khóa dù token chưa hết hạn. Riêng điều chuyển nhân viên chưa thực thi đầy đủ điều kiện POS cũ trực tuyến; phần hiện thực phân tích khoảng trống này.[22]

3.4 Kiến trúc tổng thể và ranh giới dữ liệu

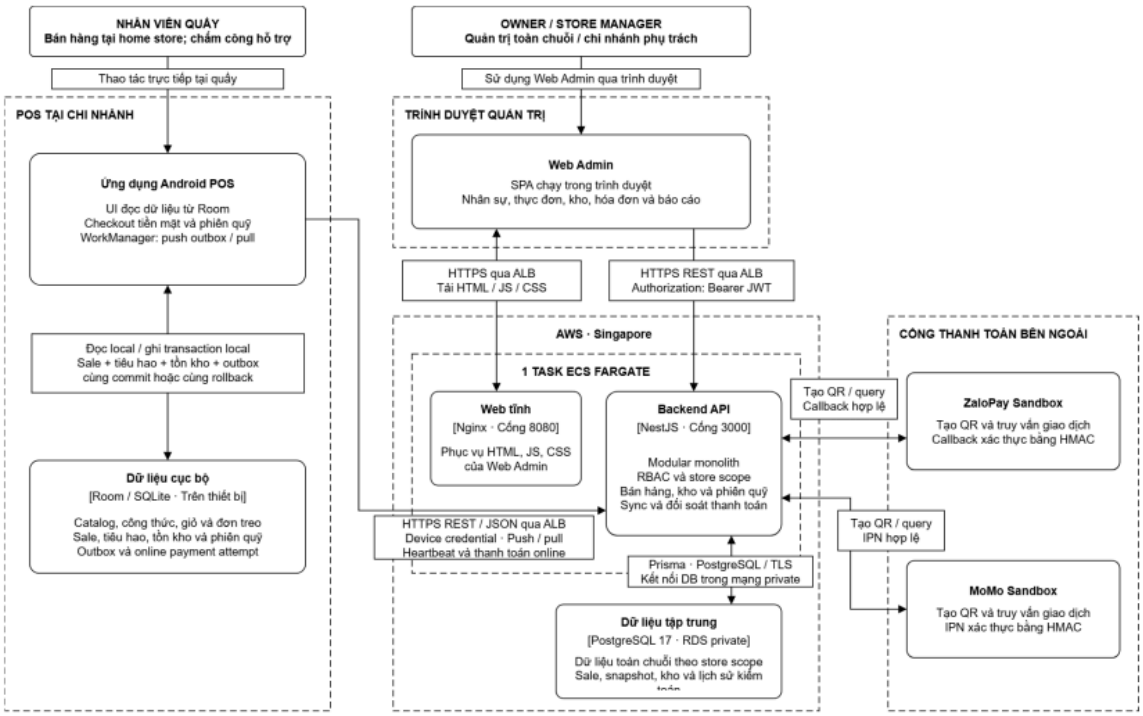
Hình 4 xác định hệ thống trong quan hệ với người dùng và cổng thanh toán. Android POS giữ trạng thái bán hàng cục bộ, Web Admin phục vụ quản trị và API tập trung thực hiện kiểm soát nghiệp vụ. Cổng thanh toán là hệ thống ngoài; kết quả của một lần gọi mạng có thể chưa xác định dù yêu cầu đã đến đích. Vì thế trạng thái đối soát được thiết kế thành dữ liệu bền vững, thay vì chỉ giữ trong cửa sổ QR.

[22]Nguồn đối chiếu: apps/api/src/common/security/web-principal.util.ts; apps/api/src/common/security/active-web-principal.guard.ts; apps/api/src/employees/employees.service.ts, transferEmployee; docs/adr/0015-owner-and-staff-employee-codes.md



Hình 4. Ngữ cảnh hệ thống, người sử dụng và các cổng thanh toán tích hợp.

Ranh giới thành phần ở Hình 5 phân biệt Room trên thiết bị với PostgreSQL trên máy chủ. Giao diện POS đọc Room; phản hồi mạng được áp dụng vào Room rồi mới phản ánh lên giao diện. Web React gọi API theo quyền của tài khoản quản trị. Hai kiểu máy khách không truy cập trực tiếp cơ sở dữ liệu PostgreSQL. Cấu trúc này cho phép bán tiền mặt trong khoảng mất mạng, đồng thời duy trì một điểm kiểm soát quyền ở backend.



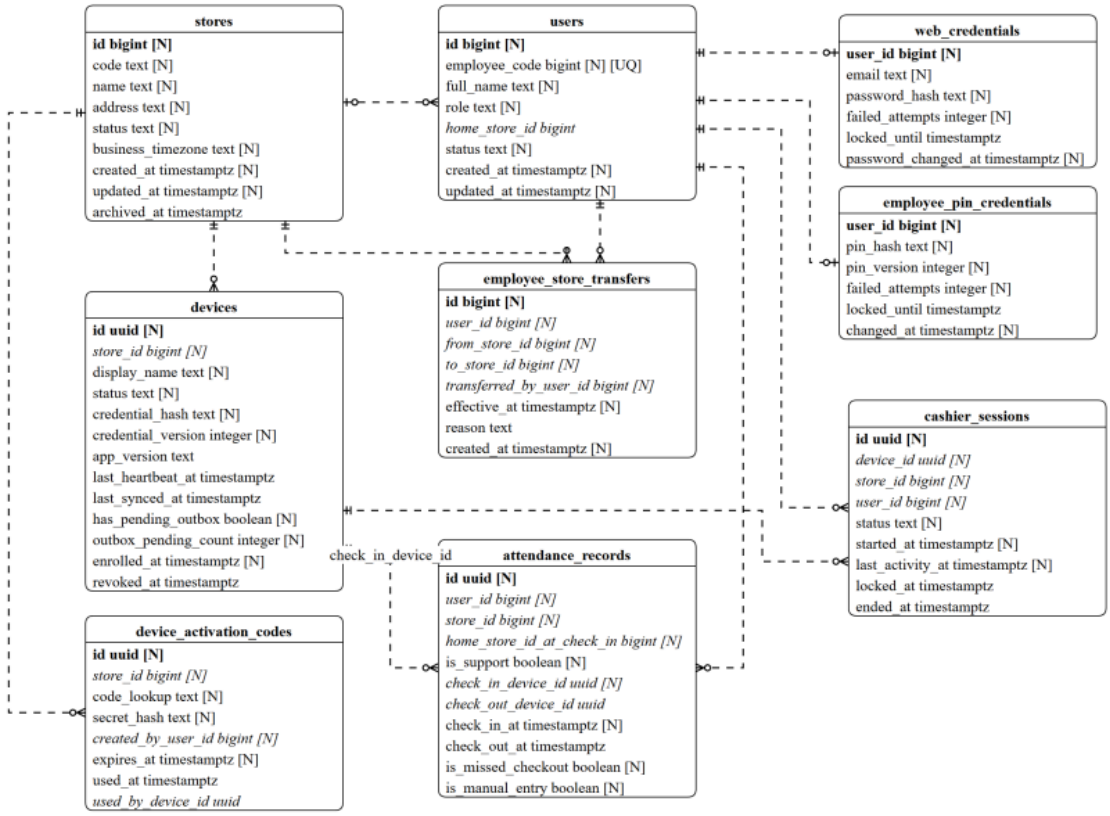
Hình 5. Các thành phần Android POS, Web Admin, API và cơ sở dữ liệu theo ranh giới local/cloud.

Backend được tổ chức thành modular monolith NestJS. Module nhân sự, thiết bị, bán hàng, kho, thanh toán và đồng bộ dùng chung PostgreSQL nhưng giữ trách nhiệm riêng trong service. Cách tổ chức này cho phép các thay đổi sale, tiêu hao và movement sử dụng cùng transaction; phiên bản MVP chưa cần một giao thức giao dịch xuyên nhiều dịch vụ độc lập. Phía triển khai đặt API và Nginx phục vụ Web trong cùng task Fargate; cấu hình mạng AWS được trình bày ở chương hiện thực.[23]

3.5 Thiết kế dữ liệu và lưu vết lịch sử

3.5.1 Tổ chức, nhân sự và thiết bị

Nhóm thực thể trong Hình 6 gắn người dùng với chi nhánh chính và lưu riêng các lần điều chuyển. Bản chấm công giữ ngữ cảnh của lần ghi nhận, còn cashier session theo dõi quyền vận hành quầy. Device giữ nguồn gốc thao tác và credential xác thực; lịch sử thiết bị không bị thay thế bằng một trường nhận diện quầy không đổi. Việc tách các quan hệ này cho phép xem lại giao dịch cũ ngay cả khi nhân viên đã được chuyển chi nhánh.



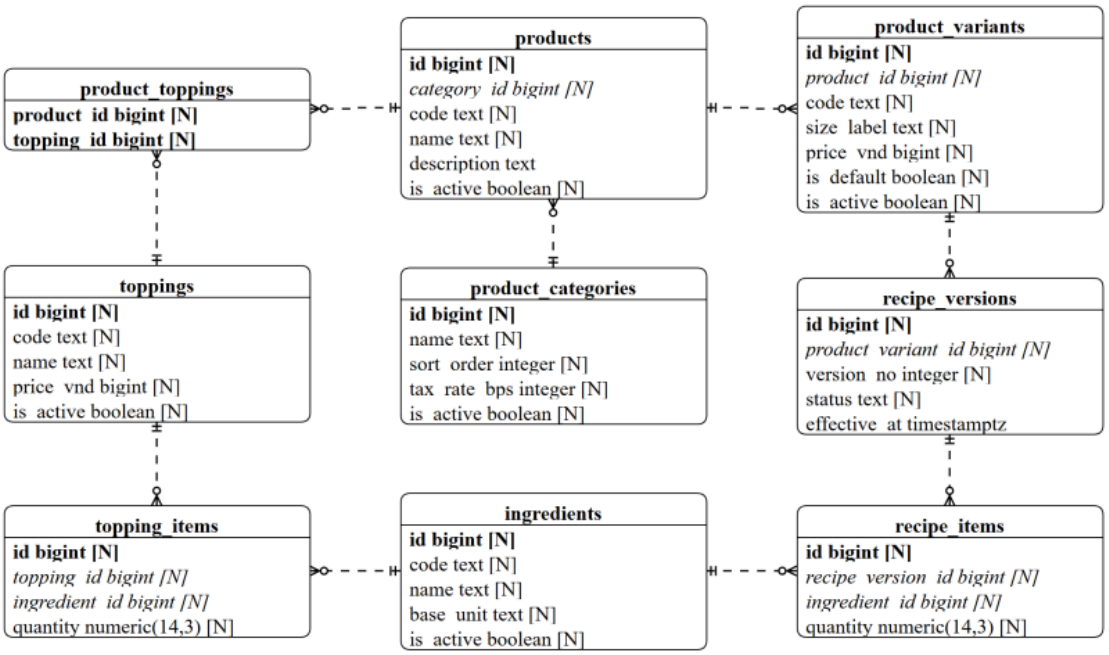
Hình 6. Quan hệ dữ liệu của chi nhánh, nhân sự, thiết bị và phiên làm việc.

3.5.2 Thực đơn, công thức và tiêu hao

Món và biến thể kích cỡ có quan hệ với các phiên bản công thức. Topping có định mức nguyên liệu riêng; số phần topping làm tăng tiêu hao của dòng món tương ứng. Hình 7 cho thấy việc định giá và mô tả công thức được lưu thành dữ liệu có quan hệ, thay vì suy

[23]Nguồn đối chiếu: docs/adr/0003-modular-monolith.md; apps/api/src/app.module.ts; infra/lib/kaffein-stack.ts; docs/adr/0013-web-admin-ecs-alb.md

ra từ tên món hoặc ghi chú tự do. Với một dòng bán, lượng cần trừ là lượng món nhân định mức biến thể, cộng lượng topping đã chọn theo số phần.

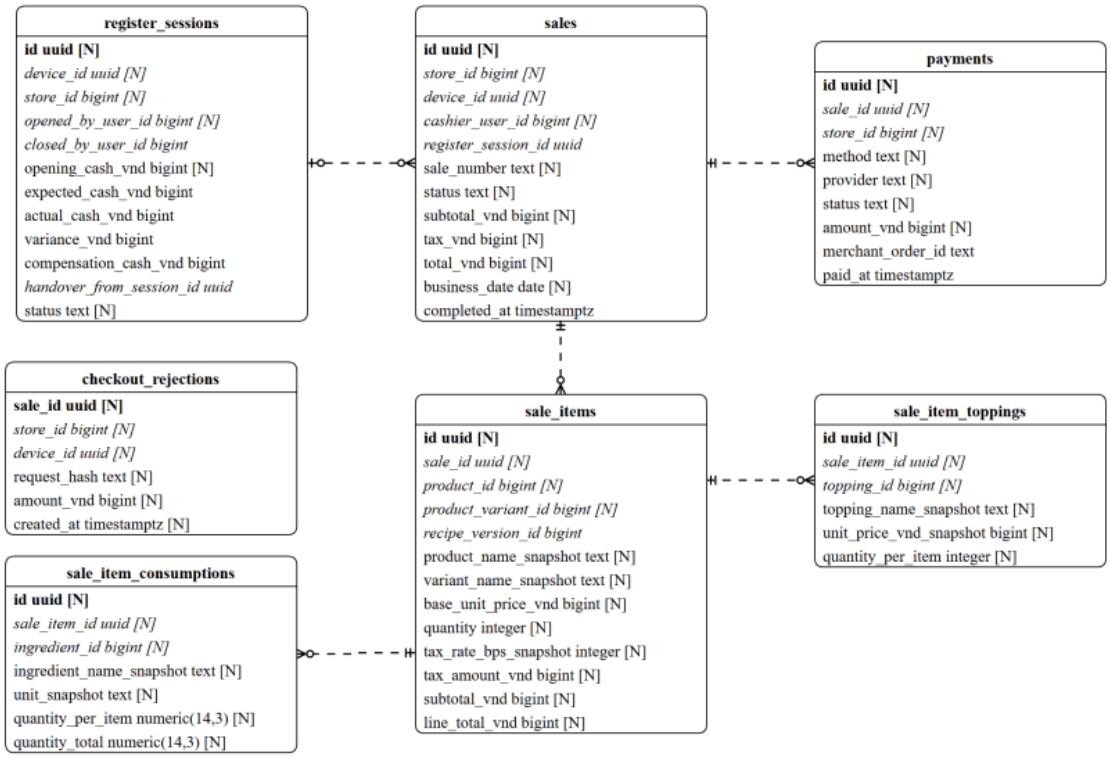


Hình 7. Cấu trúc sản phẩm, biến thể, topping và phiên bản công thức.

Khi thanh toán, dòng bán giữ tên món, tên kích cỡ, giá và mức VAT tại thời điểm bán. Bảng tiêu hao lưu tên nguyên liệu, đơn vị và lượng đã sử dụng. Do đó đổi thực đơn về sau không làm tính lại hóa đơn cũ theo công thức mới. Ghi chú như "không đá" vẫn là thông tin phục vụ pha chế, chưa tự động làm thay đổi định mức; số dư theo công thức vì vậy là tồn lý thuyết, cần đối chiếu với kiểm kê thực tế.

3.5.3 Hóa đơn, thanh toán và quỹ tiền mặt

Trong Hình 8, hóa đơn gồm các dòng món, topping và tiêu hao; payment giữ phương thức và kết quả thanh toán. Phiên quỹ liên kết hóa đơn với vốn đầu phiên và lần đóng quỹ. Việc một hóa đơn có bản ghi payment đang chờ không làm nó trở thành doanh thu đã hoàn tất. Các tổng hợp doanh thu chỉ sử dụng giao dịch đã trả theo điều kiện của API.



Hình 8. Quan hệ hóa đơn, thanh toán, tiêu hao và phiên quỹ tiền mặt.

Bảng 12. Các lựa chọn biểu diễn dữ liệu có ảnh hưởng đến giao dịch và đối soát.

Nhóm dữ liệu	Biểu diễn	Ý nghĩa thiết kế
Định danh danh mục	BigInt	Phân biệt bản ghi master; khi trao đổi cần giữ đúng giá trị định danh.
Sale, device, operation	UUID	Giữ định danh qua retry và cho phép POS tạo giao dịch trước khi server tiếp nhận.
Tiền thanh toán và tổng tiền	Số nguyên VND	Tiền nhận, tiền thối, tổng tiền và tiền bù. Riêng đơn giá nhập kho dùng Decimal(14,2).
Lượng nguyên liệu	Decimal(14,3)	Lưu lượng có phần thập phân ở PostgreSQL; POS lưu chuỗi thập phân và tính bằng BigDecimal.
Thời điểm nghiệp vụ	UTC	Lưu thống nhất; giao diện và ngày kinh doanh sử dụng Asia/Ho_Chi_Minh.
Bảng kê mệnh giá	JSON	Giữ số tờ theo từng mệnh giá để đối chiếu thực đếm và tiền bù.

Schema PostgreSQL hiện có 33 model; Room có 19 bảng phục vụ danh mục cục bộ, giỏ, hóa đơn, kho, phiên quỹ, attempt và outbox. Hai cơ sở dữ liệu phục vụ các vai trò khác nhau nên không sao chép nguyên lược đồ của nhau. Các ERD nhóm kho và đồng bộ được giữ trong bộ tệp thiết kế; những quan hệ quan trọng là số dư theo chi nhánh, movement theo nguyên nhân nghiệp vụ và operation gắn với thiết bị gửi.[24]

[24]Nguồn đối chiếu: apps/api/prisma/schema.prisma; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/entity/Entities.kt; data/local/AppDatabase.kt

3.5.4 Từ điển dữ liệu của các ràng buộc trọng tâm

Các bảng sau chọn những trường ảnh hưởng trực tiếp đến quyền chi nhánh, tính bất biến và phục hồi đồng bộ. Tên bảng, cột dùng dạng PostgreSQL; PK là khóa chính, FK là khóa ngoại, UQ là ràng buộc duy nhất. Trường có thể rỗng được nêu khi ảnh hưởng đến cách diễn giải. Migration SQL bổ sung unique index có điều kiện, CHECK và trigger ngoài các ràng buộc biểu diễn trong Prisma schema.

Bảng 13. Từ điển dữ liệu nhân sự và nguồn thiết bị

Bảng, trường	Kiểu và khóa	Ý nghĩa, điều kiện
users.id; employee_code	BigInt PK; BigInt UQ	ID nội bộ khác mã đăng nhập. Owner giữ 1000001; staff cấp theo sequence từ 1000701, không tái sử dụng.
users.home_ store_id	BigInt, FK stores, có thể rỗng	Owner không có home store; STORE_ MANAGER và EMPLOYEE phải có chi nhánh chính theo CHECK trong migration.
devices.id; store_id; status	UUID PK; BigInt FK; text	Một store có lịch sử nhiều device, nhưng unique index chỉ cho tối đa một dòng ACTIVE.
attendance_ records	UUID PK; FK user, store, device	home_store_id_at_check_in giữ bối cảnh cũ; is_ support phân biệt hỗ trợ. Unique index có điều kiện giới hạn một attendance đang mở mỗi user.
employee_store_ transfers	BigInt PK; FK user, hai store và actor	Giữ nơi đi, nơi đến, người điều chuyển, thời điểm và lý do; không suy lịch sử chỉ từ home store hiện tại.

CHECK của home store phân biệt vai trò Owner với nhân sự cửa hàng; partial unique index phân biệt lịch sử thiết bị với thiết bị còn được phép hoạt động. Cả hai ràng buộc cần được giữ khi tái tạo database. Chỉ sinh bảng từ Prisma model mà bỏ migration có thể làm mất những kiểm soát không biểu diễn trực tiếp trong model.[25]

Bảng 14. Từ điển dữ liệu bán hàng, công thức và sổ kho

Bảng, trường	Kiểu và khóa	Ý nghĩa, điều kiện
recipe_versions	BigInt PK; UQ variant và version_no	Mỗi phiên bản thuộc một biến thể; dòng bán có thể tham chiếu phiên bản đã dùng.
sales	UUID PK; UQ store_id và sale_number	Giữ device, cashier, business_date, completed_at và received_at. PAID là trạng thái nghiệp vụ; thời điểm server nhận có thể muộn hơn hoàn tất local.
sale_items	UUID PK; FK sale, product, variant	recipe_version_id có thể rỗng; tên, giá và tax_ rate_bps_snapshot giữ dữ liệu lịch sử, không đọc lại VAT hiện hành để sửa sale cũ.
sale_item_ consumptions	UUID PK; UQ sale_item_id và ingredient_id	quantity_per_item và quantity_total là Decimal(14,3); giữ tên và đơn vị nguyên liệu tại lần bán.
stock_balances	PK ghép store_id và ingredient_id	quantity Decimal(14,3), version BigInt. Cùng nguyên liệu có số dư độc lập tại từng chi nhánh.
stock_ movements	UUID PK; FK store và ingredient	Lưu quantity_delta, balance_after, nguồn nghiệp vụ và thời điểm. receipt_line_id nếu có là duy nhất.

[25]Nguồn đối chiếu: apps/api/prisma/schema.prisma, User/Device/AttendanceRecord/EmployeeStoreTransfer; apps/api/prisma/migrations/20260916220814_foundation/migration.sql; 20260926090000_attendance_sessions/migration.sql; 20261023040000_unique_owner_employee_codes/migration.sql

Movement bán hàng trong service được tổng hợp theo nguyên liệu từ toàn bộ dòng của sale. Tiêu hao và biến động kho do đó được đối chiếu theo nguyên liệu và nguồn thao tác; hai loại bản ghi không có quan hệ một–một trong đường ghi này. Bảng số dư cho biết trạng thái cuối, còn sổ movement cùng snapshot giải thích căn cứ thay đổi.[26]

Bảng 15. Từ điển dữ liệu đồng bộ, thanh toán và bàn giao

Bảng, trường	Kiểu và khóa	Ý nghĩa, điều kiện
sync_operations	operation_id UUID PK; FK device và store	payload_hash, status và result gắn kết quả với đúng nội dung và nguồn gửi. ID có tính duy nhất trên backend.
sync_changes; device_sync_checkpoints	cursor BigInt PK; device_id UUID PK	Delta có store_id rỗng cho dữ liệu chung; checkpoint theo device giữ cursor server đã trả, không xác nhận Room đã áp dụng.
payments	UUID PK; FK sale; merchant_order_id UQ	Giữ provider, số tiền, xác nhận và metadata phục hồi. provider_confirmed_at khác thời điểm hoàn tất ghi sổ.
checkout_rejections	sale_id UUID PK; FK device và store	Giữ request_hash khi từ chối trước khi gửi provider, ngăn retry trễ của cùng sale ID khởi tạo một lần thu sau khi POS đã giải phóng.
register_sessions	UUID PK; handover_from_session_id FK và UQ	Giữ actual, variance, compensation và bảng kê riêng. Một phiên nguồn chỉ liên kết tới tối đa một phiên nhận.
stock_receipts; stock_receipt_lines	UUID PK; idempotency_key UQ ở phiếu	Dòng giữ nguyên liệu, lượng và đơn vị; cặp receipt_id/line_number duy nhất. Đăng phiếu mới tạo hiệu ứng tồn.

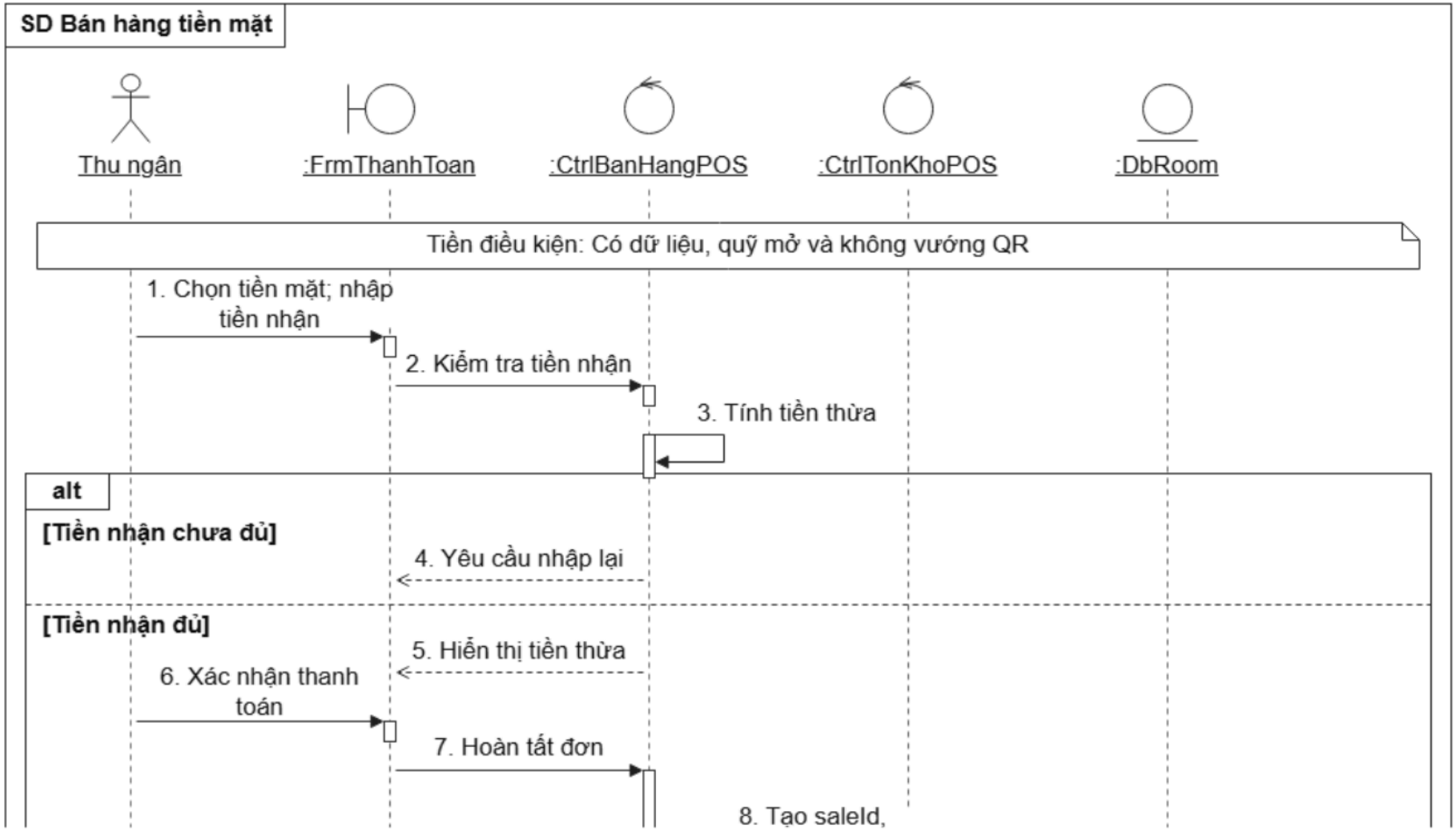
Phía Room, `outbox_operations` dùng operationId làm khóa chính và giữ payloadJson, loại thao tác, số lần thử, trạng thái và lỗi gần nhất. `sync_state` giữ cursor áp dụng cục bộ. Những trường này có trách nhiệm khác checkpoint trên server: nếu HTTP response bị mất, trạng thái local vẫn phải đủ để thử lại mà không dựng payload từ catalog mới. Bảng quỹ Room cũng có unique index trên nguồn bàn giao để giữ cùng giới hạn nhận một lần tại thiết bị.[27]

3.6 Ghi nhận tiền mặt và đồng bộ operation

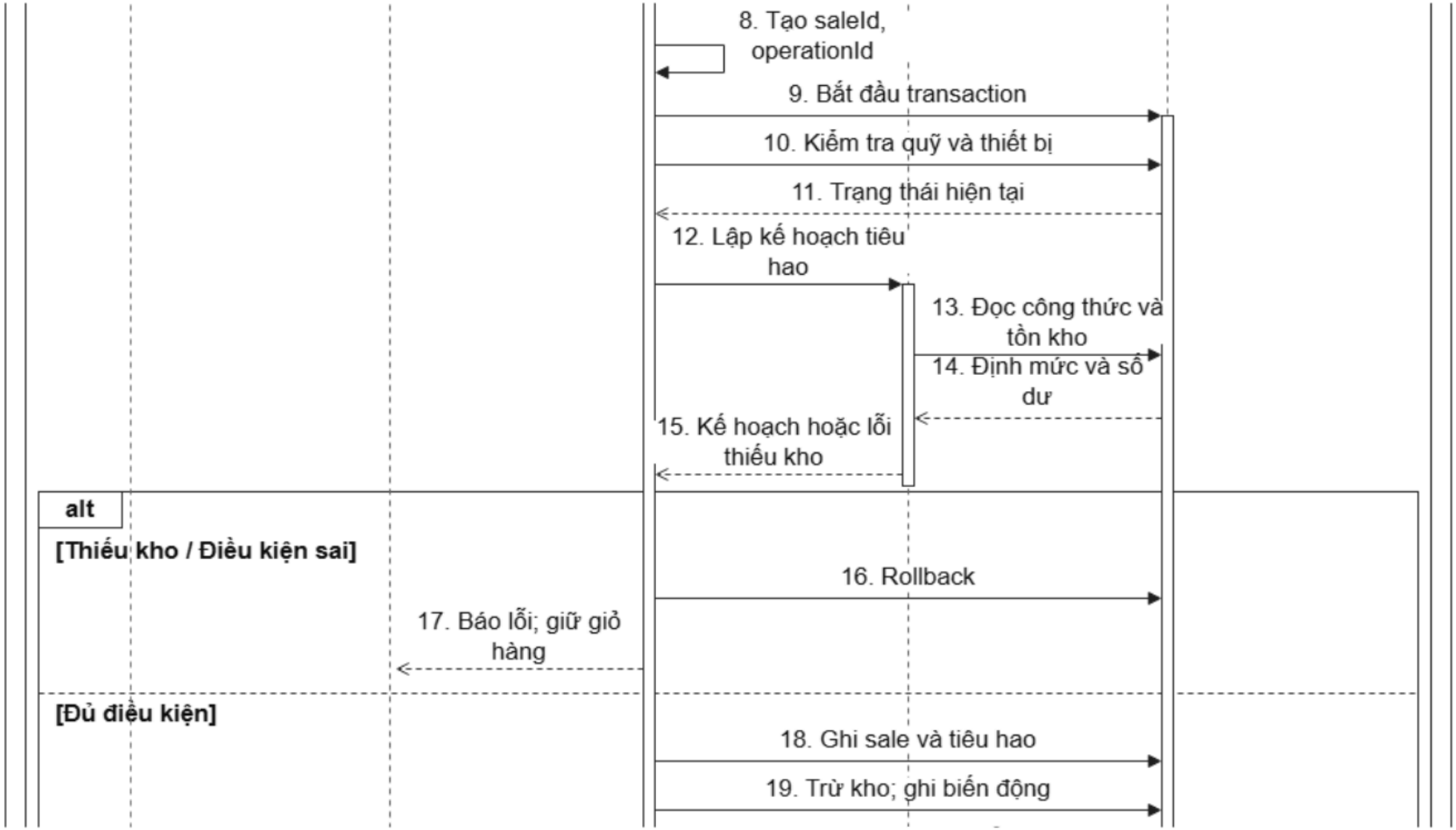
Hình 9 mô tả điểm chuyển giỏ thành giao dịch đã trả. Trong transaction Room, POS đọc công thức và số dư để lập kế hoạch tiêu hao, sau đó ghi hóa đơn, tiêu hao, movement, số dư mới và outbox. Toàn bộ thay đổi thuộc cùng một phạm vi nguyên tử cục bộ. Lần gửi dữ liệu lên máy chủ dùng lại sale ID đã được tạo tại POS.

[26]Nguồn đối chiếu: apps/api/src/sales/sales.service.ts, completeSaleInsideTx/deductStockForSale; apps/api/prisma/schema.prisma, SaleItemConsumption/StockBalance/StockMovement

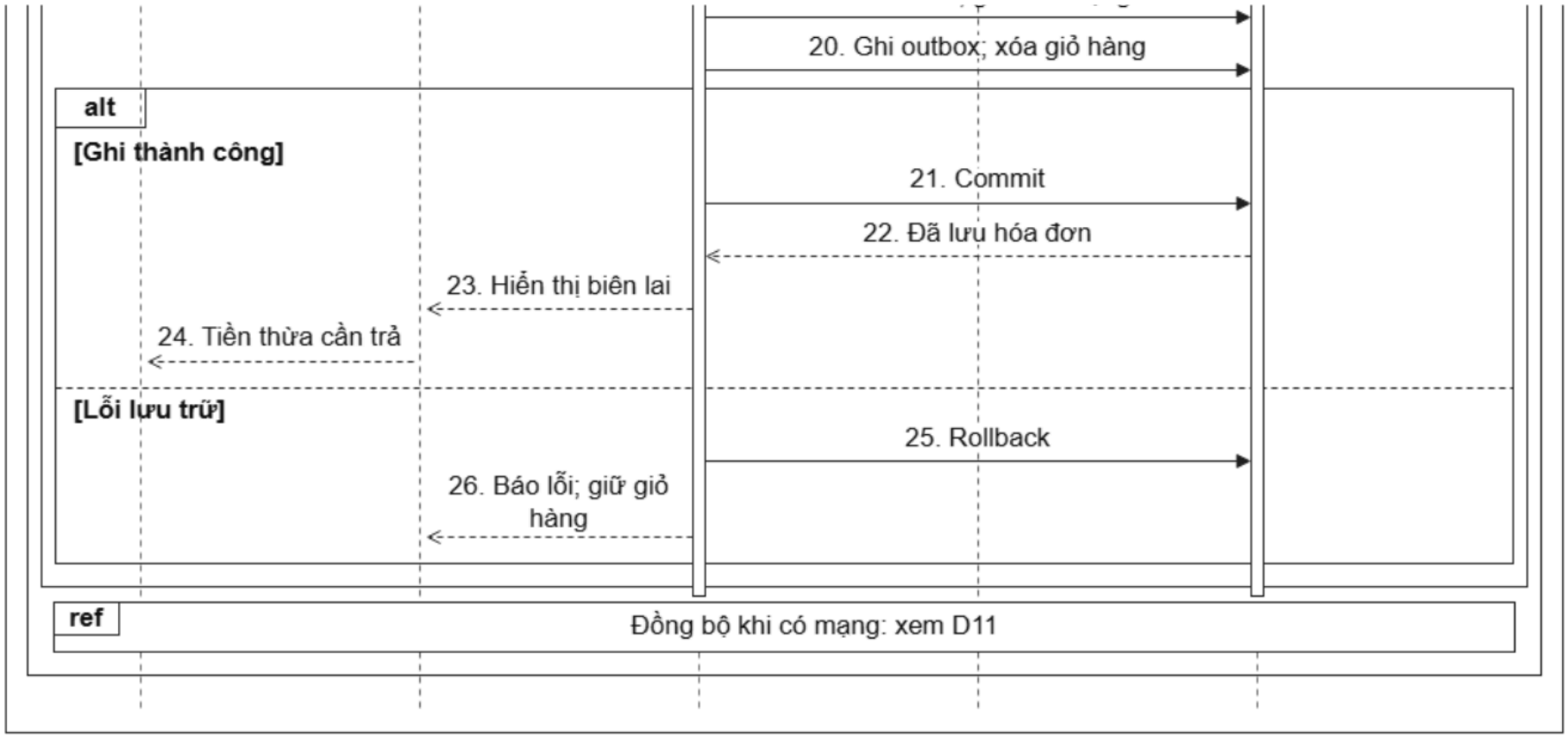
[27]Nguồn đối chiếu: apps/api/prisma/schema.prisma, SyncOperation/SyncChange/DeviceSyncCheckpoint/Payment/CheckoutRejection/RegisterSession/StockReceipt; apps/api/prisma/migrations/20261019130000_register_compensation_handover/migration.sql; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/entity/Entities.kt



Hình 9. Trình tự hoàn tất bán tiền mặt và ghi dữ liệu nguyên tử tại POS (phần 1/3)



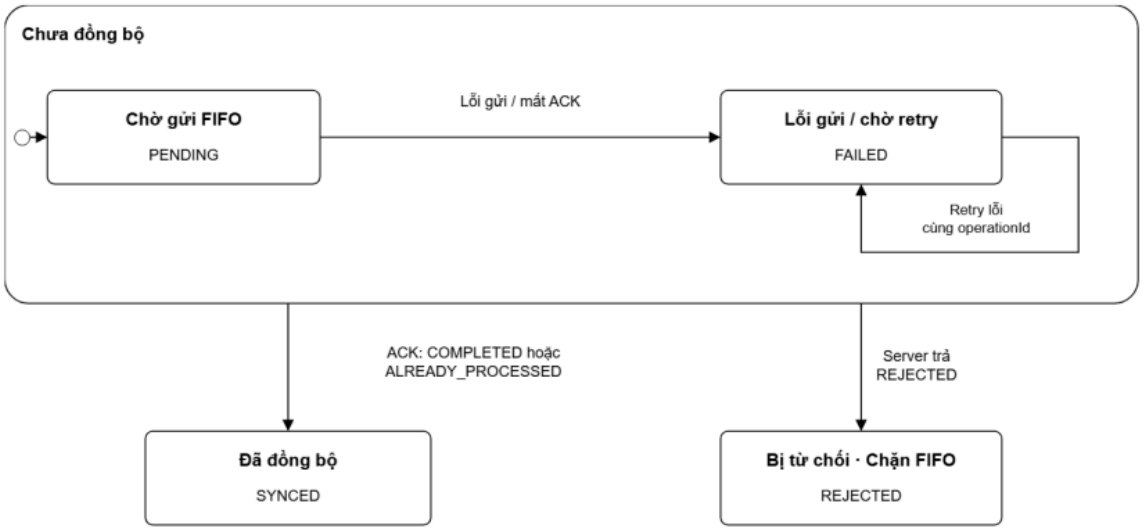
Hình 10. Trình tự hoàn tất bán tiền mặt và ghi dữ liệu nguyên tử tại POS (phần 2/3)



Hình 11. Trình tự hoàn tất bán tiền mặt và ghi dữ liệu nguyên tử tại POS (phần 3/3)

Outbox được đọc theo thứ tự enqueue bằng `rowid`. `PENDING` và `FAILED` đều là thao tác chưa đồng bộ; lỗi gửi lại giữ operationId cùng payload. Máy chủ kiểm tra định danh operation, thiết bị và hash nội dung trước khi xử lý. Nếu thao tác đã hoàn tất và các điều kiện khớp, API trả `ALREADY_PROCESSED`; nếu chưa có, hiệu ứng nghiệp vụ và bản ghi operation được commit cùng transaction PostgreSQL. Thiết kế này xử lý trường hợp server đã ghi sổ nhưng POS mất phản hồi.[28]

Trạng thái trong Hình 12 phản ánh các giá trị lưu ở Room. POS chỉ đánh dấu `SYNCED` khi nhận `COMPLETED` hoặc `ALREADY_PROCESSED`. `REJECTED` giữ lỗi và chặn các thao tác phía sau; hóa đơn đã thanh toán vẫn được giữ để đối soát. Một yêu cầu có thể được truyền nhiều lần, trong khi tác động nghiệp vụ chỉ được chấp nhận một lần theo định danh.



Hình 12. Vòng đời operation trong Room khi gửi thành công, lỗi, retry hoặc bị từ chối.

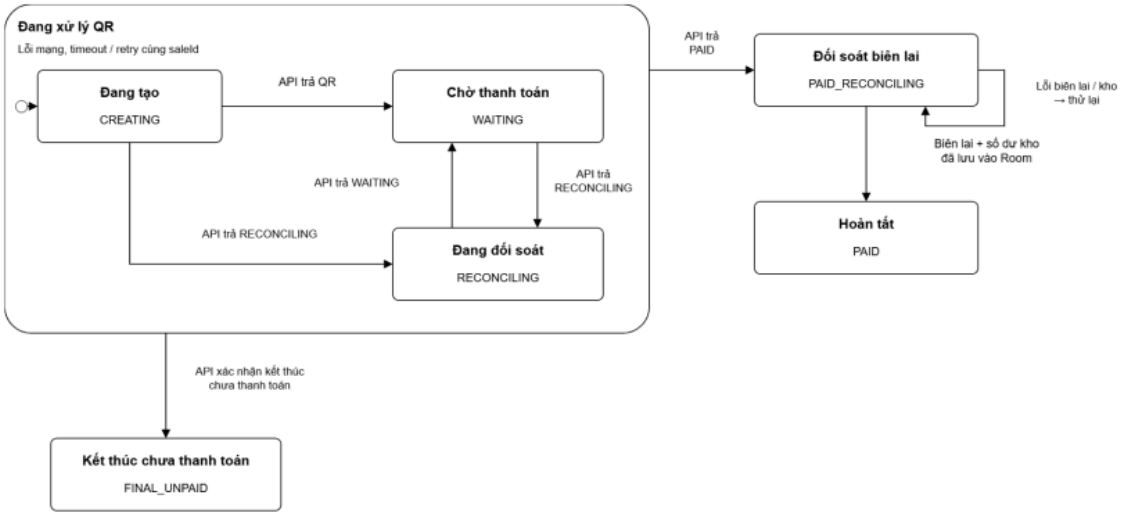
Số dư kho từ pull là giá trị tuyệt đối. Nếu áp dụng trong lúc outbox còn giao dịch đã trừ kho cục bộ, số dư nhận từ máy chủ có thể ghi đè phần tiêu hao chưa được gửi. POS vì vậy chỉ áp dụng pull khi hàng đợi sạch; sau ACK của cash, checkpoint được đặt về 0 để lấy snapshot mới. Mutex cục bộ điều phối ACK và pull. Giao thức vẫn có rủi ro cạnh tranh: heartbeat sạch không khóa một checkout mới trên Room, còn sequence cursor chưa bảo đảm thứ tự commit giữa mọi tiến trình ghi.[29]

3.7 Vòng đời QR và xử lý kết quả chưa xác định

Sơ đồ Hình 13 dùng trạng thái attempt trên POS. `CREATING` được lưu trước khi gửi yêu cầu khởi tạo giao dịch; `WAITING` cho phép hiển thị QR, còn `RECONCILING` yêu cầu tiếp tục xác minh. Hết thời gian hiển thị làm QR bị ẩn; POS chỉ giải phóng giao dịch khi máy chủ trả kết quả hủy hoặc chưa thanh toán cuối cùng, hoặc khi luồng thanh toán thành công đã hoàn tất.

[28]Nguồn đối chiếu: apps/api/src/sync/sync.service.ts; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/sync/OutboxSyncRepository.kt; data/local/dao/Daos.kt

[29]Nguồn đối chiếu: docs/04-offline-sync.md; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/sync/CatalogSyncRepository.kt



Hình 13. Vòng đời attempt QR trên POS và đối soát biên lai.

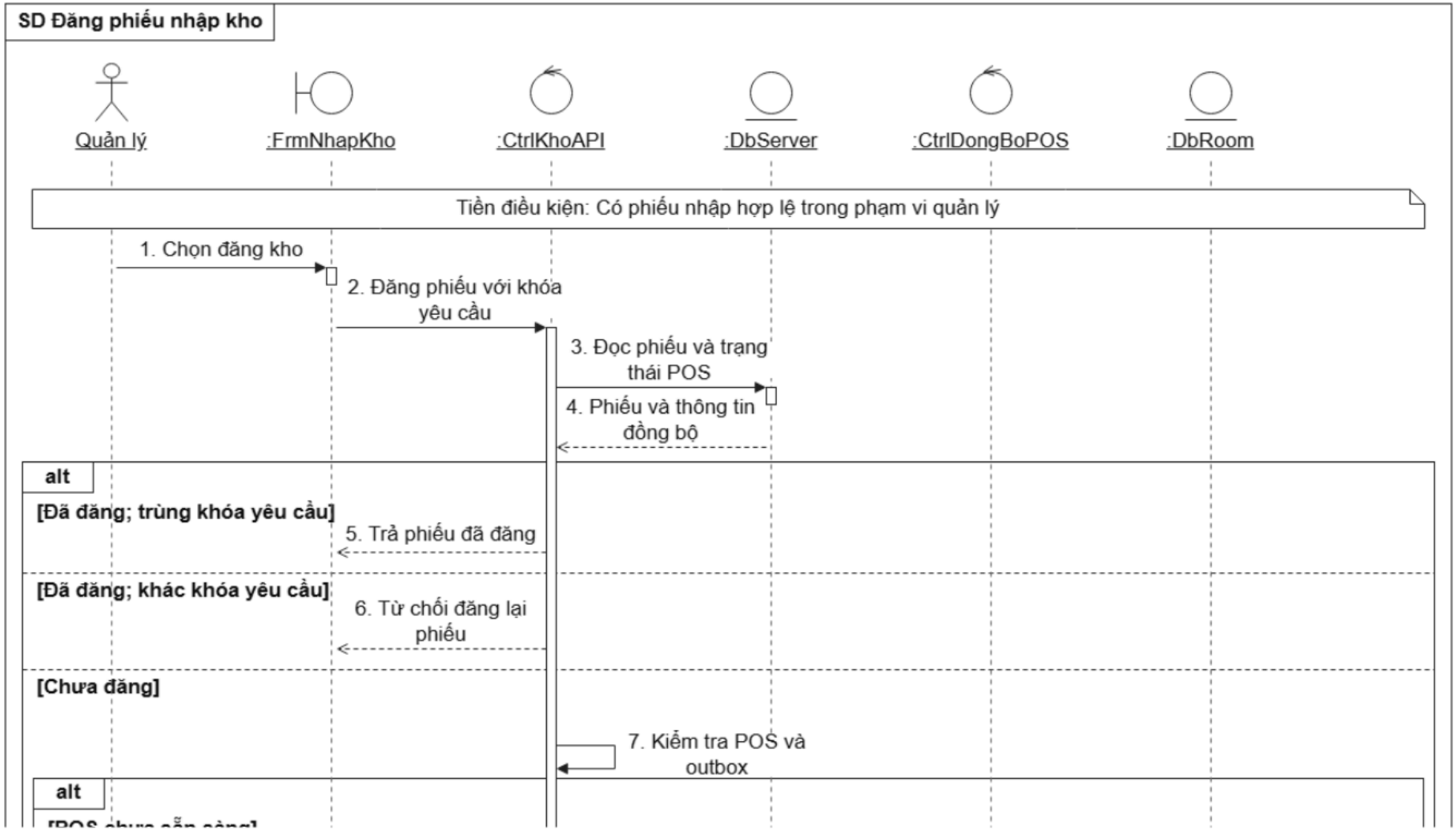
Máy chủ lưu xác nhận thanh toán hợp lệ từ cổng trước khi ghi sổ giao dịch và trừ kho trong transaction riêng. Nhờ đó lỗi ghi sổ hoặc tồn kho không xóa mất xác nhận đã nhận. Khi API trả `PAID`, POS chuyển sang `PAID_RECONCILING`, kiểm tra biên lai và tải số dư kho. Sau các bước cục bộ này, attempt mới chuyển `PAID`; POS không tạo thêm outbox tiền mặt hoặc trừ nguyên liệu của hóa đơn QR lần nữa.

Nhánh hủy MoMo được theo dõi riêng bằng trạng thái `CANCELLED`, sau khi server xác nhận hủy nội bộ. Khi chưa có bằng chứng nhận tiền, hủy thành công cho phép tạo giao dịch mới với định danh mới. QR cũ vẫn có thể nhận tiền; nếu tiền về muộn, hệ thống giữ `CANCELLED`, lưu bằng chứng và đưa khoản này vào danh sách đối soát cho quản lý. Nó không tự hoàn tất sale đã hủy, không tăng doanh thu, không trừ kho và không tự hoàn tiền. ZaloPay giữ cơ chế kết thúc chưa thanh toán dựa trên xác nhận provider tương ứng.[30]

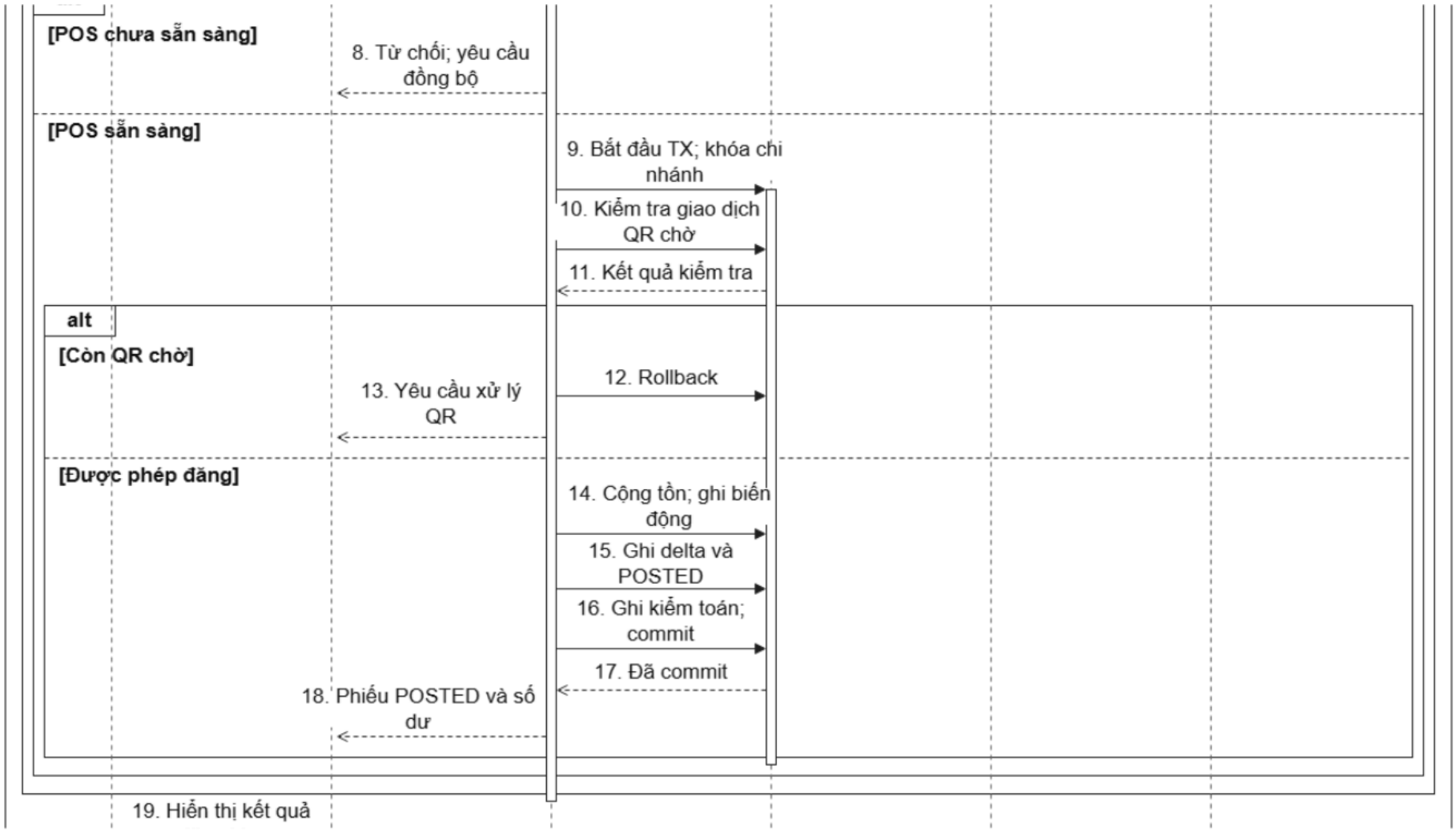
3.8 Đăng kho và kiểm soát thay đổi từ Web

Hình 14 phân biệt việc đăng một chứng từ với việc soạn nháp. Khi đăng, API kiểm tra khóa yêu cầu để nhận diện lần gửi lại, sau đó kiểm tra điều kiện POS và QR đang chờ. Nhánh được chấp nhận cộng tồn, ghi biến động và chuyển POSTED trong transaction. Kết quả thành công chỉ trả sau commit; POS tiếp nhận số dư qua cơ chế đồng bộ đang dùng, không cộng nhập thêm một lần theo tín hiệu giao diện.

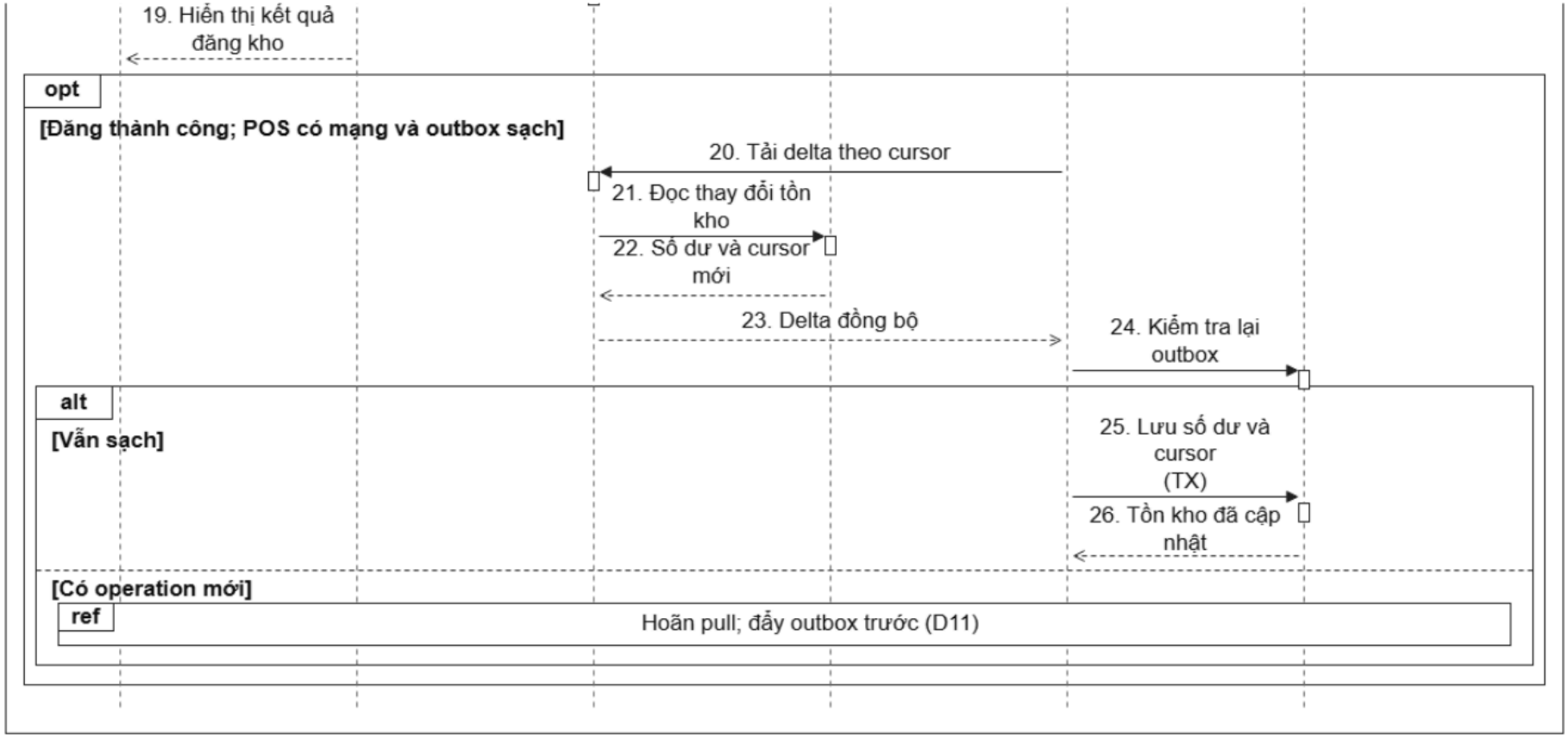
[30]Nguồn đối chiếu: docs/adr/0010-momo-local-cancellation.md; apps/api/src/payments/payments.service.ts; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/OnlinePaymentRepository.kt



Hình 14. Trình tự đăng phiếu nhập và cập nhật số dư kho về POS (phần 1/3)



Hình 15. Trình tự đăng phiếu nhập và cập nhật số dư kho về POS (phần 2/3)

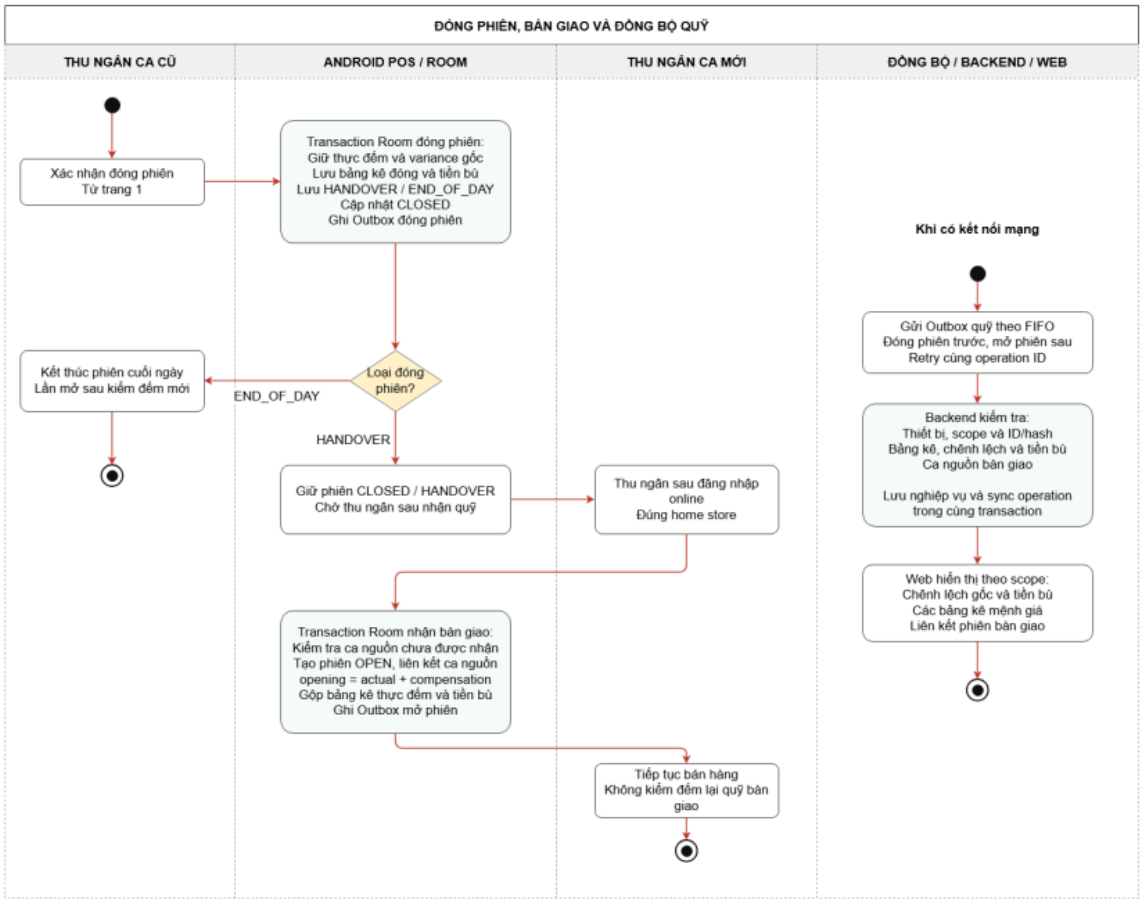


Hình 16. Trình tự đăng phiếu nhập và cập nhật số dư kho về POS (phần 3/3)

Guard trạng thái thiết bị và outbox làm giảm khả năng thao tác trên một số dư chưa cập nhật. Guard này dựa trên thông tin quan sát gần nhất và không phải khóa phân tán bao trùm cả Web với checkout cục bộ. Nếu muốn mở rộng sang nhiều POS hoặc nhiều nguồn bán, cần thiết kế lại giao thức phối hợp tồn thay vì chỉ nâng giới hạn số thiết bị.[31]

3.9 Đối soát phiên quỹ và nhận bàn giao

Số tiền theo sổ của phiên được tính từ vốn đầu ca cộng doanh số tiền mặt; chênh lệch bằng thực đếm ban đầu trừ số theo sổ. Tiền bù là một đại lượng độc lập. Ví dụ theo ADR 0011, số theo sổ 60.000 đồng, thực đếm 25.000 đồng và bù 50.000 đồng tạo chênh lệch gốc âm 35.000 đồng; két sau bù có 75.000 đồng. Giữ hai giá trị này cho phép quản lý xem cả khoản thiếu đã phát sinh lẫn số quỹ thực tế chuyển sang ca tiếp theo.



Hình 17. Hoạt động kiểm đếm kín, ghi tiền bù và bàn giao quỹ sang phiên tiếp theo.

Hình 17 cho thấy bảng kê ban đầu và bảng kê bù có thời điểm xác nhận riêng. Sau khi cố định thực đếm, POS chặn giao dịch mới để số theo sổ không tiếp tục thay đổi trong quá trình chốt. Liên kết `handoverFromSessionId` có ràng buộc duy nhất ở server và Room để giữ quan hệ nhận một lần. Quy tắc ẩn số theo sổ là một kiểm soát trong luồng vận hành; dữ liệu local vẫn tồn tại để phục vụ bán ngoại tuyến, nên không suy ra khả năng ngăn

[31] Nguồn đối chiếu: docs/adr/0006-branch-receiving-sources-and-draft.md; docs/specs/25-catalog-receiving-cash-rehearsal.md; apps/api/src/stock-receipts/stock-receipts.service.ts

mọi hình thức truy cập trực tiếp hoặc gian lận.[32]

Các ranh giới transaction trong chương này xác định vị trí cần kiểm tra ở phần hiện thực: Room bảo vệ kết quả bán tại quầy, PostgreSQL bảo vệ lần ghi sổ tập trung, còn QR cần cơ chế đối soát qua nhiều bước. Từ mô hình thiết kế đó, chương tiếp theo trình bày các module và giao diện đã triển khai cho từng đường xử lý.

[32]Nguồn đối chiếu: docs/adr/0011-blind-cash-count-compensation-handover.md; apps/api/prisma/schema.prisma; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/dao/Daos.kt

CHƯƠNG 4. HIỆN THỰC VÀ TRIỂN KHAI HỆ THỐNG

4.1 Tổ chức mã nguồn và kiểm soát truy cập

Hệ thống được hiện thực bằng ba ứng dụng trong cùng kho mã nguồn: API NestJS, POS Android và Web quản trị React. Backend là một đơn vị triển khai, chia thành các module theo nghiệp vụ. Các module bán hàng, kho, thanh toán và đồng bộ cùng truy cập PostgreSQL qua Prisma, nhờ đó một thao tác ghi sổ có thể bao gồm sale và biến động nguyên liệu trong cùng transaction. Ranh giới module thể hiện ở controller, service và các hàm kiểm tra dùng chung; chúng chưa phải các dịch vụ có cơ sở dữ liệu độc lập.[33]

POS dùng Kotlin, Jetpack Compose, Room và WorkManager. ViewModel cung cấp trạng thái cho màn hình; dữ liệu danh mục và số dư kho dùng khi bán được đọc từ Room. Công việc mạng cập nhật kho dữ liệu cục bộ thay vì yêu cầu từng thao tác chọn món phải đợi phản hồi máy chủ. Cơ sở dữ liệu Android hiện ở phiên bản 13, có migration cho các lần bổ sung danh mục, outbox, thanh toán online và bàn giao quỹ. Các migration được khai báo khi khởi tạo Room; cách làm này giữ dữ liệu qua những lần cập nhật có đường chuyển đổi tương ứng.[34]

Xác thực Web và thiết bị POS sử dụng hai ngữ cảnh khác nhau. Web gửi Bearer JWT; backend kiểm tra lại tài khoản đang hoạt động và quyền thao tác. POS sử dụng credential của thiết bị đã kích hoạt; phạm vi chi nhánh được suy ra từ thiết bị và đối chiếu với dữ liệu nghiệp vụ. Mã thiết bị hoặc giá trị chi nhánh trong payload tự nó không cấp quyền truy cập. Đăng nhập thu ngân còn kiểm tra chi nhánh chính của nhân viên, tách khỏi quyền chấm công tại chi nhánh khác được phép.[35]

Bảng 16. Các điểm kiểm soát trong hiện thực API và POS

Thành phần	Điểm xử lý chính	Ràng buộc được thực thi
API	`ActiveWebPrincipalGuard`, service nghiệp vụ	Kiểm tra tài khoản hiện hành, vai trò và phạm vi chi nhánh.
POS	`DeviceCredentialStore`, repository	Credential thiết bị và trạng thái chặn được kiểm tra trước khi ghi nghiệp vụ.
Bán tiền mặt	`CashSaleRepository`	Sale, tiêu hao, tồn kho và outbox cùng transaction Room.
Đồng bộ	`OutboxSyncRepository`, `Sync Service`	Gửi lại cùng mã thao tác; chỉ xác nhận các kết quả hợp lệ.
QR	`OnlinePaymentRepository`, `PaymentsService`	Nhận tiền, ghi sổ server và nhập biên lai POS có bước phục hồi riêng.

[33]Nguồn đối chiếu: apps/api/src/app.module.ts; docs/adr/0003-modular-monolith.md

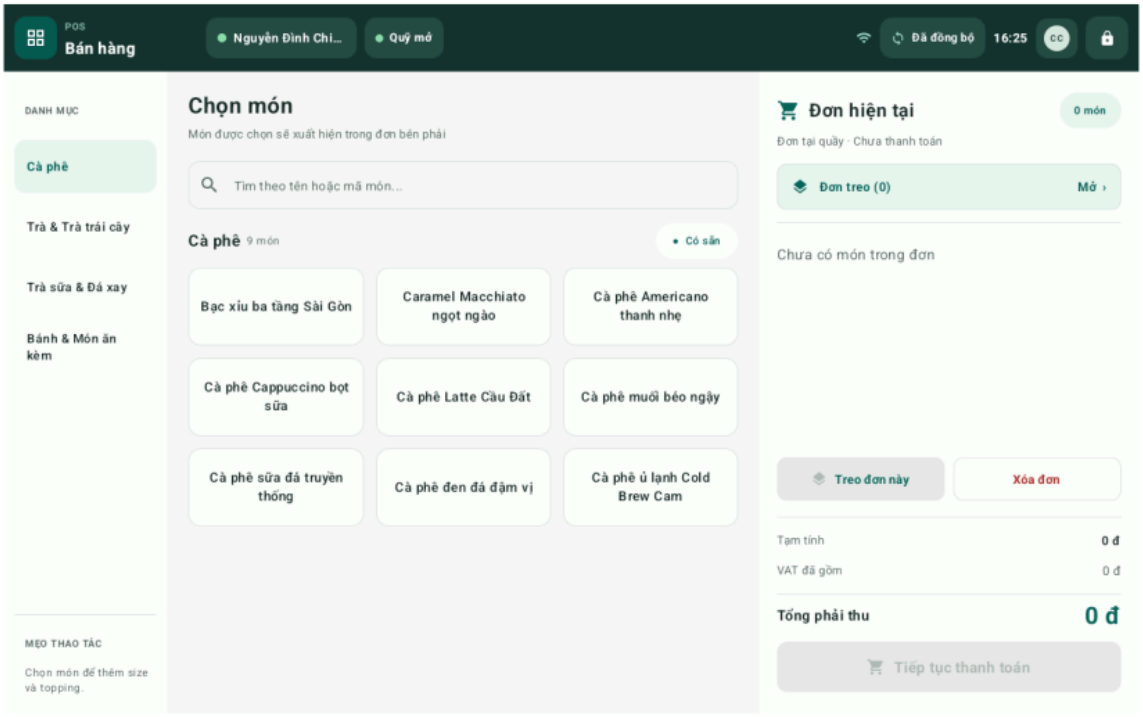
[34]Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/AppDatabase.kt; docs/adr/0004-android-native.md

[35]Nguồn đối chiếu: apps/api/src/common/security/; apps/api/src/devices/; apps/api/src/attendance/; apps/api/src/cashier-sessions/

4.2 Hoàn tất giao dịch tiền mặt trên POS

4.2.1 Chọn món và ghi giỏ hàng

Màn hình bán hàng bố trí danh mục bên trái, món ở giữa và đơn hiện tại bên phải. Ảnh tại Hình 18 ghi nhận giỏ trống; nút tiếp tục thanh toán chưa khả dụng. Khi có dòng hàng, mỗi dòng giữ sản phẩm, size, topping, ghi chú và số lượng để tính tiền cũng như lập kế hoạch tiêu hao. Giỏ đang thao tác và đơn treo được lưu cục bộ, nhưng chưa được tính là hóa đơn đã thanh toán.[36]



Hình 18. Màn hình chọn món trên POS khi giỏ hiện tại chưa có sản phẩm

4.2.2 Ghi nguyên tử sale và tiêu hao nguyên liệu

Đường hoàn tất tiền mặt được tập trung trong `CashSaleRepository.complete`. Trước khi ghi, repository kiểm tra thiết bị không bị chặn, không còn thanh toán online chưa giải quyết, đã tải dữ liệu ban đầu và phiên quỹ còn mở tại đúng chi nhánh. Số tiền nhận phải đủ thanh toán; phiên quỹ đang chốt kiểm đếm không được nhận thêm sale. Những điều kiện này được kiểm tra ở lớp dữ liệu, nên không phụ thuộc hoàn toàn vào trạng thái bật hoặc tắt nút trên màn hình.[37]

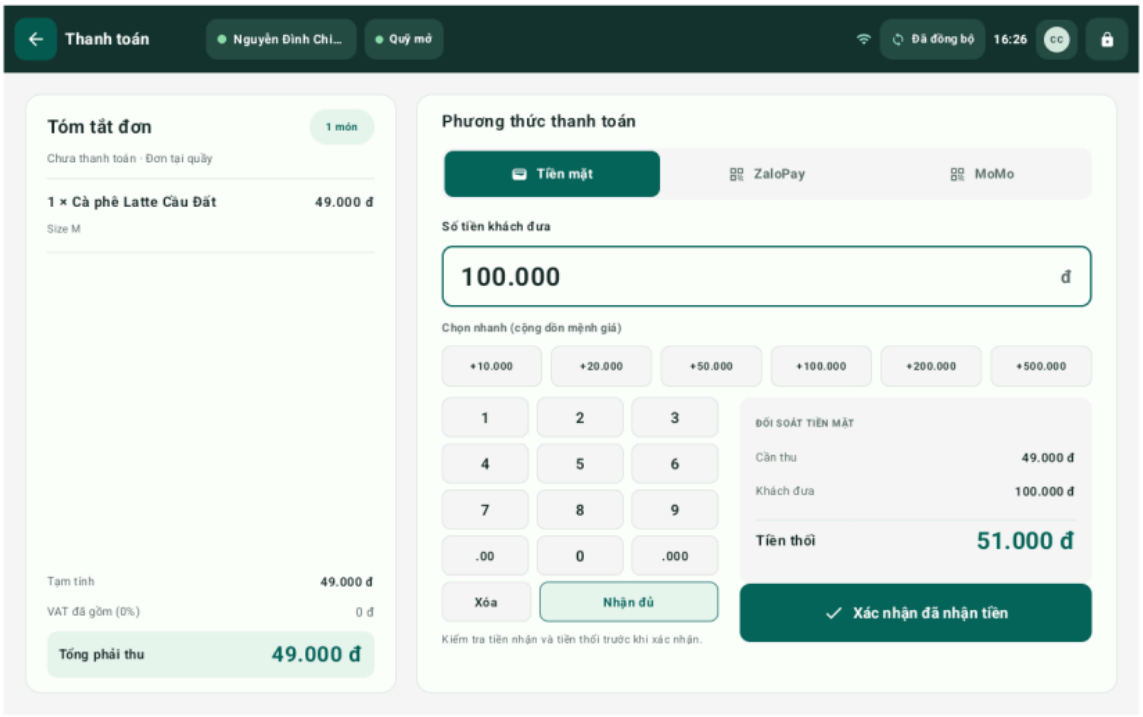
Trong cùng transaction, `InventorySalePlanner` đọc công thức đang hoạt động của biến thể và cộng tiêu hao topping. Lượng dùng cho từng nguyên liệu được nhân với số lượng món rồi cộng gộp trong toàn đơn. Planner từ chối công thức thiếu nguyên liệu, dữ liệu chưa có số dư hoặc lượng còn lại âm. POS dùng `BigDecimal` và lưu lượng dạng chuỗi thập phân, giới hạn ba chữ số sau dấu thập phân. Snapshot giữ mã phiên bản công thức, tên,

[36]Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/ui/order/; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/entity/Entities.kt

[37]Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/CashSaleRepository.kt

đơn vị và lượng tiêu hao tại thời điểm bán.[38]

Repository sau đó ghi sale, snapshot tiêu hao, số dư mới, biến động kho và operation `CASH_SALE_COMPLETED`, rồi xóa giỏ hiện tại. Lỗi ở một bước làm transaction không hoàn tất. Vì vậy không tồn tại một bước độc lập chỉ ghi hóa đơn trước rồi mới tìm cách tạo outbox sau. Điều kiện kết nối không nằm trong đường ghi tiền mặt này; khả năng bán khi mất mạng vẫn phụ thuộc dữ liệu đã tải và ngữ cảnh thu ngân, quỹ đã có trước đó.[39]

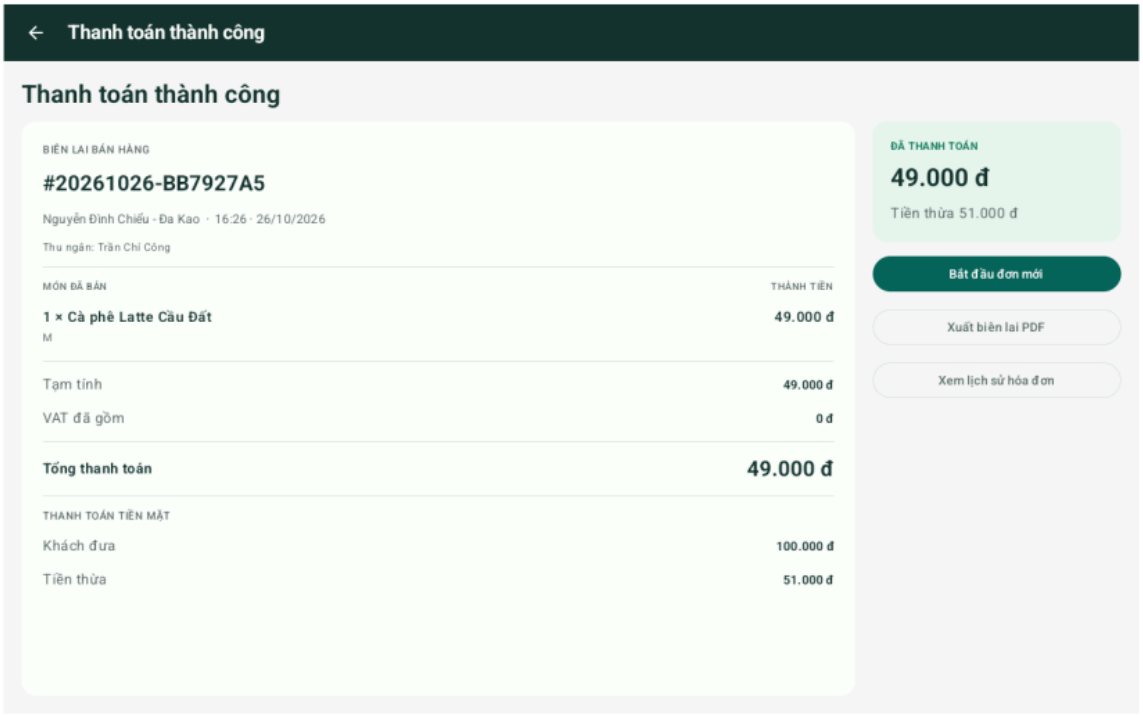


Hình 19. Thanh toán tiền mặt cho đơn 49.000 đồng, tiền nhận 100.000 đồng và tiền thối 51.000 đồng

Biên lai tại Hình 20 hiển thị cùng mặt hàng, tổng tiền và tiền thối của màn hình thanh toán. Mã biên lai xác định giao dịch đã lưu cục bộ; trạng thái đồng bộ cho biết máy chủ đã tiếp nhận hay chưa. Giao diện hỗ trợ xuất biên lai PDF và xem lịch sử. Đây là chứng từ bán hàng nội bộ của MVP.

[38]Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/InventorySalePlanner.kt

[39]Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/CashSaleRepository.kt



Hình 20. Biên lai tiền mặt trên POS sau khi hoàn tất đơn 49.000 đồng

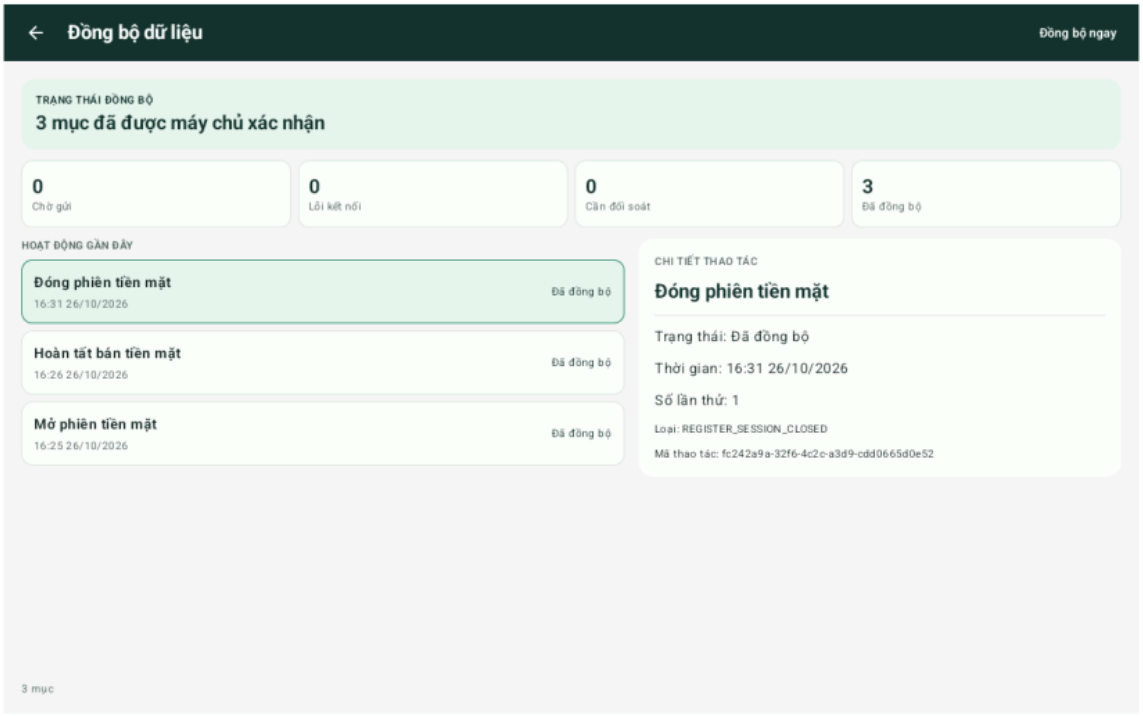
4.3 Đẩy outbox và cập nhật dữ liệu cục bộ

`OutboxSyncRepository` được dùng bởi công việc đồng bộ nền và đọc operation chưa đồng bộ theo thứ tự đưa vào hàng đợi. Một mutex tránh hai lượt đẩy cùng chạy tại máy. Nếu request lỗi hoặc phản hồi không hợp lệ, operation chuyển sang `FAILED` và giữ dữ liệu để thử lại. Mã thao tác và payload không được thay bằng một giao dịch mới chỉ vì mất phản hồi. Ở server, `SyncService` đối chiếu mã, thiết bị và hash payload trước khi áp dụng hiệu ứng nghiệp vụ.[40]

POS chỉ ghi `SYNCED` với kết quả `COMPLETED` hoặc `ALREADY_PROCESSED`. Kết quả `REJECTED` được giữ cùng lỗi và dừng hàng đợi phía sau để tránh bỏ qua một nghiệp vụ có quan hệ thứ tự. Với ACK bán tiền mặt, checkpoint được đặt về 0 trong transaction cục bộ để lần kéo tiếp theo nhận snapshot kho mới. Pull chỉ áp dụng khi outbox sạch, bởi số dư từ máy chủ là giá trị tuyệt đối và có thể ghi đè lượng đã trừ cho sale chưa gửi.[41]

[40] Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/sync/OutboxSyncRepository.kt; apps/api/src/sync/sync.service.ts

[41] Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/sync/OutboxSyncRepository.kt; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/sync/CatalogSyncRepository.kt

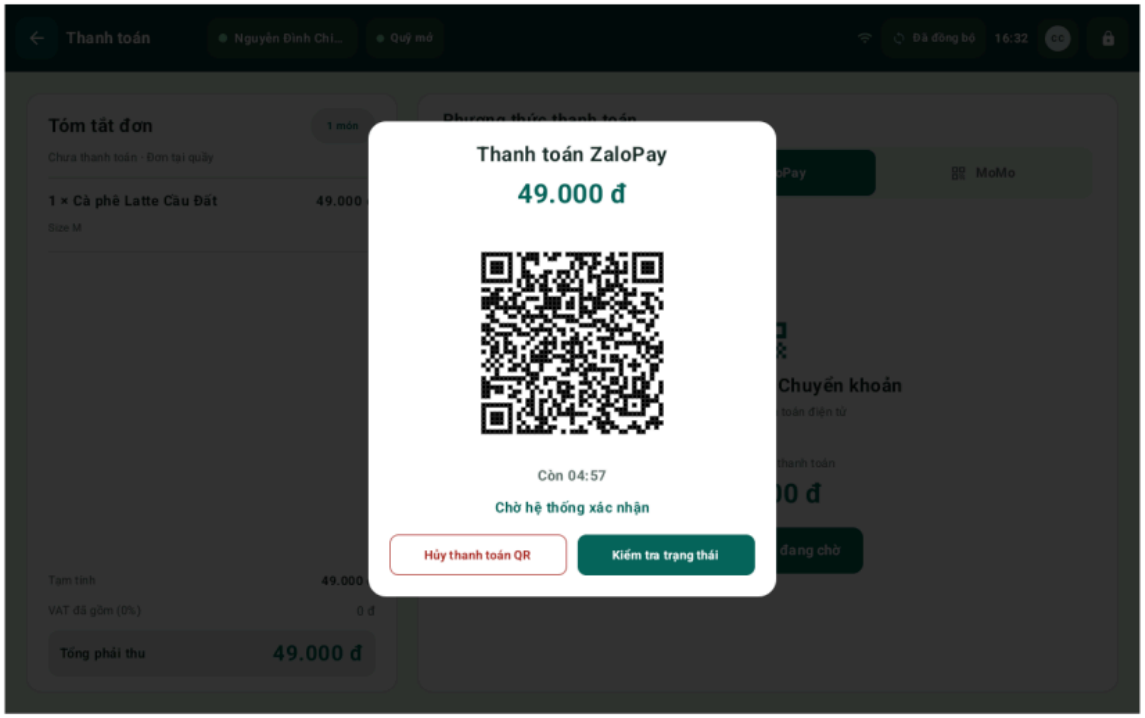


Hình 21. Danh sách ba operation mở quỹ, bán tiền mặt và đóng quỹ đã được máy chủ xác nhận trên màn hình đồng bộ

4.4 Thanh toán QR và phục hồi đối soát

POS kiểm tra khả năng sử dụng thanh toán online, rồi ghi attempt ở trạng thái `CREATING` trước khi gửi yêu cầu khởi tạo giao dịch. Attempt giữ mã sale và nội dung yêu cầu ổn định để tìm lại cùng giao dịch nếu phản hồi tạo QR bị mất. POS kiểm tra định danh, số tiền, nhà cung cấp và mã thanh toán của phản hồi. Một attempt đã sang `PAID_RECONCILING` bị chặn tạo lại QR và thu tiền mặt cho cùng giao dịch.[42]

[42]Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/OnlinePaymentRepository.kt



Hình 22. Hộp thoại QR ZaloPay cho đơn 49.000 đồng ở trạng thái chờ xác nhận, có thao tác kiểm tra và yêu cầu hủy

Ở backend, `recordConfirmation` lưu xác nhận thanh toán hợp lệ từ cổng trong transaction riêng, trước khi ghi sổ giao dịch và trừ kho. Lỗi ghi sale hoặc kho do đó vẫn giữ được xác nhận để tiếp tục xử lý. Callback và phản hồi truy vấn phải qua kiểm tra chữ ký hoặc thông tin giao dịch tương ứng. Ảnh Hình 22 minh họa trạng thái chờ xác nhận trên POS.[43]

Sau khi API trả `PAID`, POS nhập biên lai trong trạng thái `PAID_RECONCILING`, kéo snapshot kho rồi mới dọn các món đã trả và ghi `PAID`. POS không tạo thêm outbox tiền mặt hoặc trừ nguyên liệu lần thứ hai cho biên lai online. Với MoMo, hủy nội bộ được giữ thành một trạng thái kết thúc có dấu vết; tiền về muộn được ghi nhận để quản lý đối soát, không tự hoàn tất lại sale đã hủy. Ranh giới này giữ được thông tin sai lệch mà không bổ sung nghiệp vụ hoàn tiền ngoài MVP.[44]

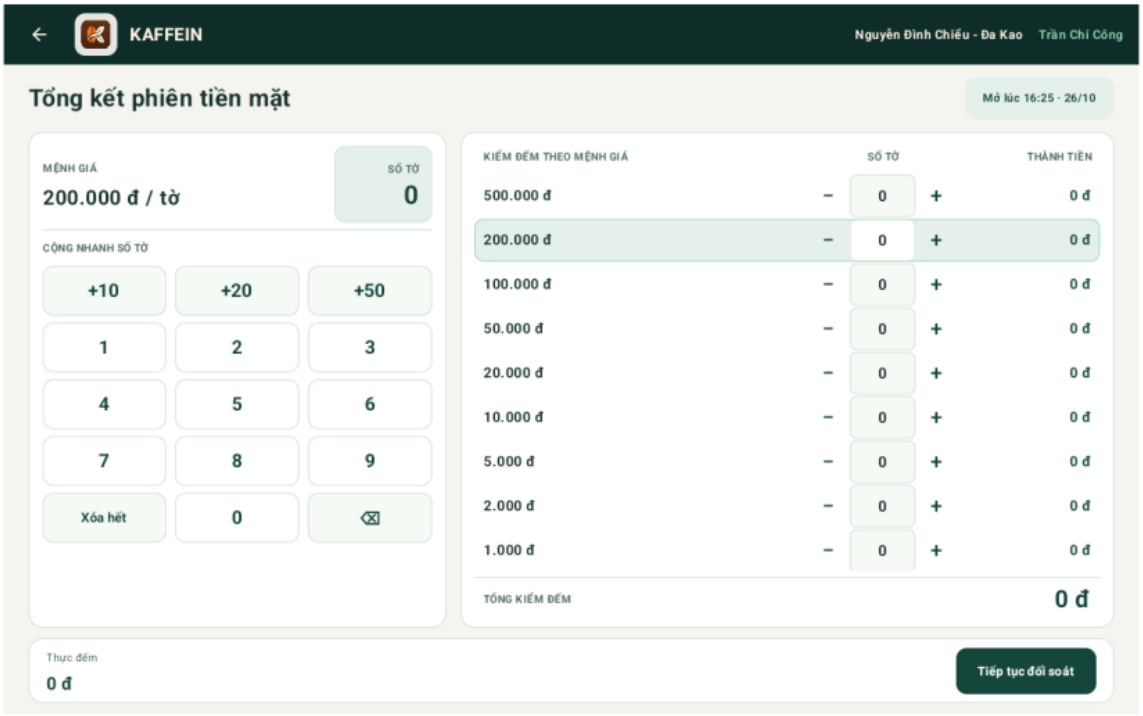
4.5 Kiểm đếm và bàn giao quỹ

Màn hình kiểm đếm nhận số tờ theo mệnh giá và tính tổng thực đếm. Hình 23 ghi lại thời điểm chưa nhập bảng kê; số dư kỳ vọng không xuất hiện trên màn hình này. Khi chốt, hệ thống giữ bảng kê gốc và chênh lệch ban đầu, còn tiền bù được ghi bằng trường và bảng kê riêng. Tiền bù làm thay đổi lượng tiền bàn giao nhưng không tăng doanh thu bán hàng.[45]

[43] Nguồn đối chiếu: apps/api/src/payments/payments.service.ts; docs/adr/0007-momo-online-payments.md; docs/adr/0008-momo-query-primary-desktop.md

[44] Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/OnlinePaymentRepository.kt; docs/adr/0010-momo-local-cancellation.md

[45] Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/RegisterCashRepository.kt; docs/adr/0011-blind-cash-count-compensation-handover.md



Hình 23. Kiểm đếm quỹ theo mệnh giá trước khi nhập số tờ và tiếp tục đối soát

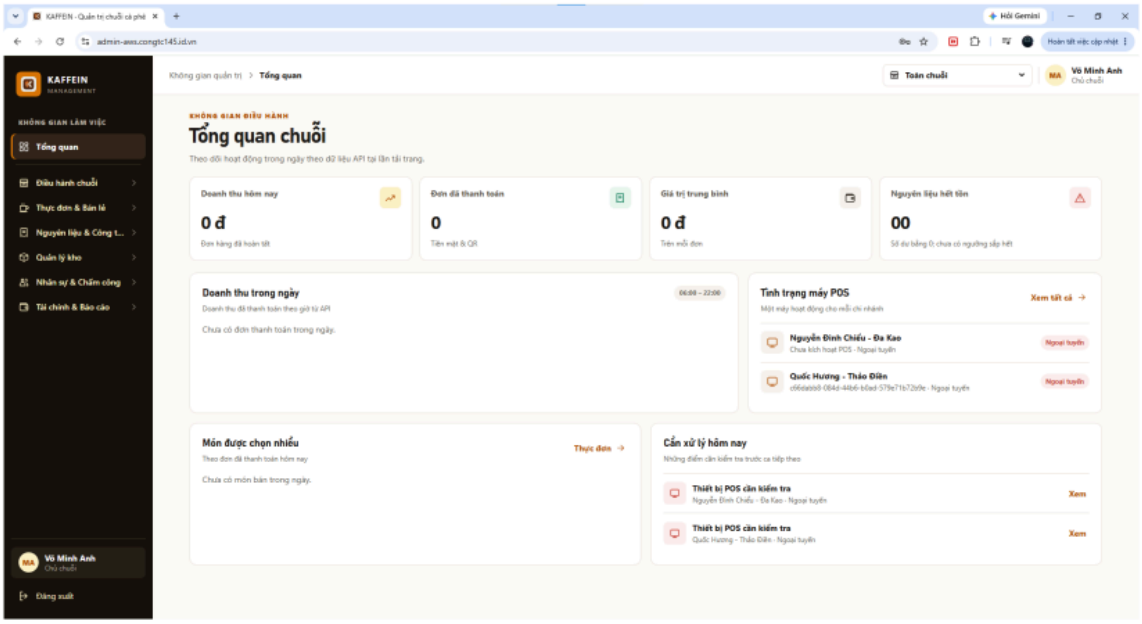
Chế độ `HANDOVER` chuyển thực đếm cộng tiền bù sang phiên sau và giữ liên kết nguồn duy nhất. `END_OF_DAY` không tạo một khoản chờ nhận; phiên kế tiếp phải mở bằng bảng đếm mới. Cả Room và PostgreSQL có ràng buộc để tránh nhận cùng một nguồn hai lần. Xác thực ở đầu nghiệp vụ vẫn cần kết nối theo hiện thực hiện hành; che số kỳ vọng trên UI không bảo vệ được dữ liệu trước người có quyền đọc trực tiếp cơ sở dữ liệu thiết bị.[46]

4.6 Web quản trị và dữ liệu theo chi nhánh

Web React cung cấp các trang danh mục, kho, nhân sự, hóa đơn và phiên quỹ. Dữ liệu được lấy qua API client; bộ chọn chi nhánh hỗ trợ chọn phạm vi hiển thị trong quyền do backend áp dụng. Màn hình tổng quan tại Hình 24 ghi lại thời điểm chưa có đơn thanh toán trong ngày và hai thiết bị đang hiển thị ngoại tuyến.[47]

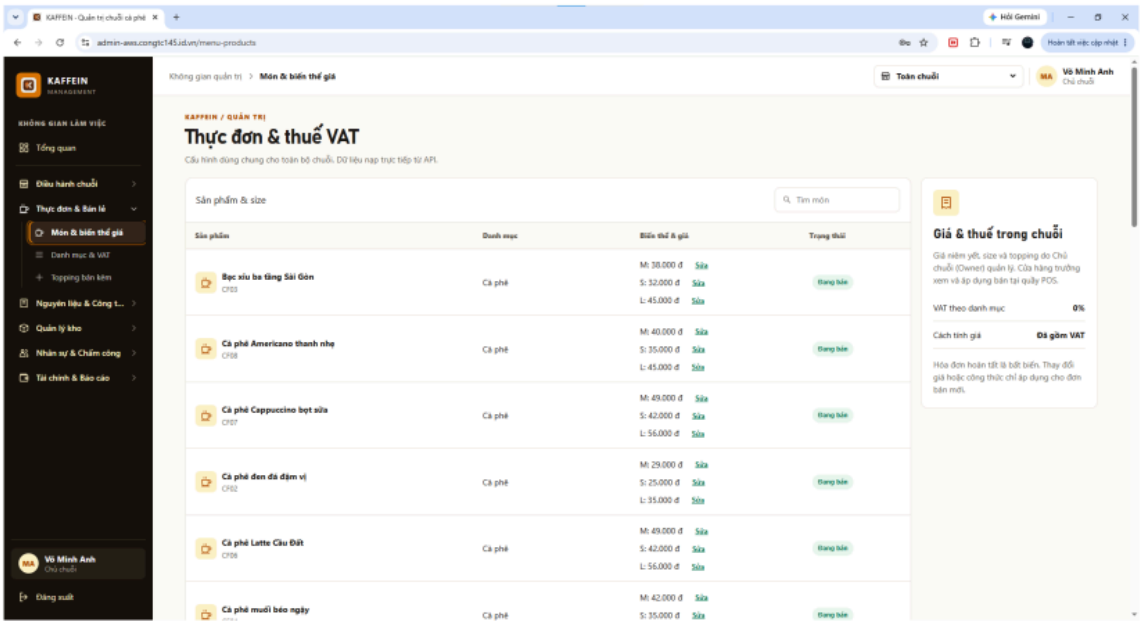
[46] Nguồn đối chiếu: docs/adr/0011-blind-cash-count-compensation-handover.md; apps/api/src/register-sessions/register-sessions.service.ts

[47] Nguồn đối chiếu: apps/web/src/App.tsx; apps/web/src/services/api.ts; apps/web/src/context/AuthContext.tsx



Hình 24. Tổng quan Web ở phạm vi toàn chuỗi khi chưa có đơn trong ngày và các POS hiển thị ngoại tuyến

Ở phần thực đơn, người có quyền có thể sửa giá biến thể, giá topping, thuế theo danh mục và trạng thái bán. Snapshot trong hóa đơn giữ dữ liệu của giao dịch cũ khi cấu hình được thay đổi. Trang nguyên liệu và công thức hiện phục vụ tra cứu; controller chưa có đầy đủ API tạo sản phẩm, topping, danh mục hoặc sửa phiên bản công thức.[48]



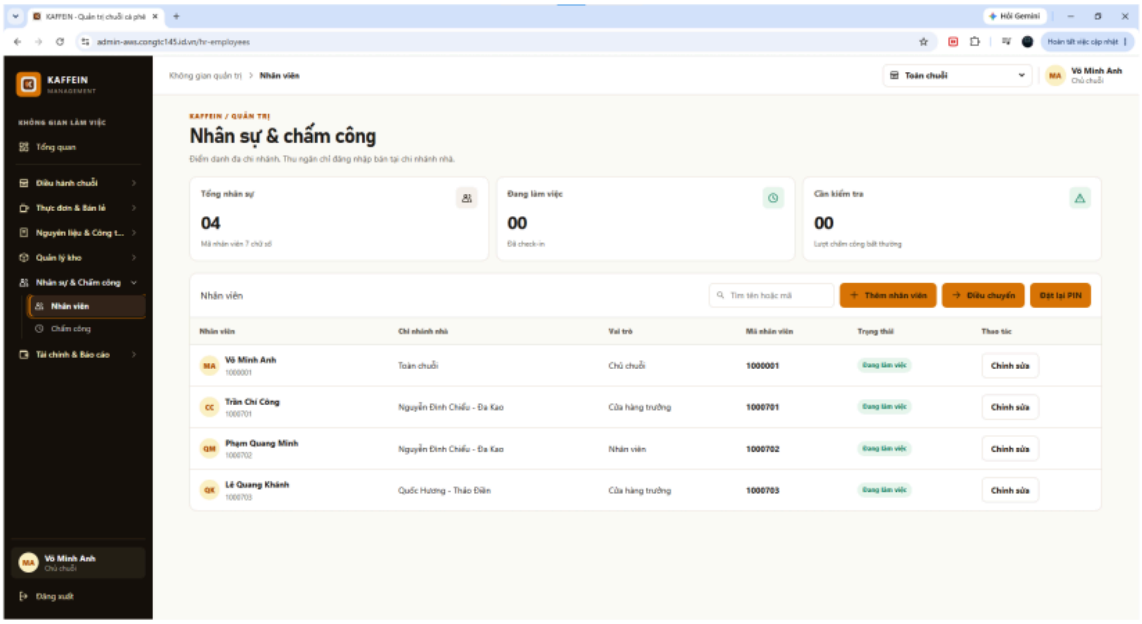
Hình 25. Danh sách món và giá từng size trên Web; giá và VAT được cấu hình dùng chung trong chuỗi

Quản trị nhân viên bao gồm tạo hồ sơ, thay đổi thông tin, đặt lại PIN và điều chuyển bởi Owner. Phương thức `transferEmployee` ghi lịch sử, đổi home store, thu hồi cashier session ACTIVE và ghi audit trong cùng transaction. Tuy nhiên, đường xử lý hiện tại

[48] Nguồn đối chiếu: apps/api/src/catalog/catalog.controller.ts; apps/api/src/inventory/inventory.controller.ts; apps/web/src/App.tsx

chưa kiểm tra heartbeat/outbox của POS cũ trước transaction, chưa kiểm tra chi nhánh đích ACTIVE và chưa phát roster delta ngay trong phương thức. Quy tắc yêu cầu POS cũ online trong tài liệu nghiệp vụ vì vậy chưa được hiện thực đầy đủ ở luồng này. Khả năng thu hồi quyền local cần kiểm thử riêng; không thể kết luận một thao tác Web đã cập nhật tức thời mọi POS ngoại tuyến.[49]

Web cũng hỗ trợ điều chỉnh bản chấm công và ghi bổ sung ca với lý do. API kiểm tra phạm vi của người thao tác, giữ dữ liệu trước và sau thay đổi trong dấu vết kiểm toán. Thao tác này sửa dữ liệu hiện diện của nhân viên, không cấp một phiên thu ngân mới và không mở rộng quyền bán hàng ngoài chi nhánh chính.[50]



Hình 26. Danh sách nhân viên ở quyền Chủ chuỗi, gồm chi nhánh chính, vai trò và thao tác điều chuyển

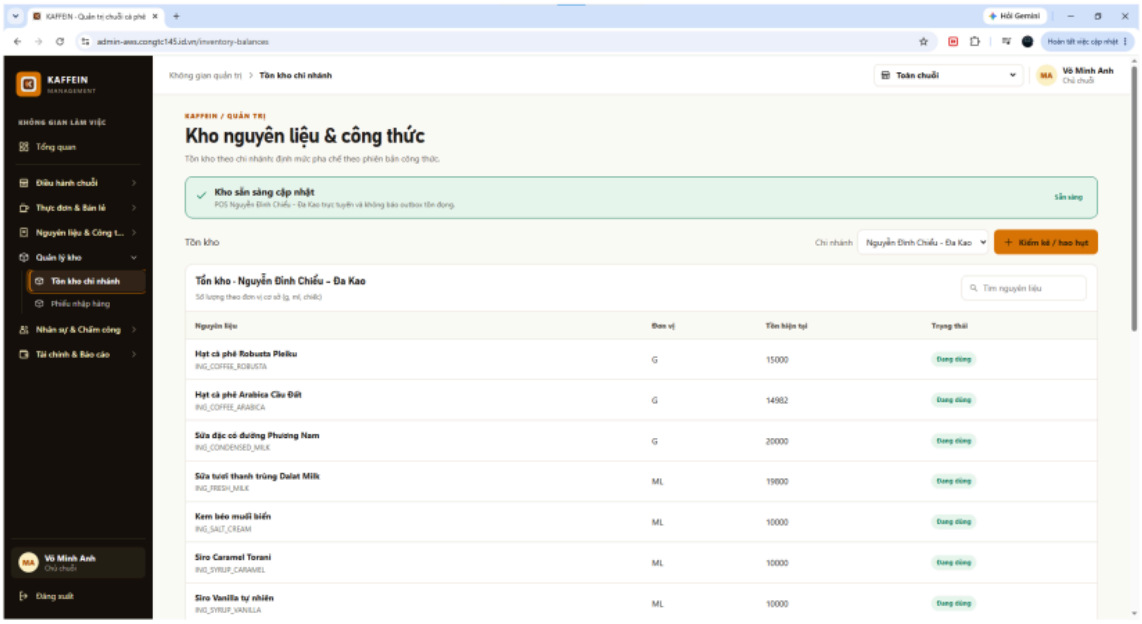
4.7 Nhập nguyên liệu và điều kiện cập nhật tồn

Trang tồn kho hiển thị số dư theo chi nhánh và đơn vị cơ sở. Ở Hình 27, trạng thái sẵn sàng được giải thích bằng heartbeat của POS và thông tin không còn outbox tồn đọng. Đây là trạng thái được máy chủ suy ra gần nhất. Nó hỗ trợ quyết định đăng kho nhưng chưa tạo một khóa phân tán ngăn POS bán cục bộ đồng thời.[51]

[49]Nguồn đối chiếu: apps/api/src/employees/employees.controller.ts; apps/api/src/employees/employees.service.ts, transferEmployee; docs/02-business-rules.md

[50]Nguồn đối chiếu: apps/api/src/attendance/attendance.controller.ts; apps/api/src/attendance/attendance.service.ts

[51]Nguồn đối chiếu: apps/web/src/App.tsx; apps/api/src/inventory/inventory.service.ts; docs/adr/0006-branch-receiving-sources-and-draft.md



Hình 27. Số dư nguyên liệu của một chi nhánh và thông báo POS sẵn sàng cập nhật kho trên Web

Phiếu nhập có thể được lưu nháp khi Web còn kết nối API nhưng POS ngoại tuyến. Chỉ khi đăng kho, service kiểm tra dữ liệu dòng, phạm vi quản lý, heartbeat, lần đồng bộ và outbox. Trong transaction PostgreSQL, service khóa chi nhánh, kiểm tra QR đang chờ, tăng số dư, ghi movement, delta đồng bộ và trạng thái POSTED. Gửi lại cùng khóa của phiếu đã đăng trả kết quả cũ; một yêu cầu đăng lại không phù hợp bị từ chối. Thông tin nguồn kho tổng chỉ mô tả xuất xứ hàng nhận, chưa có nghiệp vụ xuất kho tổng hoặc chuyển kho đối ứng.[52]

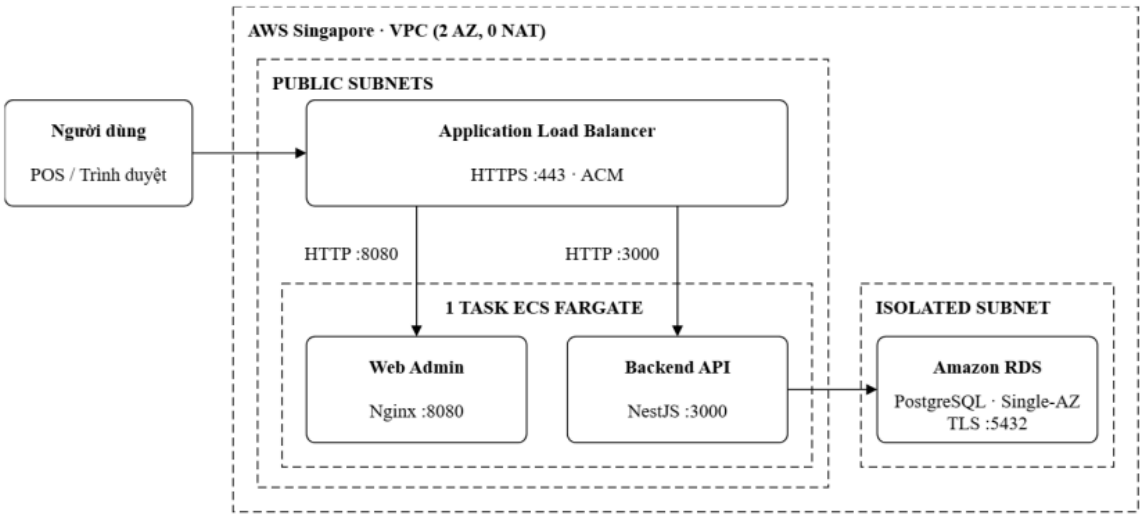
4.8 Triển khai AWS và phát hành ứng dụng

4.8.1 Vị trí thành phần và kết nối

Cấu hình CDK đặt ALB và task ứng dụng trong các public subnet; RDS đặt trong isolated subnet. Web được build thành React SPA và phục vụ bằng Nginx cổng 8080, chạy cùng API NestJS cổng 3000 trong một task Fargate. ALB tiếp nhận HTTPS, kết thúc TLS bằng chứng chỉ ACM rồi định tuyến theo host tới Web hoặc API. Cloudflare chỉ phân giải DNS. Phương án này được chốt trong ADR 0013 và thay phần Web CloudFront của phương án trước.[53]

[52]Nguồn đối chiếu: apps/api/src/stock-receipts/stock-receipts.service.ts; apps/web/src/StockReceipts.tsx; docs/adr/0006-branch-receiving-sources-and-draft.md

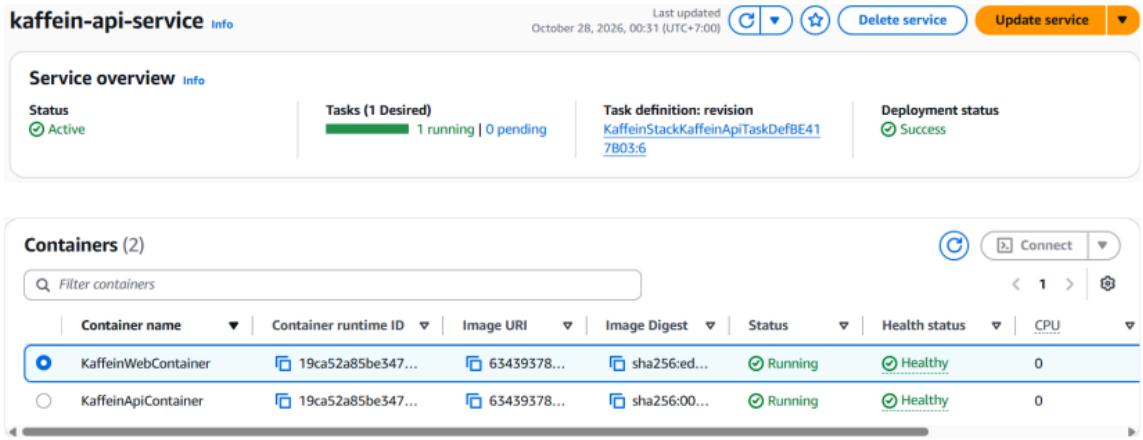
[53]Nguồn đối chiếu: infra/lib/kaffein-stack.ts; apps/web/Dockerfile; apps/web/nginx.conf; docs/adr/0013-web-admin-ecs-alb.md



Hình 28. Bố trí ALB, task Web/API và RDS trong cấu hình AWS của hệ thống

VPC có subnet tại hai AZ nhưng cấu hình thường trực chỉ có một task ứng dụng và RDS Single-AZ. Task dùng public IP để đi Internet qua Internet Gateway, nhờ đó không cần NAT Gateway. Security group giới hạn hai cổng ứng dụng chỉ nhận từ ALB; RDS không public và chỉ nhận PostgreSQL từ nhóm của ECS. Sự lựa chọn này giảm thành phần phải vận hành, đồng thời giữ Web và API chung vòng đời triển khai, chung ngân sách tài nguyên và cùng chịu ảnh hưởng khi task khởi động lại.[54]

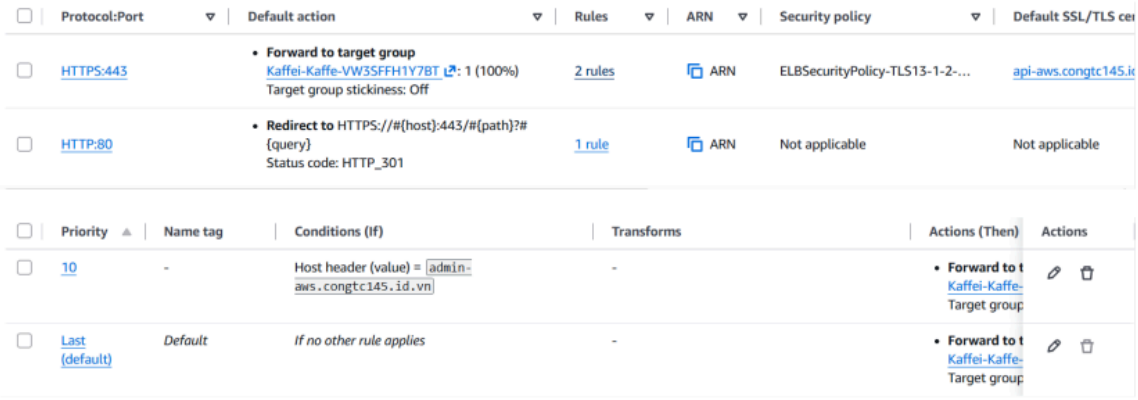
Ảnh Console tại Hình 29 ghi nhận service `kaffein-api-service` đang Active, với một task chạy, không có task chờ và deployment Success. Màn hình được cập nhật lúc 00:31 ngày 28/10/2026, UTC+07. Task sử dụng revision 6; hai container `KaffeinApiContainer` và `KaffeinWebContainer` đều Running, Healthy.



Hình 29. Trạng thái ECS service và hai container Web/API trong cùng task, revision 6

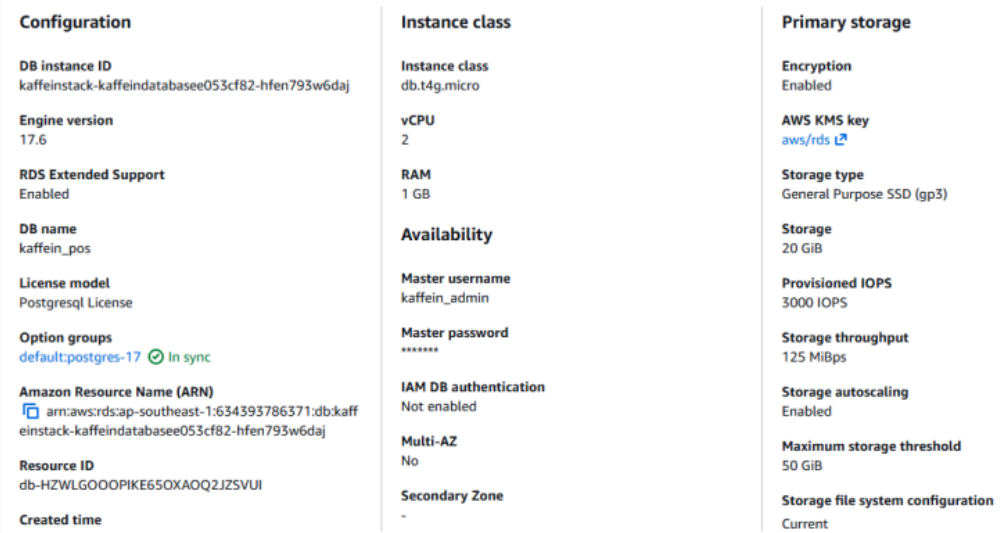
ALB `kaffein-alb` có hai listener như Hình 30. HTTP cổng 80 chuyển hướng sang HTTPS cổng 443 bằng mã 301. Trên listener 443, rule ưu tiên 10 nhận host `admin-aws.congtc145.id.vn` và chuyển tới Web; default action chuyển tới API. Hai target được đăng ký cùng địa chỉ IP của task, lần lượt ở cổng 3000 và 8080; cả hai hiển thị Healthy trong Hình 44.

[54]Nguồn đối chiếu: infra/lib/kaffein-stack.ts; docs/adr/0012-aws-demo-cdk.md; docs/adr/0013-web-admin-ecs-alb.md



Hình 30. Listener HTTP/HTTPS của ALB và rule định tuyến theo host Web Admin

RDS chạy PostgreSQL 17.6 trên `db.t4g.micro`, có hai vCPU và 1 GB RAM. Hình 31 cho thấy lưu trữ gp3 20 GiB được mã hóa, tự tăng tối đa 50 GiB. Multi-AZ đang tắt. Phần kết nối trong Hình 42 ghi cổng 5432, AZ `ap-southeast-1b` và `Publicly accessible: No`. Thời gian giữ backup tự động một ngày được khai báo trong CDK; trường này không có trong ảnh cấu hình.[55]



Hình 31. PostgreSQL 17.6 trên RDS Single-AZ, lưu trữ được mã hóa và giới hạn tự tăng 50 GiB

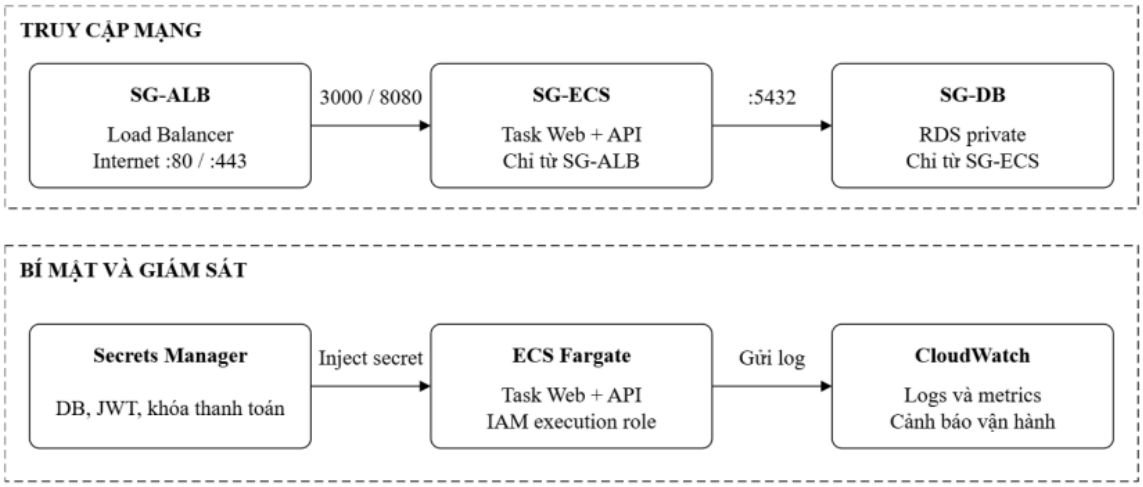
VPC có hai public subnet và hai subnet DatabaseIsolated tại `ap-southeast-1a` và `ap-southeast-1b`. Resource map thể hiện tuyến Internet của hai public subnet; hai subnet cơ sở dữ liệu chỉ có tuyến nội bộ. Các ảnh VPC, kết nối RDS và inbound rules được tập hợp ở Phụ lục E. Ràng buộc mạng được đối chiếu với CDK theo chuỗi ALB → ECS cổng 3000/8080 → RDS cổng 5432.

4.8.2 Bí mật, giám sát và trình tự phát hành

Credential cơ sở dữ liệu, JWT và khóa cổng thanh toán được inject vào API từ Secrets Manager theo quyền execution role. Image Web không chứa các giá trị này. Kết nối API tới RDS dùng TLS có kiểm tra chứng chỉ; lưu trữ RDS được mã hóa. CloudWatch tiếp

[55] Nguồn đối chiếu: infra/lib/kaffein-stack.ts, KaffeinDatabase; thesis/assets/evidence/aws-screenshots-20261028.json

nhận log, metric và cảnh báo vận hành. Sơ đồ Hình 32 tách luồng quyền mạng khỏi luồng cấp secret và gửi log để thể hiện mỗi cơ chế kiểm soát tại vị trí thực thi tương ứng.[56]



Hình 32. Quyền truy cập giữa các security group và quan hệ Secrets Manager, ECS, CloudWatch

Pipeline GitHub Actions dùng OIDC để nhận quyền phát hành, tạo image tag theo commit và lượt chạy. Sau quality gate, tooling chạy migration trước khi đổi image API/Web; các bước smoke kiểm tra bản vừa phát hành. Khi rollout hoặc smoke thất bại, cơ chế phục hồi đổi lại image cũ, không tự đảo migration. Môi trường test dùng PostgreSQL riêng và transport provider giả lập, nên kết quả CI không được coi là một lượt thanh toán ví thật.[57]

POS gọi AWS bằng package riêng, tách Room, preferences và Android Keystore khỏi bản local. Bản AWS cần kích hoạt mới với backend AWS và giữ hàng đợi đồng bộ riêng. Hồ sơ bàn giao ghi nhận APK debug; ký bản phát hành và phân phối APK tự động còn chưa hoàn tất.[58]

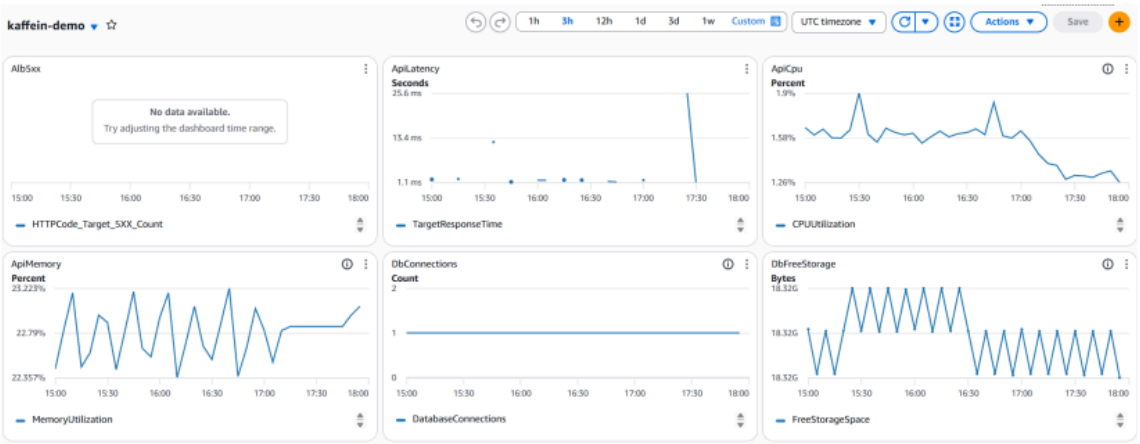
Dashboard `kaffein-demo` hiển thị độ trễ target ALB, CPU/bộ nhớ của ECS service và các chỉ số RDS trong cửa sổ ba giờ. Dù widget có tên `ApiLatency`, metric lấy từ ALB và không lọc riêng target group API. CPU và bộ nhớ cũng thuộc service chứa cả hai container. Biểu đồ `Alb5xx` đang hiển thị "No data available"; các đồ thị còn lại có dữ liệu. Hình 33 minh họa phạm vi giám sát đang cấu hình.[59]

[56]Nguồn đối chiếu: infra/lib/kaffein-stack.ts; apps/api/container-bootstrap.cjs; docs/adr/0012-aws-demo-cdk.md

[57]Nguồn đối chiếu: .github/workflows/ci.yml; .github/workflows/deploy.yml; scripts/aws-release.mjs; docs/adr/0014-github-oidc-releases-and-pos-aws.md

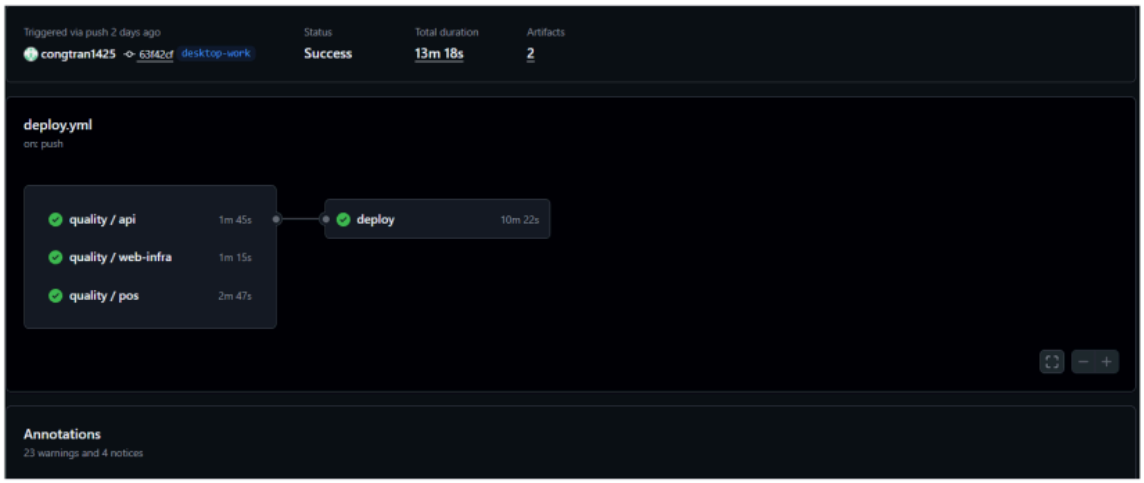
[58]Nguồn đối chiếu: apps/pos/app/build.gradle.kts; docs/adr/0014-github-oidc-releases-and-pos-aws.md; docs/handoffs/49-aws-cicd-and-pos-aws.md; docs/handoffs/50-employee-code-rds-correction.md

[59]Nguồn đối chiếu: infra/lib/kaffein-stack.ts, metrics/Dashboard



Hình 33. Dashboard CloudWatch của ALB, ECS service và RDS trong cửa sổ ba giờ UTC

Ảnh Hình 34 ghi nhận run `40222286963` của commit `63f42cf` trên nhánh `desktop-work` hoàn tất trong 13 phút 18 giây. Các job API, Web/hạ tầng, POS và deploy đều thành công. Giao diện còn ghi 23 cảnh báo và bốn thông báo; trạng thái Success là kết quả của workflow.



Hình 34. Lượt phát hành AWS thành công của commit 63f42cf trên nhánh desktop-work

CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ

5.1 Phạm vi dữ liệu và cách đánh giá

Việc đánh giá tập trung vào các tình huống có thể làm sai dữ liệu bán hàng: mất kết nối sau khi thu tiền mặt, gửi lặp thao tác, thay đổi công thức sau thời điểm bán, xác nhận QR đến muộn và truy cập sai chi nhánh. Với mỗi tình huống, dữ liệu trước và sau xử lý được đối chiếu qua hóa đơn, khoản thanh toán, lượng tiêu hao, biến động kho và operation đồng bộ.

Dữ liệu đánh giá gồm mã kiểm thử, cấu hình CI và hồ sơ thực thi ngày 23/10/2026. Log gốc ghi nhận kết quả API và Web; các tệp trạng thái phát hành liên kết commit, image tag, task revision và kết quả kiểm tra sau triển khai. Những hồ sơ này được đối chiếu lại ngày 28/10/2026, không kèm một lượt nghiệm thu mới trên ứng dụng hoặc hạ tầng. Danh mục đã lọc thông tin nhạy cảm lưu đường dẫn, SHA-256 và kết quả đọc từ từng hồ sơ.[60]

Bộ ảnh Console và GitHub tiếp nhận ngày 28/10 bổ sung trạng thái hạ tầng cùng một lượt phát hành khác. ECS ghi thời điểm cập nhật trực tiếp trên ảnh; log giữ timestamp UTC, còn ảnh GitHub chỉ ghi thời gian tương đối. Các mốc được lưu riêng trong danh mục ảnh, không thay ngày của hồ sơ kiểm thử 23/10.[61]

Đơn vị đánh giá là từng thuộc tính nghiệp vụ. Một bài kiểm thử có thể chứa nhiều phép khẳng định; nhiều bài cũng có thể cùng bao phủ một nhánh. Số bài đạt được trình bày riêng theo loại kiểm thử, môi trường và phiên bản, không cộng thành một chỉ số độ bao phủ chung.

5.2 Môi trường và tổ chức kiểm thử

Workflow CI tách công việc API, Web/hạ tầng và Android. Công việc API dùng Node.js 24, PostgreSQL 17 trong service container riêng, cài dependency, sinh Prisma Client, kiểm tra lint, chạy unit test, build, áp migration và chạy integration test. Công việc Web kiểm tra lint, test và build; cùng nhánh công việc này kiểm tra CDK, synth template và các guard phát hành. Android dùng JDK 21 để chạy JVM test và build APK debug. Việc tách database CI khỏi database vận hành giúp các bài tích hợp có thể tạo dữ liệu thử mà không tác động đến dữ liệu cửa hàng.[62]

Các bài JVM kiểm tra hàm tính tiền, số lượng nguyên liệu, giá trị giỏ, chính sách truy cập và trạng thái thanh toán. Các bài instrumentation nằm trong thư mục `androidTest`, sử dụng Room hoặc server thử để kiểm tra hợp đồng dữ liệu. Workflow đang xét chạy `testDebugUnitTest` và `assembleDebug`; nó không chạy bộ instrumentation trên emulator. Vì vậy một job Android thành công chưa xác nhận các tình huống Room, khởi động lại ứng dụng hoặc tương tác Compose đã được chạy trong cùng job.

[60] Nguồn đối chiếu: thesis/assets/evidence/review-20261028.json; docs/handoffs/49-aws-cicd-and-pos-aws.md; docs/handoffs/50-employee-code-rds-correction.md

[61] Nguồn đối chiếu: thesis/assets/evidence/aws-screenshots-20261028.json

[62] Nguồn đối chiếu: .github/workflows/ci.yml, các job api, web-infra và pos.

Với thanh toán điện tử, CI dùng transport giả lập theo ADR 0014 để chủ động tạo phản hồi timeout, sai số tiền hoặc xác nhận đến muộn. Trong log tích hợp ngày 23/10, hai bài tùy chọn bị bỏ qua: một bài tạo và truy vấn QR MoMo Sandbox, một bài diễn tập Android Room với API thật. Bài MoMo cũng chỉ tạo và truy vấn, chưa thực hiện trả tiền qua ví. Hai bài này vẫn được ghi là "skipped".[63]

5.3 Kết quả kiểm thử tại mốc phát hành

Đợt phát hành ngày 23/10/2026 sử dụng commit `0666abf`, gắn với GitHub Actions run `39893576810`. Bảng 17 tổng hợp kết quả từ log kiểm thử và hồ sơ phát hành đã lưu. Các log cục bộ không tự nhúng commit; việc đối chiếu dựa trên cùng thư mục đợt phát hành và metadata CI/CD, như ghi trong danh mục bằng chứng.

Bảng 17. Kết quả kiểm thử và phát hành ngày 23/10/2026, đối chiếu lại từ hồ sơ gốc.

Hạng mục	Kết quả	Bằng chứng và phạm vi
API unit test	164 bài đạt	Log ghi 20 bộ kiểm thử đạt; bao gồm các quy tắc dịch vụ và phân quyền.
API integration	63 đạt; 2 bỏ qua	7 bộ kiểm thử đạt. Hai bài tùy chọn chưa chạy: Mo Mo Sandbox và diễn tập Android Room.
Web	11 bài đạt; build đạt	Log ghi 11 bài đạt, không có bài lỗi; build ứng dụng hoàn tất.
Phát hành AWS	4 job thành công	API, Web/hạ tầng, POS và deploy đạt. Hồ sơ cuối ghi task revision 5, một task chạy và hai target healthy.

Log Android xác nhận `testDebugUnitTest` đã thực thi và `assembleDebug` hoàn tất. Do log không in tổng số bài và chưa có XML tương ứng, bảng không ghi số bài JVM. Một đợt phát hành trước đó trong cùng ngày, run `39888131251`, có 72 mẫu health từ 02:19:16 đến 02:25:54 UTC: 71 mẫu trả HTTP 200 với database hợp lệ, một mẫu timeout tại client sau 5 giây. Hồ sơ chưa xác định nguyên nhân timeout. Khoảng quan sát này chỉ mô tả đáp ứng tại các thời điểm lấy mẫu, chưa đủ để tính SLA hoặc độ sẵn sàng dài hạn.[64]

5.4 Quan sát bổ sung từ bộ ảnh AWS

Bảng 18 tổng hợp dữ liệu đọc trực tiếp từ ảnh. Revision 6 và run `40222286963` bổ sung hồ sơ triển khai; số bài kiểm thử trong Bảng 17 vẫn thuộc đợt trước. Ảnh Console không hiển thị tag image đầy đủ để liên kết độc lập task đang chạy với commit của run GitHub.

[63] Nguồn đối chiếu: thesis/assets/evidence/review-20261028.json; apps/api/test/momo-polling.e2e-spec.ts; apps/api/test/catalog-receiving-rehearsal.e2e-spec.ts

[64] Nguồn đối chiếu: thesis/assets/evidence/review-20261028.json; docs/handoffs/49-aws-cicd-and-pos-aws.md

Bảng 18. Trạng thái hạ tầng và phát hành được ghi nhận trong bộ ảnh tiếp nhận ngày 28/10/2026

Đối tượng	Quan sát từ ảnh
ECS	Service Active; một task chạy, không có task chờ; revision 6; Web/API đều Running và Healthy.
ALB	Hai target cùng IP task, cổng 3000/8080; mỗi nhóm có một target Healthy.
RDS và mạng	PostgreSQL 17.6, Single-AZ, mã hóa; không public; VPC có bốn subnet tại hai AZ.
CloudWatch	Các metric ALB/ECS/RDS có dữ liệu trong khung ba giờ; widget 5xx chưa có dữ liệu.
Nhật ký API	GET / trả 404 lúc 17:30:53.592 UTC ngày 27/10/2026; route health của ứng dụng là /health.
GitHub Actions	Run 40222286963, commit 63f42cf, desktop-work; bốn job thành công; thời gian 13 phút 18 giây.

Nhật ký ở Hình 45 ghi request tới đường dẫn gốc chưa được khai báo. Dữ liệu này cho thấy API đã gửi log lên CloudWatch; việc kiểm tra sức khỏe sử dụng `/health`. Các đồ thị chưa đi kèm tải thử và tập số đo nên chỉ được dùng để mô tả giám sát.[65]

5.5 Đánh giá các cơ chế giữ nhất quán

5.5.1 Truy vết từ yêu cầu đến phép kiểm chứng

Bảng 19 liên kết yêu cầu ở Bảng 3 với use case, thành phần hiện thực và phép kiểm chứng. Mã AC giữ theo đề cương; trạng thái nghiệm thu được trình bày ở Bảng 20 và Bảng 21. Tiền điều kiện, dữ liệu đầu vào và hậu kiểm của quy trình bổ sung nằm ở Phụ lục D.

Bảng 19. Truy vết yêu cầu, use case và phép kiểm chứng trong mã nguồn

Yêu cầu	Use case	Nguồn hiện thực	Phép kiểm chứng và tiêu chí
YC-01	UC-01	DevicesService; partial unique index	devices.service.spec.ts; foundation.e2e-spec.ts; kiểm tra enrollment và ràng buộc thiết bị ACTIVE cùng chi nhánh.
YC-02	UC-02	AttendanceService; Cashier SessionsService	Unit tests hai service; AC-07 và AC-08 đối chiếu attendance hỗ trợ với cashier bị chặn.
YC-03	UC-03	CashSaleRepository; Inventory SalePlanner	PosDataContractTest; PosServerRehearsal Test; AC-01, AC-02 và AC-05 trên APK đích.
YC-04	UC-07	SyncService; OutboxSync Repository	sync-sales-integration.e2e-spec.ts; AC-03 gửi ít nhất 10 lần; AC-04 đối chiếu dữ liệu nguyên tử.
YC-05	UC-04	PaymentsService; OnlinePayment Repository	momo-polling.e2e-spec.ts; OnlinePayment ContractTest; AC-09, tách mô phỏng khỏi ví Sandbox thật.
YC-06	UC-05	StockReceiptsService	catalog-receiving-rehearsal.e2e-spec.ts; nháp không đổi tồn, retry đăng không cộng lặp.
YC-07	UC-06	RegisterCashRepository; Register SessionsService	register-compensation.e2e-spec.ts; Register CashContractTest; giữ variance gốc và nhận nguồn một lần.

[65]Nguồn đối chiếu: apps/api/src/main.ts; apps/api/src/health/health.controller.ts; thesis/assets/evidence/aws-screenshots-20261028.json

Các tên lớp trong bảng được định vị ở Phụ lục B. Recipe snapshot của YC-03 và YC-04 còn liên hệ AC-06; triển khai AWS được đánh giá bằng AC-10 với hồ sơ phát hành và kiểm tra sau triển khai.[66]

5.5.2 Bán tiền mặt, retry và snapshot kho

Bài tích hợp bán tiền mặt thiết lập tồn nguyên liệu 1.000 g, định lượng 20 g mỗi ly và số lượng bán hai ly. Các phép khẳng định kiểm tra sale có trạng thái `PAID`, payment đã trả và số dư còn 960 g. Gửi lại cùng operation nhận `ALREADY_PROCESSED`, số dư vẫn 960 g. Số dư sau lần gửi lại là dữ liệu kiểm tra việc giao dịch không trừ kho lần hai.[67]

Một bài khác gửi đồng thời hai request cùng `operationId` và kiểm tra tập kết quả gồm `COMPLETED` và `ALREADY_PROCESSED`. Bài này kiểm tra cạnh tranh khi hai request cùng đến; bài gửi lại tuần tự kiểm tra trường hợp request đầu đã kết thúc. Cả hai chưa đáp ứng riêng ngưỡng ít nhất mười lượt gửi của AC-03. Quy trình bổ sung ở Phụ lục D giữ nguyên ID và payload qua mười lượt để đối chiếu số sale và movement.

Tính bền của snapshot được kiểm tra bằng tình huống công thức trên server thay đổi sau thời điểm POS ghi nhận sale. Bộ tích hợp chứa các bài kiểm tra recipe version, consumption, giá và VAT lịch sử. Mục tiêu là giữ dữ liệu đã dùng cho giao dịch cũ, đồng thời từ chối consumption âm hoặc không thuộc nguyên liệu hợp lệ. Điều kiện này giúp phân biệt đồng bộ một giao dịch đã phát sinh với tính lại một giao dịch theo danh mục mới. Snapshot giữ được lịch sử chỉ khi cả bước tạo local và bước kiểm tra server tuân thủ cùng hợp đồng dữ liệu.[68]

Ở tầng Room, `PosDataContractTest` kiểm tra việc giữ operation ID, chặn reset catalog khi outbox chưa sạch và giữ catalog cùng cursor cũ nếu bootstrap lỗi. `OnlinePaymentContractTest` kiểm tra attempt tồn tại khi khởi tạo lại repository. Các bài này thuộc instrumentation, chưa có hồ sơ chạy tương ứng trong đợt CI đã đối chiếu.

Bảng 20. Đối chiếu tiêu chí ngoại tuyến và kho với bằng chứng kiểm thử hiện có.

Tiêu chí	Đối tượng kiểm chứng	Kết quả và phần còn thiếu
AC-01, AC-02	Cash offline và phục hồi sau khi mở lại	Đã hiện thực Room/outbox; chưa có hồ sơ chuỗi ngắt mạng và khởi động lại trên APK AWS.
AC-03	Gửi cùng operation ít nhất 10 lần	Test retry và hai request đồng thời thuộc bộ tích hợp đạt; chưa thử đủ 10 lượt.
AC-04	Sale, consumption, movement và balance nguyên tử	Có kiểm tra dữ liệu sau commit; chưa có hồ sơ gây lỗi giữa các bước ghi và hậu kiểm rollback đầy đủ ở API/ Room.
AC-05	Từ chối khi thiếu nguyên liệu local	Đã hiện thực kiểm tra tồn; chưa có hồ sơ thao tác và hậu kiểm trên APK AWS.
AC-06	Công thức mới không sửa consumption cũ	Test snapshot lịch sử thuộc bộ tích hợp API đạt; chưa có hồ sơ trọn luồng POS–AWS.

[66]Nguồn đối chiếu: docs/09-proposal.md, mục 2.8.2; apps/api/test/; apps/api/src/attendance/attendance.service.spec.ts; apps/api/src/cashier-sessions/cashier-sessions.service.spec.ts; apps/pos/app/src/androidTest/java/vn/id/congtc145/coffeepos/

[67]Nguồn đối chiếu: apps/api/test/sync-sales-integration.e2e-spec.ts, bài Push CASH_SALE_COMPLETED; thesis/assets/evidence/review-20261028.json

[68]Nguồn đối chiếu: apps/api/test/sync-sales-integration.e2e-spec.ts, các bài Offline Cash Sale with recipeVersionId, POS consumptions validation và Offline sale price & VAT snapshot.

5.5.3 Thanh toán QR khi lỗi hoặc hủy

`momo-polling.e2e-spec.ts` có các ca mất kết nối, request tạo lặp, query sai số tiền, hủy đua với xác nhận và lỗi ghi sổ sau khi đã lưu xác nhận thanh toán. Trạng thái không xác định giữ giao dịch để đối soát. Với MoMo, hủy nội bộ không vô hiệu QR tại cổng; bài IPN muộn kiểm tra việc giữ giao dịch đã hủy, lưu xác nhận đúng một lần và không thay đổi kho.

Ở POS, test hợp đồng kiểm tra `PAID_RECONCILING` được giữ nếu pull kho thất bại. Lần thử lại chỉ hoàn tất sau khi biên lai và snapshot kho đã lưu hợp lệ, đồng thời không tạo outbox tiền mặt hoặc trừ kho QR lần nữa. Biên lai sai danh tính phải giữ nguyên giỏ. Đây là các ca instrumentation có trong mã nguồn; chúng và lượt thanh toán bằng ví Sandbox trên AWS chưa có hồ sơ thực thi trong đợt đánh giá.[69]

5.5.4 Chi nhánh, quyền thu ngân và kiểm đếm

Hồ sơ kiểm tra giao diện và API ngày 23/10 ghi nhận Owner đọc được bốn nhân viên, Manager chi nhánh 1 chỉ đọc được hai nhân viên trong phạm vi; request ép chi nhánh 2 bị từ chối. Các bài kiểm thử của sale và phiên quỹ cũng xét trường hợp `homeStoreId` khác chi nhánh thiết bị. Quy trình AC-07 kiểm tra riêng cặp hành vi chấm công hỗ trợ được phép và đăng nhập thu ngân trái chi nhánh bị chặn.[70]

Bộ `register-compensation.e2e-spec.ts` dùng số theo sổ 60.000 đồng, thực đếm 25.000 đồng và tiền bù 50.000 đồng. Kết quả kỳ vọng giữ chênh lệch gốc âm 35.000 đồng, trong khi phiên nhận có 75.000 đồng. Gửi lại operation không tạo audit đóng ca lần hai; nhận bàn giao lặp không tạo thêm phiên từ cùng nguồn. Các giá trị thử này kiểm tra việc lưu đồng thời khoản thiếu ban đầu và lượng tiền thực tế bàn giao.[71]

Bảng 21. Đối chiếu tiêu chí phân quyền, QR và triển khai với giới hạn bằng chứng.

Tiêu chí	Phép kiểm chứng	Giới hạn kết luận
AC-07	Attendance hỗ trợ được phép; cashier sai home store bị chặn	Các service có unit test; còn thiếu hồ sơ cặp thao tác trên cùng POS và bộ dữ liệu nghiệm thu.
AC-08	Request giả store, truy cập chéo chi nhánh	Hồ sơ ngày 23/10 xác nhận request nhân sự trái scope bị chặn; phạm vi chưa bao gồm mọi endpoint.
AC-09	Offline, chữ ký sai, callback lặp và đối soát QR	Bộ API dùng transport giả lập; bài tạo/truy vấn Mo Mo Sandbox bị bỏ qua, chưa có lượt trả tiền qua ví.
AC-10	Build, health, log và phát hành AWS	Có hồ sơ ngày 23/10 và ảnh Console/GitHub bổ sung; nghiệm thu nghiệp vụ trên APK AWS còn thiếu dữ liệu.

5.6 Giới hạn đánh giá

Hồ sơ ngày 23/10 cung cấp log kiểm thử API/Web và trạng thái phát hành, nhưng chưa bao gồm đầy đủ chuỗi nghiệm thu trên APK AWS cho AC-01 đến AC-09. Phần còn thiếu

[69]Nguồn đối chiếu: apps/api/test/momo-polling.e2e-spec.ts; apps/pos/app/src/androidTest/java/vn/id/congtc145/coffeepos/OnlinePaymentContractTest.kt; docs/adr/0010-momo-local-cancellation.md

[70]Nguồn đối chiếu: thesis/assets/evidence/review-20261028.json; apps/api/src/sales/sales.service.spec.ts; apps/api/src/register-sessions/register-sessions.service.spec.ts; docs/02-business-rules.md

[71]Nguồn đối chiếu: apps/api/test/register-compensation.e2e-spec.ts, bài requires full compensation; docs/adr/0011-blind-cash-count-compensation-handover.md

gồm thao tác ngoại tuyến và khởi động lại, hậu kiểm sale/movement theo ID, thanh toán ví Sandbox có đối chiếu và nhận bàn giao quỹ trên cùng phiên bản ứng dụng.

PostgreSQL CI và transport cổng thanh toán giả lập cho phép tạo lỗi có kiểm soát. Chúng chưa phản ánh đầy đủ thiết bị bán hàng, đường truyền cửa hàng và ứng dụng ví. Mô hình tồn cục bộ cũng chỉ xét một POS đang hoạt động tại mỗi chi nhánh; cho phép nhiều POS cùng bán sẽ làm thay đổi giả định điều phối tồn.

Đề tài chưa có tập dữ liệu đo độ trễ, thông lượng, mức dùng tài nguyên hoặc hóa đơn cloud đủ để tính chi phí tháng. Phân tích chi phí cần xét region, thời gian chạy, ALB, IPv4, lưu trữ, backup và secret. Việc bỏ NAT Gateway giảm một loại tài nguyên; mức tiết kiệm và chi phí sau khi hết credit còn chưa được đo.

Đề tài chưa tổ chức khảo sát người dùng cửa hàng. Các ảnh giao diện minh họa màn hình và trạng thái vận hành, chưa cung cấp số đo thời gian học thao tác hoặc mức hài lòng. Đánh giá giao diện cần một nghiên cứu tác vụ riêng, có người tham gia và phương pháp ghi nhận kết quả.

Một giới hạn chức năng nằm ở nghiệp vụ điều chuyển nhân viên: `transferEmployee` đã ghi lịch sử và thu hồi phiên trên máy chủ, nhưng chưa kiểm tra POS cũ trực tuyến, chi nhánh đích ACTIVE hoặc phát roster delta ngay trong phương thức. Vì POS giữ dữ liệu cục bộ, đổi chi nhánh trên PostgreSQL chưa bảo đảm thu hồi ngay quyền ở thiết bị ngoại tuyến. Nghiệm thu nghiệp vụ này cần bổ sung kiểm tra điều kiện và thử cập nhật danh sách nhân viên tại POS cũ.[72]

[72]Nguồn đối chiếu: docs/02-business-rules.md; apps/api/src/employees/employees.service.ts, transferEmployee

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được trong phạm vi đề tài

Đề tài đã xây dựng nguyên mẫu POS cho một chuỗi cà phê nhiều chi nhánh, gồm Android POS, API NestJS, Web quản trị và cấu hình triển khai AWS. Thiết kế giữ quan hệ giữa giao dịch đã thu tiền, công thức đã dùng, số dư kho và nguồn phát sinh thao tác khi kết nối gián đoạn. Các kết quả dưới đây trả lời năm câu hỏi nghiên cứu trong phạm vi kiểm thử ở Mục 5.

Room cung cấp dữ liệu cho giao diện và ghi hóa đơn tiền mặt, tiêu hao, biến động kho cùng outbox trong transaction cục bộ. Cơ chế này trả lời RQ1 ở phạm vi bán tiền mặt sau khi tải danh mục, xác thực thu ngân và mở quỹ. Kích hoạt thiết bị, chấm công, đăng nhập lại và thanh toán điện tử vẫn cần mạng.

Với RQ2, outbox giữ định danh và nội dung qua các lần gửi; máy chủ kiểm tra thiết bị và kết quả operation đã lưu để xử lý lặp. Hàng đợi giữ thứ tự mở, bán và đóng phiên quỹ. Các bài retry và hai request đồng thời thuộc bộ tích hợp API đạt; ngưỡng mười lượt gửi của AC-03 và phục hồi trọn luồng trên APK AWS còn chưa được xác nhận.

Recipe version, snapshot tiêu hao và sổ biến động trả lời RQ3 bằng cách giữ căn cứ tính kho của giao dịch lịch sử. Thay đổi thực đơn không tính lại hóa đơn đã trả tiền. Số dư thu được là tồn lý thuyết theo định mức; hao hụt, sai số pha chế và lượng nguyên liệu vật lý cần được đối chiếu bằng kiểm kê.

Việc tách kích hoạt thiết bị, chấm công và phiên thu ngân xác định các ngữ cảnh quyền của RQ4. Máy chủ suy ra hoặc kiểm tra chi nhánh theo tài khoản và thiết bị đã xác thực. Hồ sơ ngày 23/10 ghi nhận Manager bị chặn khi yêu cầu nhân sự ngoài phạm vi. Giới hạn tại luồng điều chuyển và các endpoint chưa thử được giữ trong phần đánh giá.

RQ5 được hiện thực bằng CDK và pipeline từ kiểm tra chất lượng, tạo image, migration đến triển khai và kiểm tra dịch vụ. Hồ sơ phát hành cùng bộ ảnh AWS ghi nhận Web/API hoạt động trên cùng task và có dữ liệu giám sát. Một task Fargate dùng chung cho Web/API và RDS Single-AZ giảm số đơn vị vận hành, đồng thời giữ các điểm lỗi đơn và gắn vòng đời hai ứng dụng. Khả năng sẵn sàng cao và chi phí tháng chưa được đánh giá định lượng.[73]

6.2 Các giới hạn còn lại

MVP giữ phạm vi một chuỗi, một POS hoạt động mỗi chi nhánh, hóa đơn đã trả tiền bất biến và chưa hỗ trợ hoàn tiền. MoMo hủy nội bộ vẫn có thể nhận tiền muộn; hệ thống lưu xác nhận để quản lý đối soát, không tự hoàn tiền hoặc ghi lại doanh thu. Kiểm đếm kín giới hạn việc tiết lộ số theo sổ trong giao diện; dữ liệu vẫn có thể được đọc bởi người có quyền truy cập trực tiếp cơ sở dữ liệu thiết bị.

Phần đánh giá đã đối chiếu log API/Web, hồ sơ phát hành ngày 23/10 và bộ ảnh hạ tầng, CI/CD tiếp nhận ngày 28/10. Các phần còn cần bổ sung gồm nghiệm thu cash/QR/

[73] Nguồn đối chiếu: docs/09-proposal.md, RQ1–RQ5; docs/adr/0002-one-active-pos-per-store.md; docs/adr/0013-web-admin-ecs-alb.md; thesis/assets/evidence/review-20261028.json; thesis/assets/evidence/aws-screenshots-20261028.json

quỹ trên APK AWS, kiểm chứng phục hồi bản sao lưu và phép đo hiệu năng, chi phí, khả dụng. Đánh giá người dùng cần dữ liệu khảo sát riêng.

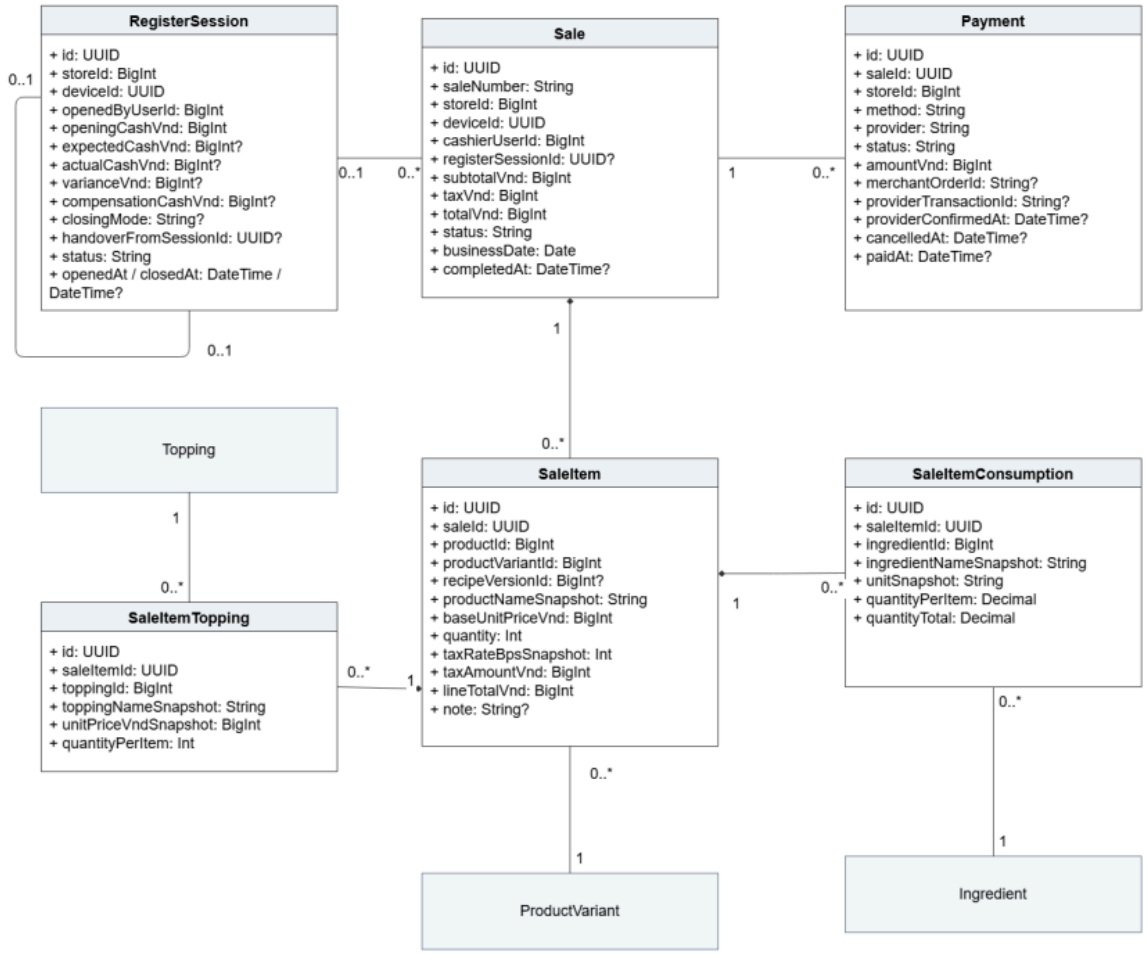
6.3 Hướng phát triển

Công việc tiếp theo gồm hoàn tất AC-01 đến AC-10 trên APK AWS, lưu dữ liệu hậu kiểm theo mã giao dịch, ký APK với khóa phát hành được quản lý và thử phục hồi bản sao lưu trong môi trường tách biệt. Các phép đo thời gian đồng bộ, độ trễ tương tác và gián đoạn dịch vụ ngoài cần ghi rõ tải thử, cấu hình và khoảng quan sát. Khảo sát tác vụ tại cửa hàng cần xác định người tham gia, kịch bản và cách đo.

Nhiều POS tại một cửa hàng, chuyển kho, quản lý lô/hạn sử dụng và hoàn tiền là những mở rộng ngoài MVP, cần ADR cùng thiết kế dữ liệu và kiểm thử riêng. Nhiều POS làm thay đổi giả định điều phối tồn ngoại tuyến; hoàn tiền bổ sung giao dịch bù vào vòng đời hóa đơn và sổ tiền.

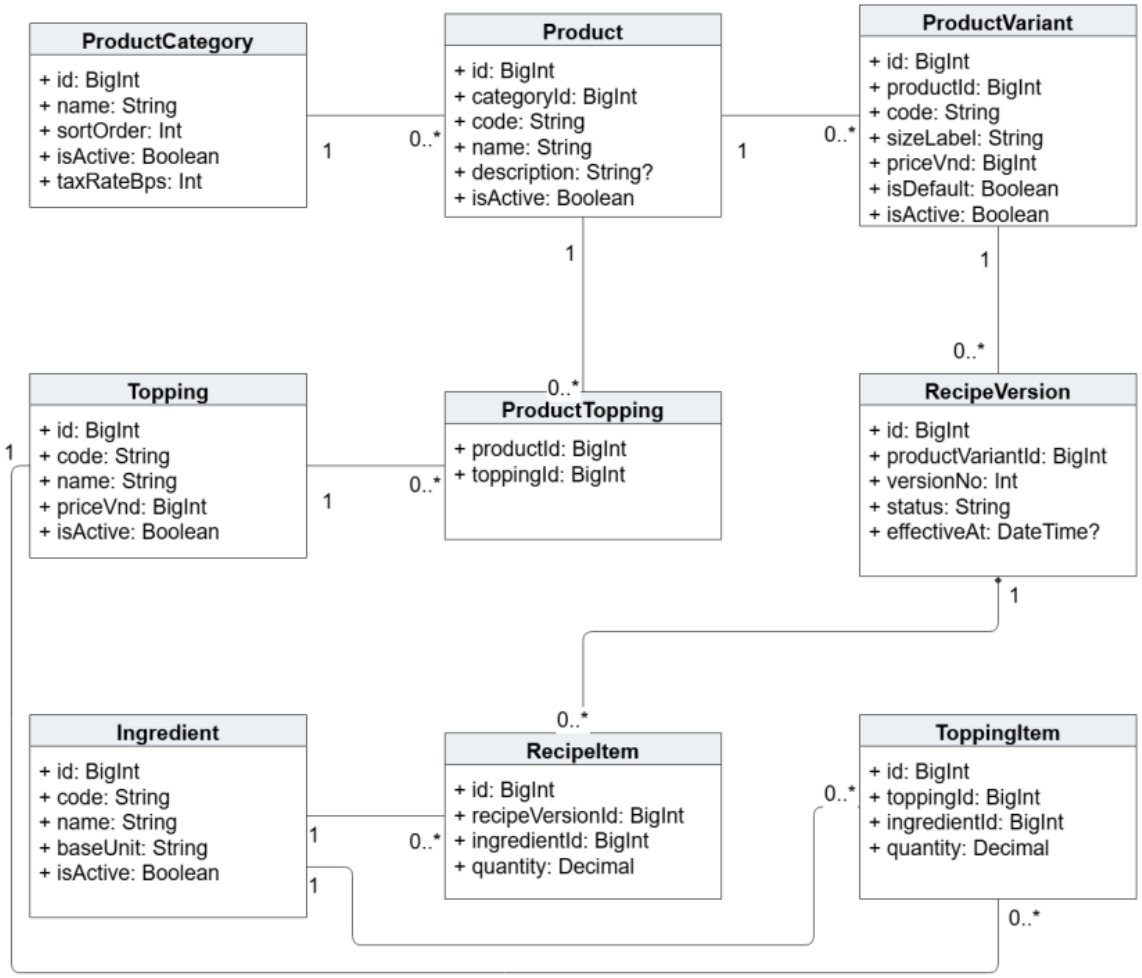
Phụ lục A. Mô hình lớp và dữ liệu bổ sung

Các mô hình dưới đây bổ sung góc nhìn lớp cho thiết kế ở Mục 3. Mô hình miền biểu diễn quan hệ nghiệp vụ; mô hình triển khai chỉ rõ những lớp tham gia ghi dữ liệu hoặc phối hợp thanh toán. Tên trong hình được giữ theo bản vẽ; khi tìm mã nguồn cần đối chiếu package và trách nhiệm thực tế, vì đối tượng điều khiển ở mức phân tích có thể đại diện cho nhiều lớp triển khai.



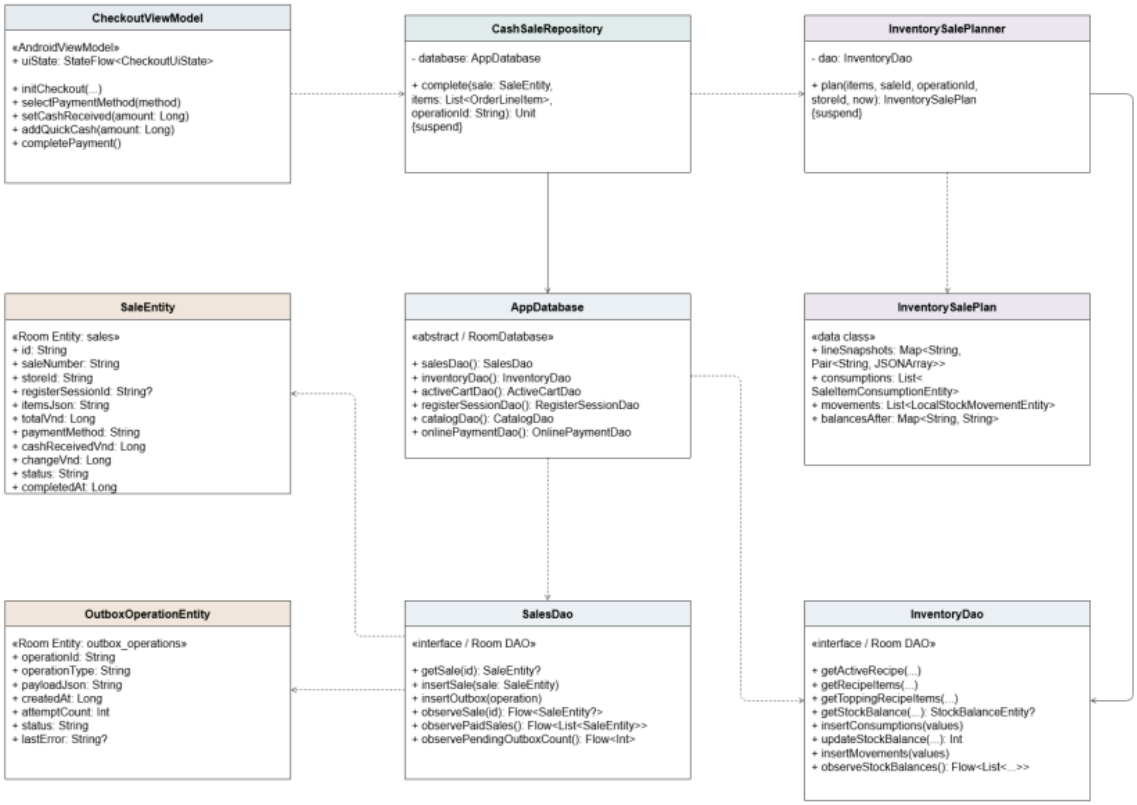
Hình 35. Mô hình lớp miền của bán hàng, thanh toán và phiên quỹ

Trong miền bán hàng, hóa đơn đã trả tiền giữ lịch sử giá, payment và tiêu hao. Phiên quỹ tổng hợp trách nhiệm tiền mặt nhưng không thay thế phiên thu ngân. Tách riêng hai vòng đời này cho phép khóa thao tác bán mà vẫn giữ dữ liệu phục vụ chốt và bàn giao.



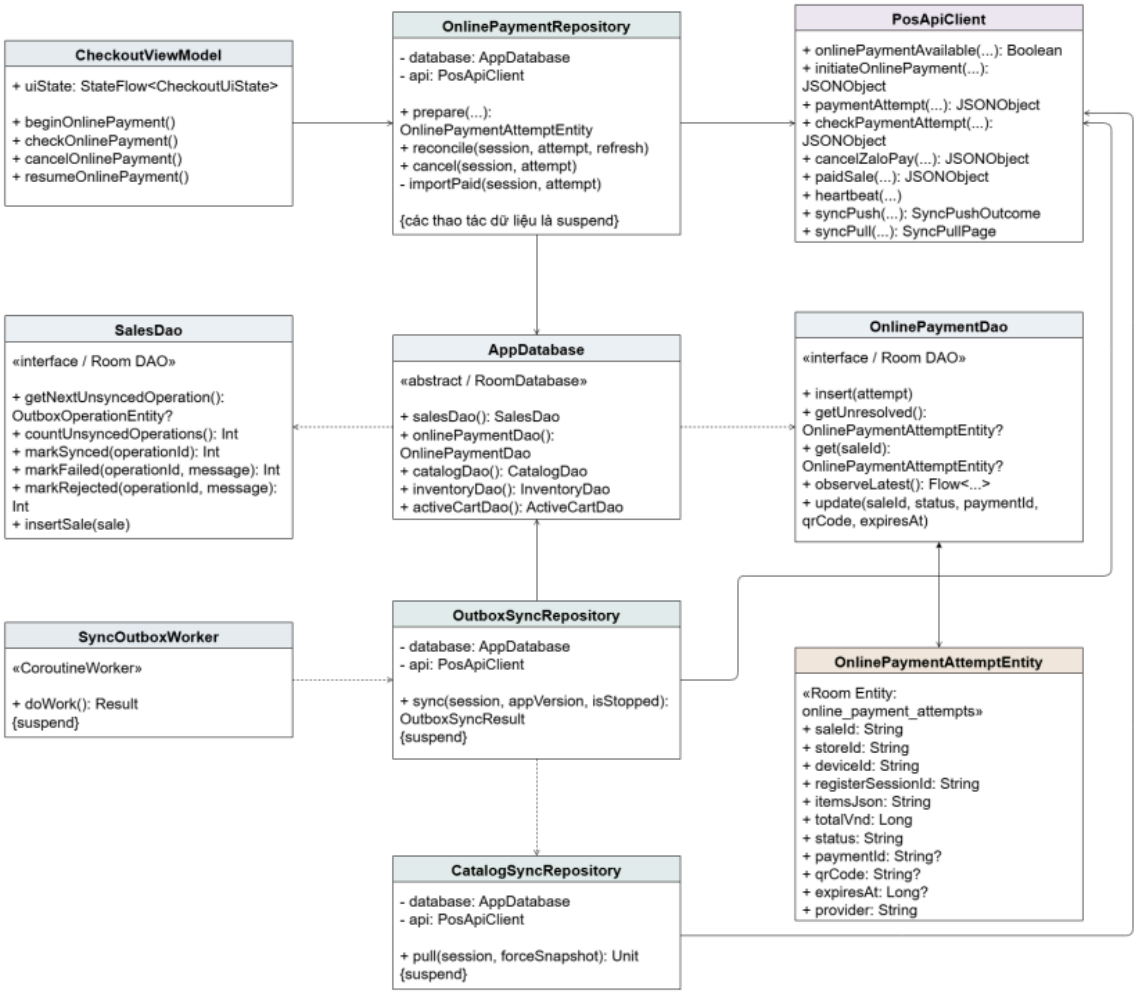
Hình 36. Mô hình lớp miền của sản phẩm, size, topping và công thức nguyên liệu

Các lớp miền thực đơn mô tả lựa chọn hiện hành; snapshot ở hóa đơn giữ lựa chọn đã dùng trong quá khứ. Quan hệ với phiên bản công thức phục vụ truy vết, không được dùng để tính lại một giao dịch cũ theo định mức vừa sửa.



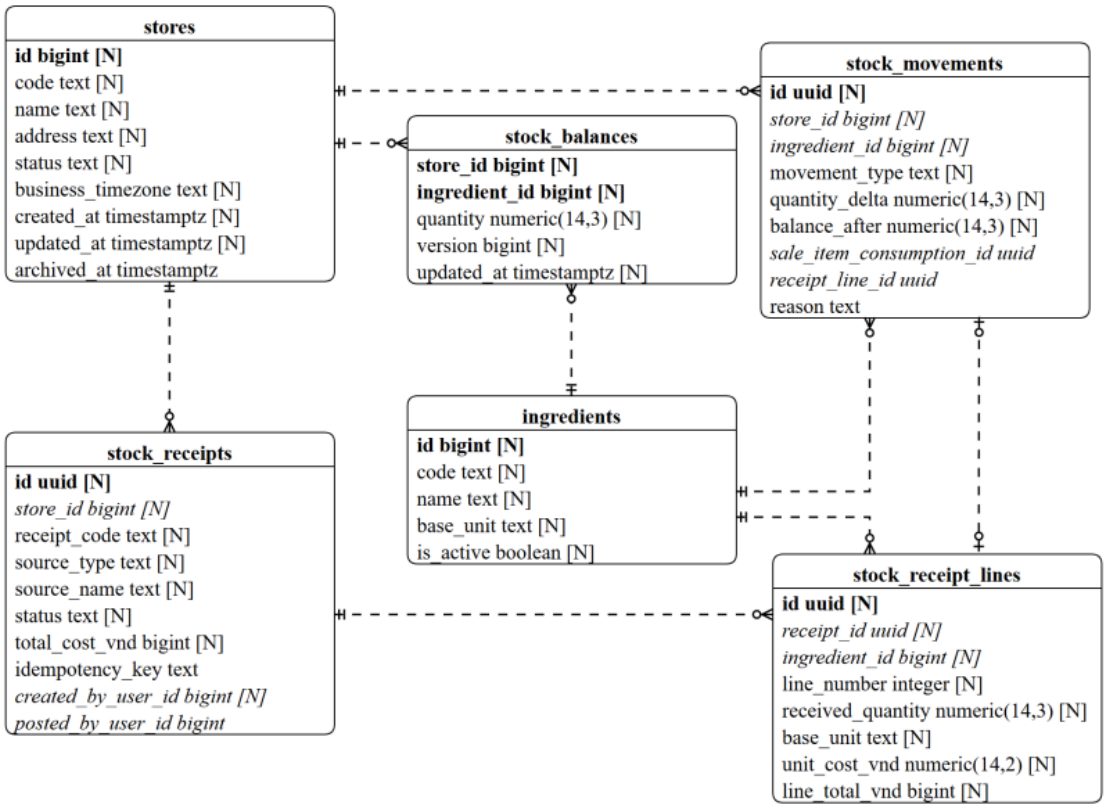
Hình 37. Các lớp triển khai phối hợp hoàn tất bán tiền mặt và ghi tiêu hao nguyên liệu

Luồng bán tiền mặt đi từ ViewModel tới repository và DAO. `CashSaleRepository.complete` xác định phạm vi transaction Room, bao gồm lập kế hoạch tiêu hao và các lệnh ghi hóa đơn, kho, outbox.



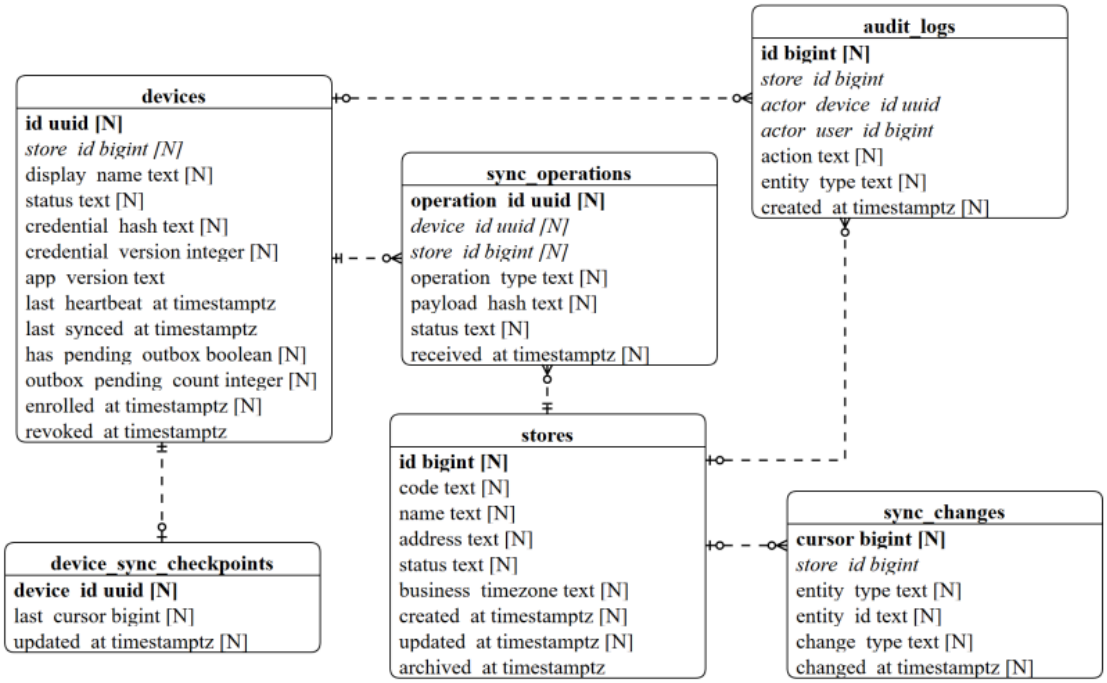
Hình 38. Các lớp triển khai lưu attempt QR, đối soát biên lai và phối hợp đồng bộ

Nhánh QR giữ riêng attempt và các bước đối soát. Nhánh cash lưu operation trong outbox. Hai nhánh có thể hội tụ về dữ liệu hóa đơn và kho, nhưng không được gửi lại một sale QR đã xác nhận như operation bán tiền mặt mới.



Hình 39. Quan hệ dữ liệu của số dư, biến động nguyên liệu và chứng từ nhập kho

Phiếu nhập có vòng đời chứng từ riêng, còn số dư và movement chỉ thay đổi khi đăng thành công. Lưu nháp không làm tăng lượng nguyên liệu mà POS được phép bán. Nhóm đồng bộ và kiểm toán bên dưới giữ khóa tiếp nhận operation và dấu vết thao tác phục vụ việc đối chiếu lại khi lỗi phát sinh.



Hình 40. Quan hệ dữ liệu phục vụ đồng bộ, chống xử lý lặp và lưu dấu vết kiểm toán

Phụ lục B. Danh mục nguồn đối chiếu

Các đường dẫn trong Bảng 22 tính từ thư mục gốc repository và xác định nơi đối chiếu cơ chế mô tả trong báo cáo.

Bảng 22. Nguồn mã và tài liệu dùng đối chiếu nội dung báo cáo

Nội dung	Tệp hoặc thư mục đối chiếu
Phạm vi và quyết định	docs/00-project-charter.md; docs/01-scope.md; docs/02-business-rules.md; docs/adr/
Schema tập trung	apps/api/prisma/schema.prisma
Schema cục bộ	apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/AppDatabase.kt; data/local/entity/Entities.kt
Bán tiền mặt và kho	apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/CashSale Repository.kt; InventorySalePlanner.kt
Push và pull	apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/sync/; apps/api/src/sync/
Thanh toán QR	apps/api/src/payments/; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/OnlinePaymentRepository.kt
Quỹ và bàn giao	apps/api/src/register-sessions/; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/RegisterCashRepository.kt
Giao diện Web	apps/web/src/App.tsx; apps/web/src/services/api.ts; apps/web/src/context/Auth Context.tsx
AWS và phát hành	infra/lib/kaffein-stack.ts; .github/workflows/; scripts/aws-release.mjs
Kết quả thực thi	thesis/assets/evidence/review-20261028.json; docs/handoffs/49-aws-cicd-and-pos-aws.md; docs/handoffs/50-employee-code-rds-correction.md

Ảnh giao diện được lưu ở `assets/screenshots/pos/` và `assets/screenshots/web/`; ảnh cấu hình AWS nằm trong `assets/screenshots/aws/`. Các sơ đồ đã đối chiếu lại nằm trong `assets/diagrams/reviewed-20261028/`, kèm tệp Drawio. Hình 13 dùng ảnh D22 đen trắng gốc trong `assets/diagrams/export/states/`. Bộ hình gốc và các bản vẽ được giữ trong các thư mục nguồn tương ứng. Danh mục bằng chứng đã lọc lưu SHA-256 của các hồ sơ kiểm thử và phát hành dùng ở Mục 5.

Thư mục tư liệu của đồ án gồm tệp gốc có thể chỉnh sửa của sơ đồ, hình sơ đồ đã xuất, ảnh giao diện POS/Web và ảnh chụp cấu hình AWS.

Thư mục tư liệu gốc của đồ án

https://drive.google.com/drive/folders/1wdwwHIi8vouFsOt2kym_0qYmnL6fI6R4?usp=drive_link

Phụ lục C. Danh mục API và hợp đồng trao đổi dữ liệu

Phụ lục ghi nhận các đường dẫn nghiệp vụ đã có trong controller NestJS. Prefix chung là `/api/v1`; riêng health check sử dụng `/health`, còn giao diện Swagger được đặt tại `/api/docs`. Trong bảng, dấu `:id`, `:storeId` và `:saleId` biểu thị tham số đường dẫn. Ký hiệu D chỉ header xác thực thiết bị `Authorization: Device <credential>`; B chỉ `Authorization: Bearer <token>` của Web. Các chuỗi trong ngoặc nhọn góc là tên giá trị cần truyền, không phải credential mẫu.[74]

C.1. Các route nghiệp vụ chính

Các route dưới đây được chọn để đối chiếu các luồng trong báo cáo. Những alias như `sync/operations`, `sync/changes`, `attendance:check-in` và `cashier-sessions:login` cùng tồn tại trong mã nguồn; bảng sử dụng một dạng đường dẫn cho mỗi thao tác. Quyền truy cập được kiểm tra tại controller hoặc service, và Bearer còn đi qua `ActiveWebPrincipalGuard` để đối chiếu trạng thái tài khoản, role và chi nhánh hiện tại với token. Owner xem được phạm vi toàn chuỗi; Store Manager bị giới hạn theo chi nhánh phụ trách ở các nghiệp vụ quản trị tương ứng.[75]

Bảng 23. Các endpoint chính đối chiếu từ controller của phiên bản mã nguồn

Method	Đường dẫn	Xác thực	Mục đích và quy tắc
GET	`/health`	Public	Kiểm tra dịch vụ và kết nối cơ sở dữ liệu.
POST	`/api/v1/auth/web/login`	Public	Đăng nhập bằng email, mật khẩu; trả JWT.
GET	`/api/v1/auth/me`	B	Lấy hồ sơ tài khoản hiện tại.
GET	`/api/v1/auth/stores`	B	Danh sách chi nhánh theo quyền, kèm trạng thái POS.
POST	`/api/v1/device-activations`	B	Owner cấp mã kích hoạt một lần.
POST	`/api/v1/device/enroll`	Mã kích hoạt	Đổi mã hợp lệ lấy định danh và credential thiết bị.
POST	`/api/v1/device/heartbeat`	D	Báo app version, hàng đợi và thời điểm liên lạc.
POST	`/api/v1/devices/:id/revoke`	B	Owner thu hồi thiết bị và phiên thu ngân liên quan.
POST	`/api/v1/attendance/check-in` `/api/v1/attendance/check-out`	D + PIN	Chấm công online; check-in khác home store được đánh dấu hỗ trợ.
POST	`/api/v1/cashier-sessions/login`	D + PIN	Xác thực thu ngân; chỉ đăng nhập tại home store.
POST	`/api/v1/sync/push`	D	Nhận operation; trả kết quả riêng từng operation ID.
GET	`/api/v1/sync/pull`	D	Snapshot khi cursor bằng 0; delta có phân trang.

[74] Nguồn đối chiếu: apps/api/src/main.ts, bootstrap; apps/api/src/health/health.controller.ts, HealthController

[75] Nguồn đối chiếu: apps/api/src/sync/sync.controller.ts; apps/api/src/attendance/attendance.controller.ts; apps/api/src/cashier-sessions/cashier-sessions.controller.ts; apps/api/src/common/security/active-web-principal.guard.ts, canActivate

Method	Đường dẫn	Xác thực	Mục đích và quy tắc
POST	`/api/v1/sales`	D	Tạo sale trực tiếp; nhánh cash ghi nhận trả tiền và trừ kho cùng transaction.
GET	`/api/v1/sales`	B	Danh sách hóa đơn PAID, lọc ngày, store và phương thức.
GET	`/api/v1/sales/:id`	D/B	Chi tiết sale và biên lai theo phạm vi được xác thực.
POST	`/api/v1/payments/online/initiate`	D	Tạo hoặc khôi phục attempt ZaloPay/MoMo với sale ID ổn định.
GET	`/api/v1/payments/online/pending`	D	Khôi phục giao dịch còn chờ của thiết bị.
GET	`/api/v1/payments/online/attempts/:saleId`	D	Đọc trạng thái attempt và đối soát theo lease.
POST	`/api/v1/payments/online/attempts/:saleId/check`	D	Kiểm tra ZaloPay thủ công; không tạo merchant order mới.
POST	`/api/v1/payments/online/attempts/:saleId/cancel`	D	Yêu cầu hủy; quy tắc MoMo và ZaloPay khác nhau.
POST	`/api/v1/webhooks/zalopay` `/api/v1/webhooks/momo`	Chữ ký provider	Nhận callback/IPN; kiểm tra chữ ký và thông tin giao dịch trước khi ghi sổ.
GET	`/api/v1/payments/:id/status`	D/B	Tra cứu bằng payment ID, có kiểm tra phạm vi store.
GET	`/api/v1/payments/online/cancelled-receipts`	B	Quản lý đối soát tiền tới sau hủy nội bộ; không ghi doanh thu.
GET	`/api/v1/register-sessions`	B	Lịch sử quỹ, bảng kê và bộ lọc chênh lệch.
POST	`/api/v1/register-sessions/open`	D	Mở phiên với số tiền và bảng kê mệnh giá.
POST	`/api/v1/register-sessions/:id/close`	D	Đóng phiên, giữ thực đếm gốc và tiền bù riêng.
GET	`/api/v1/stores/:storeId/stock-balances`	D/B	Xem tồn; device chỉ được xem store đã enroll.
POST	`/api/v1/stores/:storeId/stock-adjustments`	B	Điều chỉnh/hao hụt; POS không được gọi với quyền device.
POST	`/api/v1/stores/:storeId/stock-receipts`	B	Tạo phiếu nhập DRAFT; chưa tăng tồn kho.
PATCH	`/api/v1/stock-receipts/:id`	B	Sửa phiếu DRAFT/REVIEWED trong scope.
POST	`/api/v1/stock-receipts/:id/post`	B	Đăng kho; kiểm tra POS sẵn sàng, ghi movement và delta.
POST	`/api/v1/employees/:id/transfer`	B	Chỉ Owner; đổi home store, lưu lịch sử và thu hồi phiên.
PATCH	`/api/v1/product-categories/:id/tax`	B	Chỉ Owner đổi VAT theo danh mục cho sale tương lai.
GET	`/api/v1/reports/dashboard`	B	Tổng hợp số liệu theo ngày nghiệp vụ và scope quản lý.

Nhóm route được đối chiếu trực tiếp trong các controller tương ứng. Cấp credential lần đầu dựa trên mã kích hoạt; callback dùng chữ ký provider thay cho Bearer/Device. Redirect `GET /api/v1/payments/momo/return` chỉ hiển thị trang quay lại, không xác nhận đã

thu tiền. Các route ZaloPay riêng vẫn được giữ để tương thích, bên cạnh nhóm `payments/online`.[76]

C.2. Hợp đồng push, ACK và cursor

Body push chứa mảng `operations`. Mỗi phần tử mang `operationId`, `payload` và tên loại ở `type` hoặc `operationType`; thông tin thời điểm được DTO cho phép ở `occurredAt` hoặc `createdAt`. Server hiện nhận ba nghiệp vụ: `CASH_SALE_COMPLETED`, `REGISTER_SESSION_OPENED` và `REGISTER_SESSION_CLOSED`. Dù DTO liệt kê `STOCK_ADJUSTED`, service chủ động từ chối loại này từ POS. Store và device của thao tác được lấy từ credential đã xác thực.[77]

Phản hồi push chia thành `processed` và `errors`, mỗi phần tử giữ operation ID. `COMPLETED` cho biết nghiệp vụ và ledger đã commit; `ALREADY_PROCESSED` trả kết quả đã lưu khi device và hash payload khớp. Cùng ID nhưng khác device hoặc payload bị từ chối. Mỗi operation có transaction riêng, nên HTTP 200 của batch không biểu thị toàn bộ batch đã thành công. Android đối chiếu kết quả đúng ID trước khi đánh dấu synced, giữ bản ghi bị từ chối và dừng hàng đợi ở thao tác đó. Retry do lỗi mạng phải dùng lại ID cũ.[78]

Pull nhận `cursor` dạng chuỗi số và `limit`, mặc định lần lượt là `0` và `100`; service giới hạn trang delta ở 500 bản ghi. Cursor bằng 0 yêu cầu snapshot đầy đủ trong transaction Repeatable Read. Cursor dương đọc thay đổi có số thứ tự lớn hơn, thuộc dữ liệu chung hoặc store của thiết bị. Phản hồi gồm `changes`, `nextCursor`, `hasMore` và `serverTime`. Android ghi thay đổi cùng checkpoint trong transaction Room; khi còn trang, cursor phải tiến. Checkpoint server cập nhật lúc trả trang không thay thế xác nhận đã áp dụng ở client.[79]

C.3. Định danh và kết quả thanh toán online

Yêu cầu initiate gồm `saleId` UUID ổn định, `registerSessionId`, `expectedTotalVnd`, `items` và `provider` bằng `ZALOPAY` hoặc `MOMO`. Retry cùng attempt dùng lại sale ID và nội dung đã chốt. `paymentId` là định danh thanh toán nội bộ; `merchantOrderId` liên hệ với đơn phía provider. Route attempt nhận sale ID, còn route status nhận payment ID. Phân biệt này tránh truy vấn nhầm hoặc tạo lần thu mới khi phản hồi trước bị mất.[80]

Trường `checkoutState` có giá trị `WAITING` hoặc `RECONCILING` yêu cầu tiếp tục đối soát; đây là trạng thái luồng checkout, khác trường status của bản ghi payment. Timeout, hết countdown hay query lỗi không chứng minh chưa trả tiền. Nhánh kiểm tra ZaloPay thủ công bỏ cache polling; request đồng thời được gộp trong process, và lỗi truy vấn trả 503 mà vẫn giữ attempt. MoMo hủy tại server lưu `CANCELLED`; tiền tới QR cũ được lưu riêng.

---

[76] Nguồn đối chiếu: apps/api/src/auth/auth.controller.ts; devices/devices.controller.ts; sales/sales.controller.ts; register-sessions/register-sessions.controller.ts; inventory/inventory.controller.ts; stock-receipts/stock-receipts.controller.ts; employees/employees.controller.ts; catalog/catalog.controller.ts; reports/reports.controller.ts; payments/payments.controller.ts, trong apps/api/src/

[77] Nguồn đối chiếu: apps/api/src/sync/dto/sync.dto.ts, SyncOperationItemDto; apps/api/src/sync/sync.controller.ts, push; apps/api/src/sync/sync.service.ts, pushOperations

[78] Nguồn đối chiếu: apps/api/src/sync/sync.service.ts, pushOperations; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/network/PosApiClient.kt, syncPush; data/sync/OutboxSyncRepository.kt, sync

[79] Nguồn đối chiếu: apps/api/src/sync/dto/sync.dto.ts, SyncPullQueryDto/SyncPullResponseDto; apps/api/src/sync/sync.service.ts, pullChanges; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/sync/CatalogSyncRepository.kt, pull

[80] Nguồn đối chiếu: apps/api/src/payments/dto/online-payment.dto.ts, InitiateOnlinePaymentDto; payments/dto/zalopay.dto.ts, InitiateZaloPayDto/CreateZaloPayOrderResponseDto; payments/payments.service.ts, initiate/getAttempt

Khi yêu cầu hủy ZaloPay, chỉ mở khóa theo kết quả chưa thanh toán cuối cùng được xác nhận. Chỉ đọc mã HTTP 200 của route cancel sẽ bỏ sót sự khác biệt trạng thái này.[81]

DTO kiểm tra hình dạng yêu cầu qua ValidationPipe với whitelist, chuyển đổi kiểu và từ chối trường thừa đối với DTO khai báo. Các lỗi HTTP được bộ lọc chung đóng gói dưới `application/problem+json`, kèm status, detail và trace ID. Riêng callback có hợp đồng phản hồi với provider: ZaloPay nhận `return_code`/`return_message`; MoMo trả HTTP 204 khi xử lý hoàn tất. Vì vậy client cần đọc hợp đồng của từng nhóm endpoint trước khi quyết định retry hoặc chuyển trạng thái nghiệp vụ.[82]

[81] Nguồn đối chiếu: apps/api/src/payments/payments.controller.ts, checkAttempt/cancelAttempt; payments/payments.service.ts, checkAttempt/getPaymentStatus/cancelAttempt; docs/adr/0010-momo-local-cancellation.md

[82] Nguồn đối chiếu: apps/api/src/main.ts, bootstrap; common/filters/problem-details.filter.ts, catch; payments/payments.controller.ts, handleWebhook/momoIpn

Phụ lục D. Quy trình tái lập và thu thập bằng chứng nghiệm thu

D.1. Điều kiện thực hiện và hồ sơ đầu vào

Phụ lục mô tả quy trình nghiệm thu bổ sung theo AC-01–AC-10 của đề cương. Đây là quy trình dự kiến; trạng thái đã kiểm chứng được trình bày ở Mục 5. Mỗi lượt dùng cơ sở dữ liệu thử nghiệm riêng, ghi commit, phiên bản APK, thiết bị, API đích và thời điểm UTC. Dữ liệu quan sát được trình bày theo múi giờ nghiệp vụ `Asia/Ho_Chi_Minh`.[83]

Trước lượt chạy, lưu baseline gồm store/device, cashier, phiên quỹ, catalog/recipe version, số dư nguyên liệu, cursor, số operation chưa ACK và trạng thái payment. Chuẩn bị hai chi nhánh, nhân viên có home store khác nhau và tài khoản Manager đúng phạm vi. POS phải hoàn tất bootstrap, đăng nhập online và mở quỹ hợp lệ. Mỗi mẫu thử có sale ID và operation ID riêng; hậu kiểm theo các ID này để tránh lẫn dữ liệu lượt khác.

D.2. Cash ngoại tuyến và phục hồi: AC-01, AC-02

Chọn món có định mức đã xác nhận, ghi giá và lượng tồn trước bán. Ngắt riêng đường POS tới API, đồng thời xác nhận máy quan sát vẫn truy cập health của server. Tạo đơn, thu tiền mặt và mở receipt. Hậu kiểm Room phải có sale, consumption, movement và outbox; server chưa được có sale tương ứng. So sánh lượng tồn giảm với tổng định mức đã nhân số phần và topping.

Đóng rồi mở lại ứng dụng khi mạng vẫn bị ngắt; kiểm tra receipt và operation còn tồn tại, giữ nguyên ID/payload. Xác thực đầu phiên hiện cần mạng, nên việc ứng dụng yêu cầu đăng nhập lại không được diễn giải thành mất sale hoặc thành khả năng login offline. Khôi phục mạng, xác thực nếu cần và đồng bộ; kiểm tra ACK, outbox sạch, một sale ở server và số dư Room khớp server. Lưu ảnh trước/sau cùng bản đối chiếu dữ liệu.[84]

D.3. Gửi lặp, transaction và tồn kho: AC-03–AC-06

AC-03 sử dụng harness trong môi trường test: gửi nguyên operation ít nhất mười lần, giữ cùng ID, payload và thiết bị. Ghi digest payload cùng từng phản hồi; hậu kiểm chỉ có một sale, một tập consumption/movement và một lần giảm balance. Bấm Đồng bộ sau khi operation đã SYNCED không chứng minh phép gửi lặp. Thử payload khác dưới cùng ID là ca kiểm tra từ chối riêng, không được tính vào mười lượt.

Với AC-04, bố trí lỗi có kiểm soát tại bước ghi kho trong transaction thử nghiệm. Sau lỗi, đọc lại sale, consumption, movement và balance: không được tồn tại dữ liệu nghiệp vụ ghi dở. Kiểm tra riêng transaction Room và PostgreSQL. Bản ghi operation bị từ chối có thể được giữ để phục vụ đối soát; không nhầm dấu vết này với sale đã thanh toán. Lưu cả fixture, vị trí gây lỗi và assertion rollback.

AC-05 dùng món có một nguyên liệu không đủ từ đầu. Checkout phải bị từ chối, không tăng sale/outbox/movement hoặc làm tồn âm. AC-06 bán theo công thức V1, lưu snapshot, sau đó cập nhật V2 trong fixture và gửi sale cũ. Hậu kiểm consumption và

[83] Nguồn đối chiếu: docs/09-proposal.md, mục 2.8.2, bảng AC-01–AC-10; docs/specs/18-demo-and-monitoring-plan.md, các phần 5–8

[84] Nguồn đối chiếu: apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/CashSaleRepository.kt; data/sync/OutboxSyncRepository.kt; ui/login/LoginViewModel.kt; docs/specs/18-demo-and-monitoring-plan.md, D04–D06 và Restart/migration Room.

recipe version của sale vẫn là V1; phép tính định lượng đối chiếu V1. Không đổi fixture của lượt cash chính khi đang mất mạng.[85]

D.4. Attendance và quyền chi nhánh: AC-07, AC-08

Cho nhân viên B thuộc store B check-in/out ở POS store A. Hậu kiểm attendance ghi đúng nơi hỗ trợ và danh tính B. Sau đó thử cashier login B trên cùng POS: server phải từ chối, không cấp quyền bán, không thay cashier A hoặc mở quỹ mới. Hai kết quả phải được lưu trong cùng bộ dữ liệu để chứng minh attendance không cấp quyền thu ngân.

Với Manager A, thực hiện request đọc và sửa có store đích B, kể cả trường hợp sửa tham số scope. Lưu endpoint, mã phản hồi và dữ liệu trước/sau; request trái quyền phải bị chặn, dữ liệu B không đổi. Ẩn nút trên Web không thay thế phép kiểm tra server. Kết luận phải ghi rõ các endpoint được thử.[86]

D.5. QR trực tuyến và tiền về muộn: AC-09

Ngắt mạng khi chưa tạo QR; ZaloPay/MoMo không được hoàn tất sale. Trong bộ tích hợp, dùng transport giả lập tạo callback sai chữ ký, callback lặp, timeout và query đồng thời. Hậu kiểm chữ ký sai không ghi nhận trả tiền; xác nhận hợp lệ lặp chỉ tạo một sale và một lần trừ kho. Nếu server đã nhận tiền nhưng POS chưa kéo được kho, giữ trạng thái đối soát và chặn cash cho tới khi nhập biên lai cùng snapshot kho thành công.

Ca MoMo bổ sung hủy nội bộ, rồi phát xác nhận nhận tiền muộn trong fixture. Payment phải giữ `CANCELLED`; bằng chứng/audit nhận tiền muộn được ghi đúng một lần, sale/doanh thu/kho không phát sinh lại. Giao dịch mới dùng ID khác. Thử ví Sandbox thật là lượt riêng có mã provider và thời điểm đối chiếu; callback tự gửi chỉ kiểm chứng hợp đồng giả lập, không được ghi thành đã quét ví thành công.[87]

D.6. Kiểm đếm kín và bàn giao quỹ bổ sung

Dùng fixture expected 60.000 đồng, thực đếm 25.000 đồng. Trước xác nhận, màn hình không lộ số theo sổ; sau cố định bảng kê, khởi tạo lại repository và thử sửa bảng kê phải giữ 25.000 đồng. Bù 30.000 đồng chưa đủ thiếu phải bị chặn; bù 50.000 đồng cho phép đóng. Hậu kiểm variance gốc vẫn âm 35.000 đồng, compensation tách riêng và ca nhận có 25.000 + 50.000 = 75.000 đồng, gồm đúng các tờ đã giao. Nhận lặp không mở thêm phiên; tiền bù không thành doanh thu. Thử `END_OF_DAY` riêng để xác nhận ca mới đếm lại.[88]

D.7. Build, AWS smoke và kết quả: AC-10

Lưu báo cáo build POS/API/Web và unit/integration theo đúng commit; bài bỏ qua ghi riêng. Với lượt AWS, liên kết quality jobs, image tags, migration, task revision, rollout

---

[85] Nguồn đối chiếu: apps/api/test/sync-sales-integration.e2e-spec.ts; apps/pos/app/src/main/java/vn/id/congtc145/coffeepos/data/local/InventorySalePlanner.kt; docs/specs/18-demo-and-monitoring-plan.md, các ca phụ Duplicate/lost ACK, REJECTED/rollback và Snapshot lịch sử.

[86] Nguồn đối chiếu: docs/02-business-rules.md; docs/06-auth-attendance.md; apps/api/src/attendance/attendance.service.spec.ts; apps/api/src/cashier-sessions/cashier-sessions.service.spec.ts; docs/handoffs/50-employee-code-rds-correction.md.

[87] Nguồn đối chiếu: apps/api/test/momo-polling.e2e-spec.ts; apps/pos/app/src/androidTest/java/vn/id/congtc145/coffeepos/OnlinePaymentContractTest.kt; docs/adr/0010-momo-local-cancellation.md.

[88] Nguồn đối chiếu: apps/pos/app/src/androidTest/java/vn/id/congtc145/coffeepos/RegisterCashContractTest.kt; apps/api/test/register-compensation.e2e-spec.ts; docs/adr/0011-blind-cash-count-compensation-handover.md.

và health của hai target group. Kiểm tra sau triển khai gồm HTTPS, API health/database, Web deep link, đăng nhập và request trái phạm vi; đối chiếu CloudWatch trong cùng khoảng UTC. Pipeline dùng OIDC, chạy migration trước rollout và đặt Web/API cùng task sau ALB.[89]

Mỗi AC ghi `PASS`, `FAIL` hoặc `NOT RUN`, kèm điều kiện, kỳ vọng, quan sát và đường dẫn bằng chứng. Bản gốc lưu dưới `recovery/private/`; bản đưa vào báo cáo được loại secret, PIN, token và chữ ký. Chỉ ghi đạt khi đã có thao tác cùng dữ liệu hậu kiểm theo tiêu chí tương ứng.

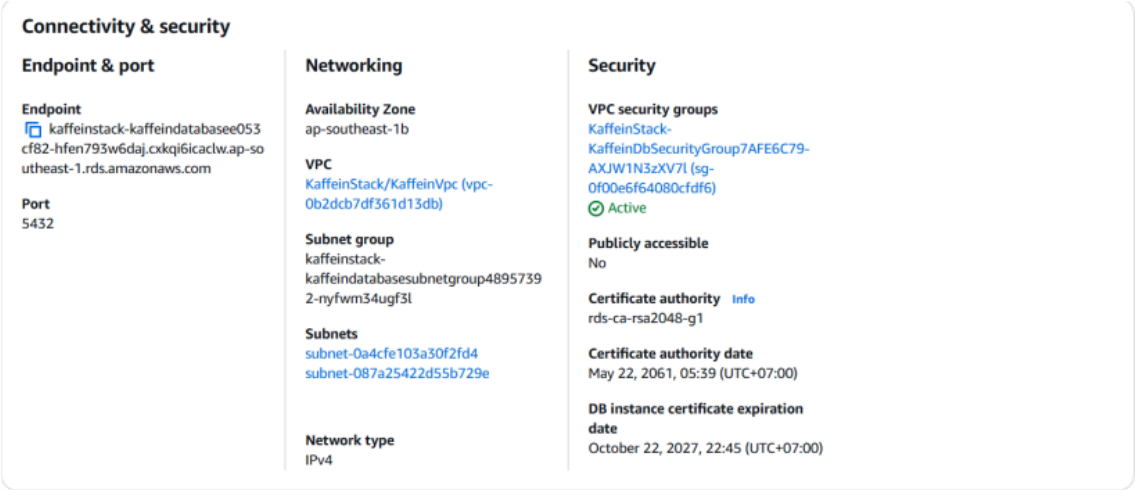
[89] Nguồn đối chiếu: .github/workflows/ci.yml; .github/workflows/deploy.yml; scripts/aws-release.mjs; docs/adr/0013-web-admin-ecs-alb.md

Phụ lục E. Ảnh cấu hình và vận hành AWS

Bộ ảnh được tiếp nhận ngày 28/10/2026, chụp tại region Singapore. Các phần cấu hình được phóng khi dàn trang; PNG gốc và SHA-256 được lưu trong danh mục tư liệu. Phụ lục tập hợp những ảnh chi tiết bổ sung cho mục 4.8.[90]

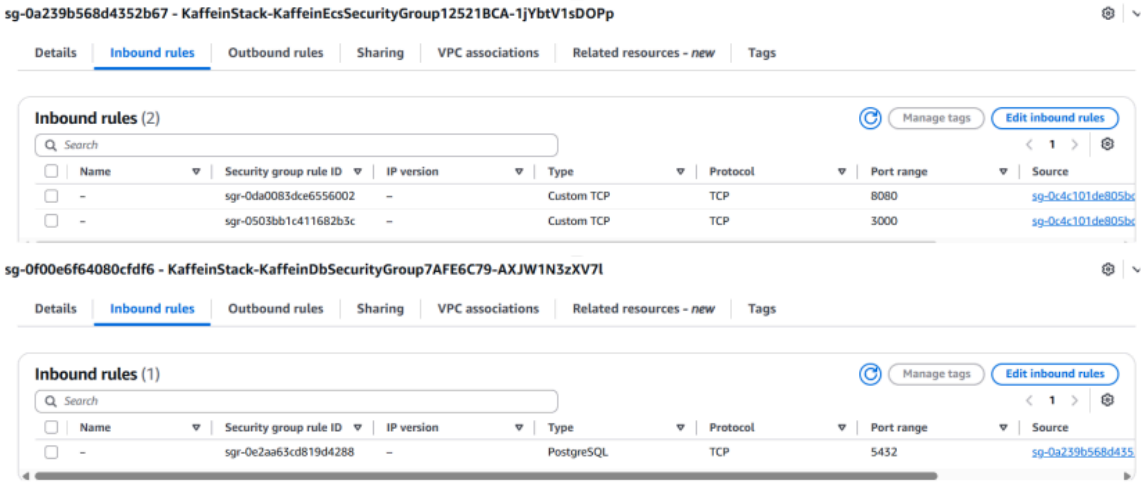


Hình 41. Hai public subnet, hai DatabaseIsolated subnet và tuyến kết nối trong VPC KAFFEIN

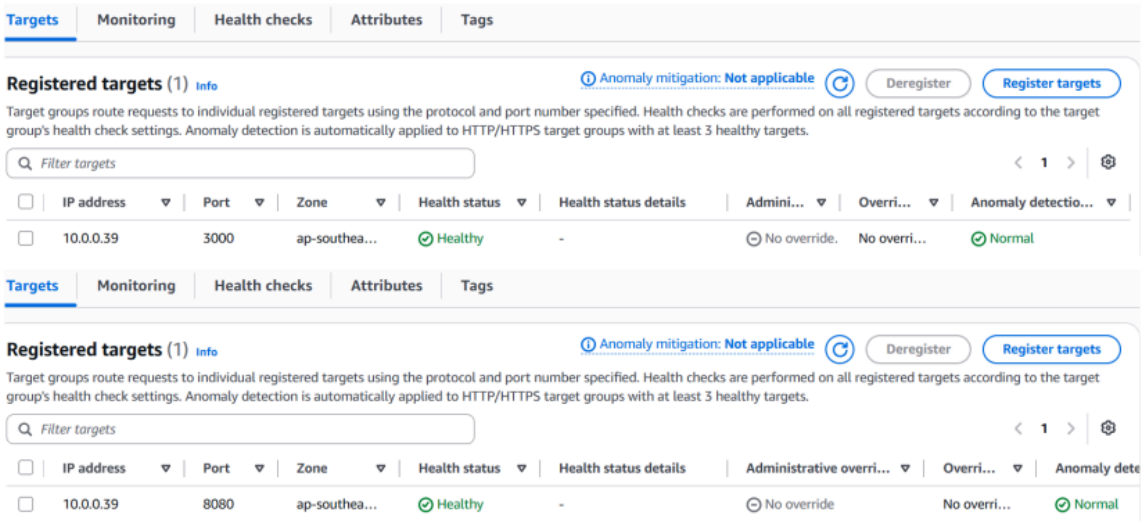


Hình 42. RDS tại ap-southeast-1b, cổng PostgreSQL 5432 và cấu hình không truy cập công khai

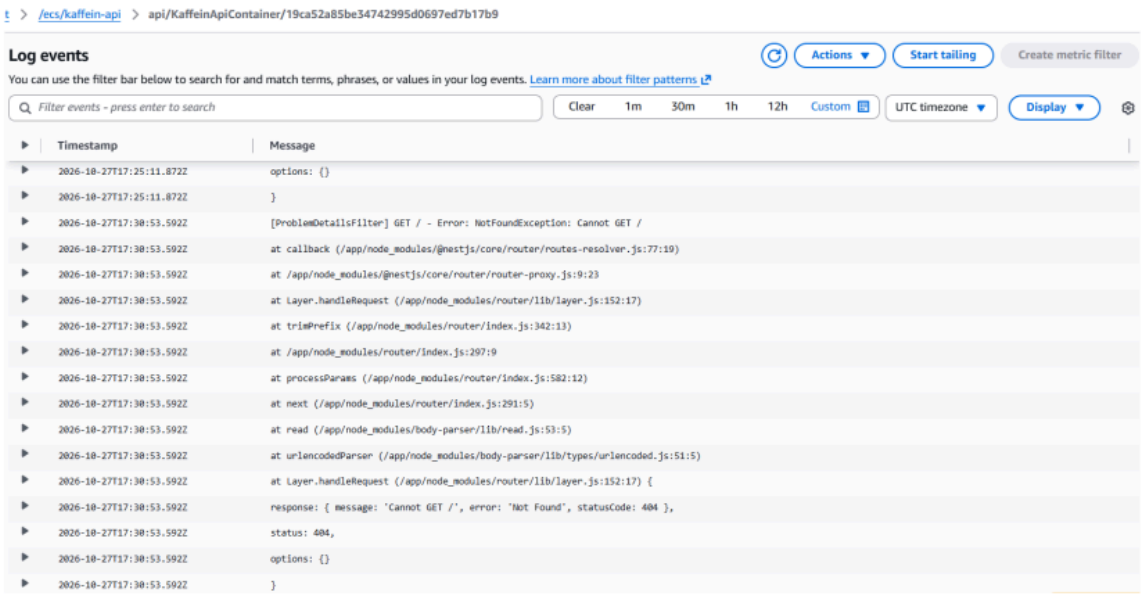
[90]Nguồn đối chiếu: thesis/assets/evidence/aws-screenshots-20261028.json



Hình 43. Inbound rules của ECS cổng 3000/8080 và RDS cổng 5432; nguồn được đối chiếu với CDK



Hình 44. Hai target API cổng 3000 và Web cổng 8080 cùng hiển thị Healthy



Hình 45. Nhật ký API trên CloudWatch ghi nhận request GET / trả HTTP 404, với timestamp UTC

Tài liệu tham khảo

[1] Android Developers, “Build an offline-first app”. Truy cập: 26 Tháng Mười 2026. [Online]. Có tại: https://developer.android.com/topic/architecture/data-layer/offline-first

[2] Android Developers, “Save data in a local database using Room”. Truy cập: 26 Tháng Mười 2026. [Online]. Có tại: https://developer.android.com/training/data-storage/room

[3] SQLite, “Atomic Commit In SQLite”. Truy cập: 26 Tháng Mười 2026. [Online]. Có tại: https://www.sqlite.org/atomiccommit.html

[4] Amazon Web Services, “Transactional outbox pattern”. Truy cập: 26 Tháng Mười 2026. [Online]. Có tại: https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

[5] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media, Inc., 2017.

[6] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley Professional, 2004.

[7] L. Bass, P. Clements, và R. Kazman, *Software Architecture in Practice*, 4th a.b. Addison-Wesley Professional, 2021.

31231027506_TranChiCong_DATN

ORIGINALITY REPORT

1%	1%	1%	%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	text.123docz.net Internet Source	<1%
2	lib.hpu.edu.vn Internet Source	<1%
3	hethongphapluat.com Internet Source	<1%
4	tailieu.vn Internet Source	<1%
5	sdh.hcmute.edu.vn Internet Source	<1%
6	etasr.com Internet Source	<1%
7	www.collegesidekick.com Internet Source	<1%
8	hocvietcode.com Internet Source	<1%
9	digilib.unila.ac.id Internet Source	<1%
10	dulieu.tailieuhoctap.vn Internet Source	<1%
11	services.phaidra.univie.ac.at Internet Source	<1%
12	Banking Academy Publication	<1%
13	codeclimate.com Internet Source	<1%
14	science.lpnu.ua Internet Source	<1%
15	thuvienso.dut.udn.vn Internet Source	<1%

Exclude quotes	On	Exclude matches	< 15 words
Exclude bibliography	On		